pdf4tcllib

Berichte, Tabellen und Formulare mit pdf4tcl

2026-10-26

Inhaltsverzeichnis

1 Was pdf4tcllib ergänzt 9
1.1 Einführung 9
1.2 Womit die Beispiele arbeiten 9
1.3 Die Module 9
1.4 Ein erster Bericht 10
1.5 Wie das Buch zu lesen ist 12
2 Einrichten und Schriften 12
2.1 Einführung 12
2.2 Das Modul laden 12
2.3 Skripte in UTF-8 13
2.4 fonts::init 13
2.4.1 Welche Dateien es sucht 13
2.4.2 Wo es sucht 13
2.4.3 Woher die Schrift kommt 14
2.5 Prüfen, was geladen ist 14
2.6 Schriftnamen immer über die Zugriffsprozeduren 15
2.7 256 Zeichen oder CID 16
2.7.1 Den Modus beim ersten Aufruf wählen 16
2.8 Zusammenfassung 16
3 Unicode sicher ausgeben 17
3.1 Einführung 17
3.2 Was sanitize tut 17
3.2.1 Festbreitenschrift: -mono 1 18
3.3 Wer sanitize aufruft 18
3.3.1 Deutsche Anführungszeichen 20
3.4 Die Schrift beim Zeichnen ist nicht die, nach der sanitize fragt 20
3.5 Emoji 21
3.6 Wissen, was gefehlt hat 21
3.7 Zusammenfassung 22
4 Einheiten und Seitenkontext 22
4.1 Einführung 22
4.2 Einheiten umrechnen 22
4.2.1 pdf4tcl in Punkt lassen 22
4.3 page::context 23
4.3.1 Was im Dict steht 23
4.3.2 top und bottom hängen an der Richtung 25
4.3.3 Zeile für Zeile weiter 25
4.3.4 Die erste Zeile 26
4.4 Kopf, Fuß und Seitenzahl 26
4.5 Hilfslinien beim Entwickeln 27
4.6 Zusammenfassung 27
5 Text setzen 27
5.1 Einführung 27
5.2 Breite messen 28
5.3 Umbrechen 29

5.3.1 Ein Wort, das nicht passt 29
5.4 Kürzen oder verkleinern 29
5.5 Einen Absatz in einem Zug 30
5.6 Text über Seiten 30
5.7 Zusammenfassung 32
6 Kopf- und Fußzeilen, Seitenzahlen 32
6.1 Einführung 32
6.2 Eine Stelle für den Seitenwechsel 32
6.3 Seite 2 von 3 33
6.4 Seitenzahl ohne Wort 35
6.5 Zusammenfassung 35
7 Spaltenfluss und Wasserzeichen 35
7.1 Einführung 35
7.2 Text in Spalten 35
7.2.1 Der Seitenwechsel gehört dem Aufrufer 37
7.2.2 Die erste Grundlinie 37
7.2.3 Was nicht geht 38
7.3 Wasserzeichen 38
7.4 Zusammenfassung 38
8 Tabellen mit Seitenumbruch 39
8.1 Einführung 39
8.2 Spalten und Zeilen 39
8.3 Eine Liste über zwei Seiten 39
8.3.1 Was der Umbruch braucht 41
8.4 Aussehen 41
8.4.1 Einzelne Zeilen und Zellen 42
8.5 Eine Zeile, ein Wert: kürzen 42
8.6 Verbundene Zellen 42
8.7 Wie breit wird die Tabelle? 43
8.8 Was man wissen sollte 43
8.9 Zusammenfassung 43
9 Gruppensummen und Kreuztabellen 44
9.1 Einführung 44
9.2 Das Beispiel 44
9.3 table::group 46
9.3.1 Die Daten müssen sortiert sein 47
9.3.2 Leere Felder sind keine Null 47
9.3.3 Die Beschriftung reicht über mehrere Spalten 47
9.3.4 Summen formatieren 47
9.4 table::pivot 47
9.5 Zusammenfassung 48
10 Diagramme 48
10.1 Einführung 48
10.2 Daten und Kasten 48
10.3 Drei Diagramme auf einer Seite 49
10.3.1 Balken 50
10.3.2 Linien 50

10.3.3 Torte 50
10.4 Optionen 50
10.4.1 Die Skala 51
10.4.2 Zahlen mit Komma 51
10.5 Barrierefrei 51
10.6 Zusammenfassung 51
11 Inhaltsverzeichnis 51
11.1 Einführung 51
11.2 Der Inhalt als Skript 52
11.2.1 Was das Skript nicht darf 53
11.2.2 Welcher Kontext gilt 53
11.3 Überschriften 54
11.4 Optionen von toc::document 54
11.5 Ein Deckblatt davor 55
11.6 Zusammenfassung 56
12 Formular-Layout 56
12.1 Einführung 56
12.2 Die Bausteine 56
12.3 Ein Bestellformular 57
12.3.1 labelField 58
12.3.2 row 59
12.3.3 orderTable und sumLine 59
12.4 Schriften in Formularen 59
12.5 Rahmen und Hintergrund 60
12.6 Einstellungen 60
12.7 Was man wissen sollte 61
12.8 Zusammenfassung 61
13 Etikettenbögen 61
13.1 Einführung 61
13.2 Die Bögen 62
13.3 Adressen auf zwei Bögen 62
13.3.1 render 63
13.3.2 Umrisse zum Prüfen 64
13.4 Text, der zu lang ist 64
13.4.1 -clip als Sicherung 65
13.5 Den Drucker vermessen 65
13.6 Zusammenfassung 65
14 Bilder 65
14.1 Einführung 65
14.2 Laden 65
14.3 Ein Bild in einen Rahmen 66
14.4 Das Beispiel 66
14.4.1 Auflösung 68
14.5 Galerie 68
14.6 Bilder aus Tk 69
14.7 Zusammenfassung 69
15 Pfeile und Callouts 69

15.1 Einführung 69
15.2 Die Prozeduren 69
15.3 Ein Ablaufschema 70
15.3.1 Gerade Pfeile 72
15.3.2 Geknickte Pfeile 72
15.4 Sprechblasen 72
15.5 Runde Ecken 73
15.6 Barrierefrei 73
15.7 Zusammenfassung 73
16 Formeln 73
16.1 Einführung 73
16.2 Die Schrift 74
16.3 Die LaTeX-Teilmenge 74
16.3.1 Leerzeichen zählen nicht 74
16.3.2 Das Minuszeichen 74
16.4 Im Satz und abgesetzt 75
16.4.1 Im Satz 76
16.4.2 Abgesetzt 76
16.5 Die eqn-Schreibweise 76
16.6 Zusammenfassung 77
17 Treeview 77
17.1 Einführung 77
17.2 Tabelle oder Gliederung 77
17.3 Ein Lagerbestand 78
17.3.1 Das Skript braucht ein exit 79
17.4 render 80
17.4.1 Seitenumbruch 80
17.5 outline 80
17.5.1 Lesezeichen 81
17.6 Beides in einem Dokument 81
17.7 Zusammenfassung 81
18 Barrierefreie PDFs 81
18.1 Einführung 81
18.2 Was ein Dokument für PDF/UA braucht 82
18.3 Ein Bericht mit Strukturbaum 82
18.4 Was die Bausteine selbst tun 84
18.4.1 Überschriften 84
18.4.2 Diagramme 85
18.4.3 Tabellen 85
18.5 Selbst gezeichnetes einordnen 85
18.5.1 getUntaggedCount 85
18.6 Prüfen 86
18.7 Zusammenfassung 86
19 Grafikzustand sichern 86
19.1 Einführung 86
19.2 Was verloren geht 86
19.3 state::with 87

19.3.1 Von Hand 88
19.4 Warum nicht gsave und grestore 88
19.5 Zusammenfassung 88
20 Messen und stapeln 88
20.1 Einführung 88
20.2 Messen durch Zeichnen 89
20.2.1 pages und overflow 89
20.3 Das Beispiel 89
20.4 layout::stack 91
20.4.1 Was keep tut und was nicht 92
20.4.2 Die Kopfzeile und der obere Rand 92
20.5 Zusammenfassung 93
21 Fallstudie: Lagerbericht 93
21.1 Einführung 93
21.2 Das Skript 93
21.3 1. Die Daten zuerst 96
21.4 2. Der Seitenwechsel 97
21.5 3. Der Inhalt 97
21.6 4. Das Dokument 97
21.7 Was bewusst fehlt 98
21.8 Zusammenfassung 98
22 Fallen 98
22.1 Einrichten 98
22.2 Text und Zeichen 99
22.3 Text setzen 99
22.4 Seiten und Spalten 99
22.5 Tabellen und Diagramme 99
22.6 Dokumente 100
22.7 Barrierefreiheit und Aufbau 100
22.8 Seite 101
23 API je Modul 101
23.1 Pakete 101
23.2 Schriften: fonts 102
23.3 Zeichen: unicode 102
23.4 Text setzen: text 102
23.5 Formeln: math 103
23.6 Seite: page, units 103
23.7 Strukturbaum: tag 103
23.8 Zustand, Messen, Stapeln: state, measure, layout 104
23.9 Tabellen: table 104
23.10 Zeichnen: drawing 105
23.11 Bilder: image 105
23.12 Formular-Layout: form 106
23.13 Diagramme: chart (pdf4tclchart) 106
23.14 Spaltenfluss: flow (pdf4tclflow) 106
23.15 Inhaltsverzeichnis: toc (pdf4tcltoc) 107
23.16 Etiketten: labels (pdf4tcllabels) 107

23.17 Ausfüllbare Formulare: forms (pdf4tclforms) 107
23.18 Tk-Widgets: canvas, tablelist, textwidget, treeview 108
23.19 Sonstiges 108
24 Glossar 109
25 Fehlermeldungen 109
25.1 Für alle Bausteine 109
25.1.1 ...: unknown option -... (allowed: ...) / (known: ...) 110
25.1.2 ...: expected an even number of option/value words 110
25.1.3 ...: expected option/value pairs 110
25.2 Seite und Schriften 110
25.2.1 unknown paper size: ... (known: ...) 110
25.2.2 pdf4tcllib::page::footer: unknown option ... 110
25.2.3 pdf4tcllib::fonts: No TTF fonts found, falling back to Helvetica 110
25.2.4 pdf4tcllib::fonts: TTF error: ... / Fallback on Helvetica 110
25.2.5 pdf4tcllib::unicode: sanitize: at least one character is missing from the font and was replaced with '?' (first: U+...). ... 110
25.2.6 pdf4tcllib::math: unknown symbol '\...' rendered as literal text 110
25.3 Text, Messen, Stapeln 110
25.3.1 text::fitSize: unknown option ... 110
25.3.2 layout::stack: an entry needs a script (or must be {gap }) 110
25.3.3 measure::block: unknown option ... 111
25.4 Tabellen 111
25.4.1 table::draw: -orient 0 is not supported. ... 111
25.4.2 table::draw: rows ... to ... are joined by rowspan and need ... pt, more than a page holds (... pt) 111
25.4.3 table::group: -by ... is outside the row (0.....) 111
25.4.4 table::group: the rows are not sorted by column ...; a group falls into several blocks and each gets its own subtotal 111
25.4.5 table::pivot: -agg must be sum, count, avg, min or max, not "..." 111
25.4.6 orderTable: column widths total ... pt, the text width is only ... pt -- the table would be drawn past the edge of the page 111
25.4.7 orderTable: ... header(s) but only ... column width(s) 111
25.4.8 tablelist::render: ... could not be reproduced (...). ... 111
25.5 Diagramme 111
25.5.1 chart::...: no data 111
25.5.2 chart::...: data must be label/value pairs or a list of {label value}, got ... element(s) 111
25.5.3 chart::bar: value for "..." is not a number: "..." 111
25.5.4 chart::bar: box too small -- ... x ... pt leaves no room for axis and labels 112
25.5.5 chart::pie: negative value for "..." -- a slice of a whole cannot be negative 112
25.5.6 chart::pie: the values add up to 0 112
25.6 Spaltenfluss und Verzeichnis 112
25.6.1 flow::...: -columns must be at least 1 112
25.6.2 flow::boxes: ... columns with a gap of ... pt leave no width in ... pt 112
25.6.3 toc::heading: level must be 1..6, got "..." 112
25.6.4 toc::document: the contents was calculated as ... page(s) and written as ... -- every page number in it would be off by ... 112
25.7 Bilder und Zeichnen 112

25.7.1 image::place: -fit must be contain, cover or stretch, not "..." 112
25.7.2 image::place: width and height must be positive 112
25.7.3 image::place: image ... has no size 112
25.7.4 image::place: ... at ... dpi (...x... pixels on ...x... points), below ... 112
25.7.5 image::gallery: -cols and -rows must be at least 1 112
25.7.6 image::gallery: the frame is too small for ...x... cells 112
25.7.7 pdf4tcllib::image needs Tk: ... 112
25.7.8 drawing::arrow: -at must be end, start or both, not "..." 112
25.7.9 drawing::callout: width and height must be positive 113
25.7.10 drawing::watermark: no text 113
25.7.11 canvas::place: no such widget: ... 113
25.7.12 canvas::place: no room left for the drawing after the caption 113
25.7.13 canvas::fitPage: the context leaves no height 113
25.8 Etiketten 113
25.8.1 unknown label sheet "..."; known: ... 113
25.8.2 label format ...: missing "..." 113
25.8.3 label format ...: paper "roll" means one label per page, so cols and rows must both be 1 113
25.8.4 argSpec must name five variables: {x y w h record} 113
25.8.5 -start must be between 0 and ... for sheet ... 113
25.8.6 -only: "..." is not a position on sheet ... (0.....) 113
25.8.7 -only names ... position(s) for ... record(s) 113
25.8.8 -frame: 0, 1 or layer, not "..." 113
25.8.9 -frame layer needs pdf4tcl 0.9.4.70 or newer: ... ignores -print 0, the grid would print 113
25.8.10 -frame layer under -pdfa ...: PDF/A forbids /AS, so -print 0 has no effect and the grid would print; use -frame 1 on a separate proof sheet 113
25.9 Ausfüllbare Formulare 113
25.9.1 forms::read: cannot read "..." / forms::write: cannot read "..." 113
25.9.2 forms::write: ... has no field(s): ... 113
25.9.3 forms::write: nothing to fill in 114
25.9.4 pdf4tclforms::renderSchema: spec needs 'sections' key 114
25.9.5 pdf4tclforms::renderSchema: 'sections' must be a dict (name -> section), not a list of sections 114
25.9.6 pdf4tclforms: field entry must be a dict, got: ... 114
25.9.7 pdf4tclforms::template: unknown template '...' (callnote inventory checklist order) 114
25.10 Tk-Widgets 114
25.10.1 treeview::render: no such widget: ... / treeview::outline: no such widget: ... 114
25.11 Aus pdf4tcl 114
25.11.1 tagged: the ... element is empty -- no content, no child element and no annotation. ... 114
25.11.2 ... must contain at least one ... 114
26 Lizenz 114

1 Was pdf4tcllib ergänzt

1.1 Einführung

pdf4tcl zeichnet: Text an eine Stelle, eine Linie von hier nach dort, ein Bild in ein Rechteck. Wer damit einen Bericht baut, schreibt bald dieselben Hilfsprozeduren wie alle anderen: Seitenränder ausrechnen, Text umbrechen, eine Tabelle über die Seitengrenze retten, Kopf- und Fußzeile setzen, Sonderzeichen abfangen.

pdf4tcllib ist diese Schicht als Bibliothek. Es zeichnet nicht selbst, sondern ruft dieselben Methoden des pdf4tcl-Objekts auf, die auch der eigene Code benutzt. Beides lässt sich daher mischen: eine Tabelle aus pdf4tcllib, darunter eine Linie direkt mit pdf4tcl.

Dieses Buch ist ein Werkstattbuch. Jedes Kapitel löst eine Aufgabe, die in Berichten, Formularen und Listen immer wieder vorkommt, mit einem Beispiel, das so läuft, wie es abgedruckt ist. Wie pdf4tcl selbst arbeitet (Koordinaten, Schriften, Formulare, Verschlüsselung), erklärt das Buch *pdf4tcl*; hier steht es nur, wo es für die Aufgabe nötig ist.

1.2 Womit die Beispiele arbeiten

Teil	Version
pdf4tcllib	0.6.7
pdf4tcl	0.9.4.73 (pdf4tcllib verlangt mindestens 0.9.4.23)
Tcl	8.6 und 9.0
Schrift	DejaVu Sans Condensed (TrueType)

Alle Beispiele liegen als Skript im Ordner `beispiele/` neben dem Buch, mit der Kapitelnummer im Namen. Die Ausgaben im Text stammen aus diesen Skripten.

1.3 Die Module

pdf4tcllib ist eine Datei, `pdf4tcllib-0.6.7.tm`, mit mehreren Namensräumen. Daneben liegen Pakete, die darauf aufbauen und einzeln geladen werden.

Namensraum	Aufgabe	Kapitel
fonts	TrueType-Schrift suchen und laden, Schriftnamen liefern	Schriften
unicode	Zeichen, die die Schrift nicht hat, ersetzen	Unicode
units, page	Einheiten, Satzspiegel, Kopf-und Fußzeile	Seitenkontext
text	Umbruch, Breite messen, Kürzen, Hoch-und Tiefstellen	Text setzen

Namensraum	Aufgabe	Kapitel
table	Tabellen mit Seitenumbruch, Gruppen, Kreuztabellen	Tabellen
drawing, image	Formen, Verläufe, Pfeile, Bilder	Bilder, Pfeile
form	Beschriftete Formularfelder in Zeilen	Formular-Layout
math	Formeln im Text	Formeln
tag, state, measure, layout	Barrierefreiheit, Grafikzustand, Höhe vorab messen, Blöcke stapeln	Barrierefrei, Zustand, Stapeln

Paket	Aufgabe	Kapitel
pdf4tcltoc	Inhaltsverzeichnis mit echten Seitenzahlen	Inhaltsverzeichnis
pdf4tclchart	Balken-, Linien- und Tortendiagramme	Diagramme
pdf4tclflow	Text über Spalten und Seiten	Spaltenfluss
pdf4tcllabels	Etikettenbögen	Etikettenbögen
pdf4tcltreeview	ein ttk::treeview als Tabelle oder Gliederung	Treeview
pdf4tclforms	ausfüllbare Formulare aus einer Beschreibung	nur Anhang B
pdf4tcltable	ein tablelist-Widget als PDF	nur Anhang B
pdf4tcltext	ein Tk-text-Widget als PDF	nur Anhang B
pdf4tclcanvas	eine Tk-canvas-Zeichnung als PDF	nur Anhang B

Die vier letzten haben im Buch kein eigenes Kapitel; ihre Aufrufe stehen in Anhang B, ihre Meldungen in Anhang D.

1.4 Ein erster Bericht

Das kleinste sinnvolle Beispiel: Kopfzeile, Überschrift, ein umbrochener Absatz, Fußzeile mit Seitenzahl.

```
# 010-erstes-dokument.tcl -- ein Bericht in wenigen Zeilen
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init
set ctx [pdf4tcllib::page::context a4 -margin 20]

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
```

```
pdf4tcllib::page::header $pdf $ctx "Monatsbericht"

set x [dict get $ctx left]
set y [expr {[dict get $ctx top] + 20}]
set w [dict get $ctx text_w]

$pdf setFont 16 [pdf4tcllib::fonts::fontSansBold]
pdf4tcllib::unicode::safeText $pdf "Umsatz im Oktober" -x $x -y $y
set y [expr {$y + 28}]

set absatz "Der Umsatz stieg gegenüber dem Vormonat um 4 %. Am\
    stärksten wuchs der Bereich Wartung, der jetzt ein Drittel des\
    Umsatzes ausmacht. Die Zahlen sind vorläufig; die Schlussrechnung\
    folgt mit dem Quartal."
set y [pdf4tcllib::text::writeParagraph $pdf $absatz $x $y $w 11]
puts "nächste Zeile bei y = $y"

pdf4tcllib::page::footer $pdf $ctx "Vertraulich" 1
$pdf endPage
$pdf write -file 010-erstes-dokument.pdf
$pdf destroy
```

Ausgabe:

```
pdf4tcllib::fonts: TTF italic faces loaded
pdf4tcllib::fonts: TTF faces loaded from
    /usr/share/fonts/truetype/dejavu
nächste Zeile bei y = 152.69291338582678
```

Die zwei ersten Zeilen schreibt `fonts::init` nach stderr. Sie sagen, dass eine TrueType-Schrift gefunden wurde und woher sie kommt.

Was die Zeilen tun:

- `fonts::init` sucht einmal je Prozess eine TrueType-Schrift und meldet sie bei pdf4tcl an. Danach liefern `fonts::fontSans`, `fonts::fontSansBold` usw. die Namen, unter denen sie angemeldet ist.
- `page::context` rechnet aus Papier und Rand den Satzspiegel aus und gibt ein Dict zurück: `left`, `top`, `text_w` und weitere Werte, alle in Punkt.
- `page::header` und `page::footer` setzen Kopf- und Fußzeile mit einer dünnen Linie, beide innerhalb des Rands.
- `unicode::safeText` schreibt Text wie `$pdf text`, ersetzt aber vorher Zeichen, die die Schrift nicht hat.
- `text::writeParagraph` bricht den Absatz in der Breite `w` um und gibt die y-Position nach der letzten Zeile zurück. Dort geht es weiter.

`-orient true` bei `pdf4tcl::new` gehört dazu: y wächst nach unten, wie im Tk-Canvas. `page::context` geht ohne weitere Angabe ebenfalls davon aus. Wie beide zusammenpassen müssen, steht im Kapitel Einheiten und Seitenkontext.

1.5 Wie das Buch zu lesen ist

- **Teil 1** (dieses Kapitel bis Kapitel 4) richtet ein, was alle anderen Kapitel voraussetzen: Schrift, Unicode, Seitenkontext.
- **Teile 2 bis 4** sind Aufgaben: Text, Tabellen, Diagramme, Inhaltsverzeichnis, Etiketten, Formulare. Sie lassen sich einzeln lesen.
- **Teil 5** macht ein Dokument barrierefrei und zeigt, wie man Blöcke vorab misst und untereinander setzt.
- Die **Fallstudie** baut einen Bericht von den Daten bis zum Inhaltsverzeichnis.
- Die **Anhänge** sammeln Fallen, die API je Modul und die Meldungen.

Getestet mit pdf4tcllib 0.6.7, pdf4tcl 0.9.4.73, Tcl 8.6.18 und 9.0.4, Debian mit DejaVu-Schriften.

Nächstes Kapitel: Einrichten und Schriften

2 Einrichten und Schriften

2.1 Einführung

pdf4tcllib braucht zwei Dinge: das Paket pdf4tcl und eine TrueType-Schrift. Ohne TrueType arbeitet es mit Helvetica weiter, dann fehlen aber alle Zeichen außerhalb von Latin-1, und das PDF lässt sich nicht als PDF/A ablegen. Dieses Kapitel richtet beides ein und zeigt, wie man prüft, was `fonts::init` gefunden hat.

2.2 Das Modul laden

pdf4tcllib ist ein Tcl-Modul: eine Datei `pdf4tcllib-0.6.7.tm` im Ordner `lib/`, daneben die Zusatzpakete (`pdf4tcltoc-0.1.tm` usw.). Tcl findet sie über den Modulpfad:

```
tcl::tm::path add /pfad/zu/pdf4tcllib/lib
package require pdf4tcllib
```

Oder ohne Änderung am Skript über die Umgebung, je nach Tcl-Version:

```
export TCL8_6_TM_PATH=/pfad/zu/pdf4tcllib/lib
export TCL9_0_TM_PATH=/pfad/zu/pdf4tcllib/lib
```

pdf4tcl selbst ist ein klassisches Paket mit `pkgIndex.tcl`; sein Ordner gehört in `auto_path` oder in `TCLLIBPATH`. `package require pdf4tcllib` lädt pdf4tcl nicht mit. Beide Zeilen stehen darum am Anfang jedes Beispiels:

```
package require pdf4tcl
package require pdf4tcllib
```

Wer pdf4tcllib mit einer Anwendung ausliefert, kopiert die gebrauchten `.tm`-Dateien in einen eigenen Modulordner der Anwendung und fügt ihn mit `tcl::tm::path add` hinzu. Mehr braucht es nicht; pdf4tcllib hat außer pdf4tcl keine Abhängigkeiten. Nur der Namensraum `image` braucht Tk.

2.3 Skripte in UTF-8

Die Beispiele enthalten Umlaute im Quelltext. Tcl 9 liest Skripte als UTF-8. Tcl 8.6 liest sie in der Systemkodierung, und die ist ohne gesetzte Locale (`LANG` leer, etwa in einem Cron-Job oder Container) ISO 8859-1. Dann wird aus "gegenüber" im PDF "gegenÃ¼ber", ohne Fehlermeldung.

Abhilfe, eins von dreien:

- die Locale setzen (`LANG=de_DE.UTF-8` oder `C.UTF-8`),
- das Skript mit `source -encoding utf-8 skript.tcl` laden,
- Zeichen außerhalb von ASCII als `\u`-Folge schreiben (`gegen\u00fcber`).

Prüfen lässt sich die Kodierung mit `encoding system`.

2.4 fonts::init

```
pdf4tcllib::fonts::init ?-fontdir dir? ?-family name? \
    ?-cid 0|1? ?-force 0|1?
```

`fonts::init` sucht eine TrueType-Familie, meldet ihre Schnitte bei pdf4tcl an und merkt sich die Namen. Es wirkt einmal je Prozess; jeder weitere Aufruf kehrt sofort zurück.

Option	Vorgabe	Wirkung
-family	DejaVuSansCondensed	Name der Familie, so wie er in den Dateinamen steht
-fontdir	–	Ordner, der vor allen anderen durchsucht wird
-cid	0	1: volle Schrift statt 256 Zeichen (Abschnitt 256 Zeichen oder CID)
-force	0	1: erneut suchen, auch wenn schon geladen

2.4.1 Welche Dateien es sucht

Eine Familie gilt als gefunden, wenn in einem Ordner **beide** Dateien liegen:

```
<familie>.ttf
<familie>-Bold.ttf
```

Aus demselben Ordner nimmt `fonts::init` dazu, wenn vorhanden:

- `<familie>-Oblique.ttf` und `<familie>-BoldOblique.ttf` für kursiv,
- `<familie>Mono.ttf`, sonst `DejaVuSansMono.ttf`, sonst `LiberationMono-Regular.ttf` für die Festbreitenschrift.

Die Namen folgen dem Schema von DejaVu. Liberation heißt `LiberationSans-Regular.ttf` und `LiberationSans-Italic.ttf`; das passt nicht, auch nicht mit `-family LiberationSans`.

2.4.2 Wo es sucht

In dieser Reihenfolge, der erste Ordner mit beiden Dateien gewinnt:

1. der Ordner aus `-fontdir`,
2. `fonts/` neben `lib/` und `lib/fonts/`, also beim Modul selbst,
3. unter Linux `/usr/share/fonts/truetype/dejavu`,

`/usr/share/fonts/dejavu-sans-fonts` (Fedora), `/usr/share/fonts/TTF` (Arch), `/usr/share/fonts/truetype`, `$HOME/.fonts`, `$HOME/.local/share/fonts`,
4. unter Windows `%SystemRoot%\Fonts` und `%LOCALAPPDATA%\Microsoft\Windows\Fonts`,
5. unter macOS `/Library/Fonts`, `/System/Library/Fonts`, `$HOME/Library/Fonts`.

Unterordner durchsucht es nicht.

2.4.3 Woher die Schrift kommt

System	Was zu tun ist
Debian, Ubuntu	apt install fonts-dejavu-core fonts-dejavu-extra fonts-dejavu-mono
Fedora	dnf install dejavu-sans-fonts dejavu-sans-mono-fonts
Windows, macOS	DejaVu ist nicht dabei: die .ttf-Dateien in fonts/neben lib/ legen oder -fontdir angeben

Unter Debian liegen die Schnitte in drei Paketen. `fonts-dejavu-core` enthält `DejaVuSans.ttf` und `DejaVuSans-Bold.ttf`, aber nicht die schmale Familie `DejaVuSansCondensed`, die `fonts::init` ohne `-family` sucht. Die liegt in `fonts-dejavu-extra`, zusammen mit allen Oblique-Schnitten. Ist nur das Kernpaket installiert, hilft `-family DejaVuSans`.

Für eine Anwendung, die auf fremden Rechnern laufen soll, ist der Ordner `fonts/` neben `lib/` der sichere Weg: Die Schrift reist mit, und das PDF sieht überall gleich aus.

2.5 Prüfen, was geladen ist

```
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init
set kursiv [pdf4tcllib::fonts::hasTtfItalic]
puts "TrueType:  [pdf4tcllib::fonts::hasTtf]  kursiv: $kursiv"
puts "CID-Modus: [pdf4tcllib::fonts::isCidMode]"
foreach p {fontSans fontSansBold fontSansItalic fontSansBoldItalic
           fontMono} {
    puts [format "%-20s %s" $p [pdf4tcllib::fonts::$p]]
}
```

Ausgabe:

```
pdf4tcllib::fonts: TTF italic faces loaded
pdf4tcllib::fonts: TTF faces loaded from
    /usr/share/fonts/truetype/dejavu
TrueType:  1  kursiv: 1
CID-Modus: 0
fontSans             Pdf4tclSans
fontSansBold         Pdf4tclSansBold
```

```
fontSansItalic       Pdf4tclSansItalic
fontSansBoldItalic   Pdf4tclSansBoldItalic
fontMono             Pdf4tclMono
```

Findet `fonts::init` nichts, sagt es auf stderr, wo es gesucht hat, und fällt auf die Standardschriften zurück:

```
pdf4tcllib::fonts: No TTF fonts found, falling back to Helvetica
pdf4tcllib::fonts: searched in:
pdf4tcllib::fonts:   [---] .../pdf4tcllib/lib/../fonts
pdf4tcllib::fonts:   [---] .../pdf4tcllib/lib/fonts
pdf4tcllib::fonts:   [exists] /usr/share/fonts/truetype/dejavu
...
TrueType: 0  Sans: Helvetica
```

`[exists]` heißt nur, dass es den Ordner gibt, nicht, dass die Familie darin liegt. Das Programm läuft weiter. Wer sich auf TrueType verlässt, prüft darum `fonts::hasTtf` und bricht selbst ab:

```
pdf4tcllib::fonts::init
if {![pdf4tcllib::fonts::hasTtf]} {
    error "keine TrueType-Schrift gefunden, siehe stderr"
}
```

2.6 Schriftnamen immer über die Zugriffsprozeduren

pdf4tcl kennt die geladene Schrift unter einem Namen, den `fonts::init` vergibt: `Pdf4tclSans` mit TrueType, `Helvetica` ohne. Der Code fragt ihn ab, statt ihn hinzuschreiben:

Prozedur	mit TrueType	ohne
fonts::fontSans	Pdf4tclSans	Helvetica
fonts::fontSansBold	Pdf4tclSansBold	Helvetica-Bold
fonts::fontSansItalic	Pdf4tclSansItalic	Helvetica-Oblique
fonts::fontSansBoldItalic	Pdf4tclSansBoldItalic	Helvetica-BoldOblique
fonts::fontMono	Pdf4tclMono	Courier

So läuft derselbe Code mit und ohne TrueType. Die kursiven Schnitte fallen einzeln auf Helvetica-Oblique zurück, wenn die Oblique-Dateien fehlen (`fonts::hasTtfItalic` ist dann 0).

`fonts::setFont` nimmt einen Stil als Wort:

```
pdf4tcllib::fonts::setFont $pdf 14 Sans Bold       ;# fontSansBold
pdf4tcllib::fonts::setFont $pdf 14 Sans Italic     ;# fontSansItalic
pdf4tcllib::fonts::setFont $pdf 14 Sans BoldItalic ;# ...BoldItalic
```

Nach `fonts::init` ist die Familie dabei egal: `Bold`, `Italic` und `BoldItalic` führen immer zu den geladenen Schnitten. Ohne Stil setzt `fonts::setFont` die Familie unverändert, `fonts::setFont $pdf 14 Helvetica` also wirklich Helvetica. Für normalen Text darum

`$pdf setFont 14 [pdf4tcllib::fonts::fontSans]`.

Mit TrueType sind alle Schnitte eingebettet:

```
$ pdffonts 020-schriften.pdf     (Spalten gekürzt)
name                             type       emb
AAAAAA+_Pdf4tcl_Base_Italic      TrueType   yes
AAAAAA+_Pdf4tcl_Base_BoldItalic  TrueType   yes
AAAAAA+_Pdf4tcl_Base_Mono        TrueType   yes
AAAAAA+_Pdf4tcl_Base_Regular     TrueType   yes
AAAAAA+_Pdf4tcl_Base_Bold        TrueType   yes
```

2.7 256 Zeichen oder CID

`fonts::init` meldet die Schrift in einem von zwei Modi an:

	ohne -cid	**-cid 1**
Zeichen	256: ASCII, Latin-1 und 32 Zeichen dazu	alle, die die Schrift hat
darunter	Umlaute, €, –, —, …, •, Pfeile ←↑→↓, Rahmenlinien, ✓ ✗ ☐ ☑, ≤ ≥	zusätzlich Griechisch, Mathematik, Kyrillisch, typografische Anführungszeichen
PDF mit einer Zeile Text	22 KB	49–52 KB, je nach Text
Zeichen außerhalb	werden "?"	werden "?", wenn die Schrift sie nicht hat

Die Größen stammen aus dem Beispiel des nächsten Kapitels; der Unterschied ist die Schrift, die im CID-Modus vollständiger eingebettet wird.

Für deutsche Texte ist `-cid 1` die bessere Wahl. Die 256 Zeichen reichen für Umlaute und ß, aber nicht für die deutschen Anführungszeichen " und ": Sie fehlen in der Auswahl. Was das für `safeText` bedeutet, zeigt das Kapitel Unicode sicher ausgeben.

2.7.1 Den Modus beim ersten Aufruf wählen

Der Modus gilt für den ganzen Prozess, und er wird beim **ersten** `fonts::init` festgelegt. Ein späteres `fonts::init -cid 1` ändert die Schrift nicht mehr, auch nicht mit `-force 1`: Die Schrift ist bei pdf4tcl schon unter ihrem Namen angemeldet und wird nicht neu angemeldet. `fonts::isCidMode` meldet danach aber 1. In einem Versuch kam so aus "αβγ" im PDF "???". Der Zähler von pdf4tcllib blieb 0, auf stderr stand nichts; nur `$pdf getSubstCount` zählte die drei Zeichen.

Darum gehört `fonts::init` mit allen Optionen an eine Stelle am Programmanfang, bevor eine Bibliothek es mit anderen Optionen aufruft.

2.8 Zusammenfassung

- `package require pdf4tcl` und `package require pdf4tcllib`, dann einmal

`pdf4tcllib::fonts::init`, mit `-cid 1` für deutsche Texte.
- Die Schrift muss als `<familie>.ttf` und `<familie>-Bold.ttf` in einem der Suchordner liegen; unter Debian `fonts-dejavu-extra` installieren oder `-family DejaVuSans` angeben.
- `fonts::hasTtf` prüfen, wenn TrueType Pflicht ist.
- Schriftnamen nur über `fonts::fontSans` und die anderen Zugriffsprozeduren.
- Tcl 8.6 braucht für Umlaute im Skript eine UTF-8-Locale.

Nächstes Kapitel: Unicode sicher ausgeben

3 Unicode sicher ausgeben

3.1 Einführung

Ein Zeichen, das die Schrift nicht hat, verschwindet im PDF: Es steht als "?" da, als leerer Kasten oder gar nicht. Dass es fehlt, merkt man erst beim Lesen. pdf4tcllib fängt das an einer Stelle ab, in unicode::sanitize, und zählt mit, was es ersetzen musste.

Dieses Kapitel zeigt, was `sanitize` mit einer Zeile macht, welche Prozeduren es aufrufen und welche nicht, und woran man erkennt, dass Zeichen verloren gegangen sind.

3.2 Was sanitize tut

```
pdf4tcllib::unicode::sanitize text ?-mono 0|1? ?-tabwidth n?
```

`sanitize` gibt den Text zurück, so wie die geladene Schrift ihn zeigen kann. Es arbeitet in zwei Stufen:

1. **Bekannte Zeichen ersetzen.** Eine feste Tabelle setzt Zeichen, die oft in Texten stehen, auf etwas Darstellbares um: ✓ → ✓, (!) → (!), → → →, (TM) → (TM). Ohne TrueType ersetzt sie mehr: → wird `->`, € wird `EUR`, ✓ wird `(OK)`, Rahmenlinien werden `+`, `-` und `|`.
2. **Jedes übrige Zeichen prüfen.** Was die Schrift im jeweiligen Modus nicht hat, wird "?". Unsichtbare Zeichen (Null-Breite-Leerzeichen, Variantenwähler, BOM) fallen ganz weg, Tabulatoren werden Leerzeichen (`-tabwidth`, Vorgabe 4).

Welche Zeichen die Schrift "hat", hängt vom Modus aus `fonts::init` ab (Kapitel Einrichten und Schriften):

- ohne TrueType: druckbares ASCII und Latin-1,
- TrueType mit 256 Zeichen: die feste Auswahl aus `fonts::subset`,
- TrueType im CID-Modus: jedes Zeichen, für das die Schrift eine Glyphe hat.

Dieselbe Zeile in beiden TrueType-Modi:

```
# 030-unicode.tcl -- was sanitize aus einer Zeile macht
#   tclsh 030-unicode.tcl          256-Zeichen-Modus
#   tclsh 030-unicode.tcl -cid 1   CID-Modus (volle Schrift)
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init {*}$argv

```

```
set zeile "Pfeil \u2192, Euro \u20ac, Haken \u2705, alpha \u03b1,\
    Summe \u2211, \u201eZitat\u201c, Schneemann \u2603, Kanji \u6f22"
puts [pdf4tcllib::unicode::sanitize $zeile]
puts [pdf4tcllib::unicode::sanitize $zeile -mono 1]
puts [pdf4tcllib::unicode::substReport]

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
$pdf setFont 11 [pdf4tcllib::fonts::fontSans]
pdf4tcllib::unicode::safeText $pdf $zeile -x 50 -y 60
$pdf endPage
set cid [expr {[pdf4tcllib::fonts::isCidMode] ? "-cid" : ""}]
$pdf write -file 030-unicode$cid.pdf
$pdf destroy
```

```
$ tclsh 030-unicode.tcl
Pfeil →, Euro €, Haken ✓, alpha ?, Summe ?, "Zitat", Schneemann ?,
    Kanji ?
Pfeil ->, Euro EUR, Haken (OK), alpha ?, Summe ?, "Zitat", Schneemann
    ?, Kanji ?
count 8 chars {U+03B1 U+2211 U+2603 U+6F22}

$ tclsh 030-unicode.tcl -cid 1
Pfeil →, Euro €, Haken ✓, alpha α, Summe ∑, "Zitat", Schneemann ☃,
    Kanji ?
Pfeil →, Euro €, Haken ✓, alpha α, Summe ∑, "Zitat", Schneemann ☃,
    Kanji ?
count 2 chars U+6F22
```

Die erste Zeile ist normaler Text, die zweite `-mono 1`. Im CID-Modus bleibt auch das Griechische und die Summe; nur das Kanji fehlt, weil DejaVu keine chinesischen Zeichen hat. Der Zähler läuft über beide Aufrufe, darum steht beim 256-Zeichen-Modus 8: je vier Zeichen pro Zeile.

3.2.1 Festbreitenschrift: -mono 1

`-mono 1` ist für Code und Tabellen in Festbreitenschrift. Lädt `fonts::init` keine TrueType-Festbreitenschrift, ist das Courier, und Courier kennt nur Latin-1; dann gilt die große Ersetzungstabelle wie ohne TrueType (→ wird `->`). Im CID-Modus mit DejaVu Sans Mono bleibt Unicode erhalten und wird gegen diese Schrift geprüft. Im 256-Zeichen-Modus ersetzt `-mono 1` immer nach der großen Tabelle, auch wenn DejaVu Sans Mono geladen ist.

3.3 Wer sanitize aufruft

Prozedur	ruft sanitize auf
unicode::safeText	ja

Prozedur	ruft sanitize auf
page::header, page::footer, page::number, page::centerText	ja, über safeText
text::writeParagraph	nein, es ruft $pdf drawTextBox direkt
$pdf text, $pdf drawTextBox (pdf4tcl)	nein

Was nicht durch `sanitize` geht, ersetzt pdf4tcl selbst, wenn die Schrift ein Zeichen nicht kodieren kann. Auch dann steht "?" im PDF. Gezählt wird es an anderer Stelle: in `$pdf getSubstCount`.

```
# 030-drei-wege.tcl -- dieselbe Zeile auf drei Wegen
#   tclsh 030-drei-wege.tcl          256-Zeichen-Modus
#   tclsh 030-drei-wege.tcl -cid 1   CID-Modus
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init {*}$argv
set t "\u201eZitat\u201c, alpha \u03b1, Pfeil \u2192"

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
$pdf setFont 11 [pdf4tcllib::fonts::fontSans]
$pdf text "pdf4tcl direkt:  $t" -x 50 -y 60
pdf4tcllib::unicode::safeText $pdf "safeText:        $t" -x 50 -y 80
pdf4tcllib::text::writeParagraph $pdf "writeParagraph:  $t" \
    50 100 400 11
set bericht [pdf4tcllib::unicode::substReport]
puts "pdf4tcllib ersetzt: [dict get $bericht count]"
puts "pdf4tcl ersetzt:    [$pdf getSubstCount]"
set cid [expr {[pdf4tcllib::fonts::isCidMode] ? "-cid" : ""}]
$pdf write -file 030-drei-wege$cid.pdf
$pdf destroy
```

256-Zeichen-Modus:

```
pdf4tcllib ersetzt: 1
pdf4tcl ersetzt:    6

pdf4tcl direkt:  ?Zitat?, alpha ?, Pfeil →
safeText:        "Zitat", alpha ?, Pfeil →
writeParagraph:  ?Zitat?, alpha ?, Pfeil →
```

CID-Modus:

```
pdf4tcllib ersetzt: 0
pdf4tcl ersetzt:    0

pdf4tcl direkt:  "Zitat", alpha α, Pfeil →
safeText:        "Zitat", alpha α, Pfeil →
writeParagraph:  "Zitat", alpha α, Pfeil →
```

Zwei Dinge fallen auf:

- Im 256-Zeichen-Modus fehlen die Zeichen auf allen drei Wegen. Nur der Zähler unterscheidet sich: `safeText` zählt bei pdf4tcllib, die beiden anderen Wege bei pdf4tcl.
- `safeText` macht aus "Zitat" immer "Zitat", auch im CID-Modus, wo die Schrift " und " hat. Die erste Stufe von `sanitize` ersetzt die typografischen Anführungszeichen (" " " ' ' ') in jedem Modus durch gerade.

3.3.1 Deutsche Anführungszeichen

Für deutsche Texte heißt das: CID-Modus, und Text mit "..." nicht über `safeText` ausgeben, sondern über `text::writeParagraph` oder `$pdf text`. Beide lassen die Zeichen stehen. Im 256-Zeichen-Modus gibt es keinen Weg zu "…"; dort bleiben nur gerade Anführungszeichen oder » « (die sind in Latin-1).

Die Kopf- und Fußzeilen aus `page::header` und `page::footer` gehen über `safeText`; ein "Titel" darin verliert seine Anführungszeichen auch im CID-Modus.

3.4 Die Schrift beim Zeichnen ist nicht die, nach der sanitize fragt

`sanitize` richtet sich nach dem, was `fonts::init` geladen hat, nicht nach der Schrift, die gerade mit `setFont` gesetzt ist. Wer nach `fonts::init` für einen Absatz Helvetica setzt, bekommt Zeichen durchgereicht, die Helvetica nicht kennt:

```
# 030-falle-schrift.tcl -- sanitize folgt fonts::init, nicht setFont
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init
set t "Pfeil \u2192, Euro \u20ac, Haken \u2713"

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
$pdf setFont 12 [pdf4tcllib::fonts::fontSans]
pdf4tcllib::unicode::safeText $pdf "fontSans:  $t" -x 50 -y 60
$pdf setFont 12 Helvetica
pdf4tcllib::unicode::safeText $pdf "Helvetica: $t" -x 50 -y 80
set bericht [pdf4tcllib::unicode::substReport]
puts "pdf4tcllib ersetzt: [dict get $bericht count]"
puts "pdf4tcl ersetzt:    [$pdf getSubstCount]"
$pdf write -file 030-falle-schrift.pdf
$pdf destroy
```

```
pdf4tcllib ersetzt: 0
pdf4tcl ersetzt:    0

fontSans:  Pfeil →, Euro €, Haken ✓
Helvetica: Pfeil ?, Euro €, Haken ?
```

Unter Tcl 8.6 bleiben beide Zähler bei 0, und trotzdem fehlen im zweiten Satz zwei Zeichen. Unter Tcl 9 meldet wenigstens pdf4tcl sie (`pdf4tcl ersetzt: 2`). Nach `fonts::init` also nur noch die Schriften aus `fonts::fontSans` und den anderen Zugriffsprozeduren setzen. Wer Courier oder Helvetica wirklich braucht, gibt den Text mit `-mono 1` aus; dann gilt die große Ersetzungstabelle, und aus → wird `->` statt "?".

3.5 Emoji

Zeichen oberhalb von U+FFFF, also die meisten Emoji, ersetzt `sanitize` durch eine Umschrift aus ASCII: :-) wird `:-)`, (+1) wird `(+1)`, [L] wird `[L]`. Unbekannte Emoji werden `(?)`. Diese Umschrift zählt nicht als Verlust und erscheint nicht in `substReport`.

```
$ tclsh 030-emoji.tcl
Tcl 8.6, direkt gelesen: Gruss :-) und (+1)
Tcl 8.6, mit readFile:   Gruss :-) und (+1)
```

Tcl 8.6.14 und Tcl 9.0 lesen die Emoji aus einer UTF-8-Datei richtig ein. unicode::readFile liest die Datei binär und setzt die Umschrift ein, bevor Tcl die Bytes umwandelt; das ist für ältere 8.6-Fassungen gedacht, die solche Zeichen beim Lesen zu U+FFFD machen. Ein U+FFFD im Text wird `(?)`, das ursprüngliche Zeichen ist dann nicht mehr zu erkennen.

3.6 Wissen, was gefehlt hat

`sanitize` zählt jedes Zeichen, das es durch "?" ersetzt, und merkt sich bis zu zwölf verschiedene davon:

```
pdf4tcllib::unicode::substCount   ;# 8
pdf4tcllib::unicode::substReport  ;# count 8 chars {U+03B1 U+2211 ...}
pdf4tcllib::unicode::resetSubst
```

Beim ersten Mal schreibt es zusätzlich eine Warnung nach stderr:

```
pdf4tcllib::unicode: sanitize: at least one character is missing from
    the
font and was replaced with '?' (first: U+03B1). Ask
::pdf4tcllib::unicode::substReport for the list, or load a font that
    has
it -- fonts::init -cid 1 for Greek and mathematical symbols.
```

Für ein Programm, das PDFs ohne Aufsicht erzeugt, gehören beide Zähler an das Ende jedes Dokuments:

```
set verloren [expr {[pdf4tcllib::unicode::substCount]
                    + [$pdf getSubstCount]}]
if {$verloren} {
    puts stderr "Warnung: $verloren Zeichen ersetzt\
        ([dict get [pdf4tcllib::unicode::substReport] chars])"
}
```

`substCount` zählt über alle Dokumente des Prozesses; vor jedem neuen Dokument `resetSubst` aufrufen. `$pdf getSubstCount` gilt für das eine pdf4tcl-Objekt. Den Fall aus dem vorigen Abschnitt (Helvetica nach `fonts::init`) meldet unter Tcl 8.6 keiner der beiden.

3.7 Zusammenfassung

- Text mit Sonderzeichen über `unicode::safeText` ausgeben, Absätze über `text::writeParagraph`, beides mit Schriften aus den Zugriffsprozeduren.
- Für Griechisch, Mathematik und deutsche Anführungszeichen `fonts::init -cid 1`.
- `safeText` ersetzt " " immer durch "; für "..." `writeParagraph` oder `$pdf text` nehmen.
- Am Ende `unicode::substCount` und `$pdf getSubstCount` prüfen.

Nächstes Kapitel: Einheiten und Seitenkontext

4 Einheiten und Seitenkontext

4.1 Einführung

pdf4tcl rechnet in Punkt (1/72 Zoll), Formulare und Briefvorlagen sind in Millimetern bemaßt. Und jede Zeichenoperation braucht dieselben Zahlen: wo der Satzspiegel links beginnt, wie breit er ist, wo unten Schluss ist. page::context rechnet diese Zahlen einmal aus und gibt sie als Dict zurück. Alle Bausteine aus pdf4tcllib, die auf der Seite etwas platzieren, nehmen dieses Dict.

4.2 Einheiten umrechnen

```
pdf4tcllib::units::mm   10      ;# 28.346...  Millimeter -> Punkt
pdf4tcllib::units::cm   2.5     ;# 70.866...
pdf4tcllib::units::inch 1       ;# 72.0
pdf4tcllib::units::to_mm 72     ;# 25.4       Punkt -> Millimeter
pdf4tcllib::units::to_cm 72     ;# 2.54
pdf4tcllib::units::to_inch 144  ;# 2.0
```

Das ist alles, was der Namensraum units tut. Die Ergebnisse sind Gleitkommazahlen; gerundet wird erst beim Zeichnen.

4.2.1 pdf4tcl in Punkt lassen

pdf4tcl kann selbst in Millimetern rechnen (`pdf4tcl::new ... -unit mm`). Mit pdf4tcllib nicht: `page::context` und alle Bausteine liefern und erwarten Punkt. Ein Objekt mit `-unit mm` nimmt die Punktwerte als Millimeter, und eine Fußzeile bei y = 813 landet 813 mm tief, weit unter der Seite. Das PDF ist dann leer, ohne Fehlermeldung.

Ebenso die Option `-margin` von pdf4tcl: Sie verschiebt den Nullpunkt um den Rand.

`page::context` rechnet den Rand schon ein; beides zusammen ergibt den doppelten Rand. Ein Versuch mit 20 mm in beiden setzte den Text bei x = 113 pt statt bei 57 pt.

Darum für pdf4tcllib:

```
# ohne -unit, ohne -margin
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
```

4.3 page::context

```
pdf4tcllib::page::context paper ?-margin mm? ?-landscape 0|1? \
    ?-orient 0|1?
```

Argument	Vorgabe	Bedeutung
paper	–	a3, a4, a5, b5, letter, legal (groß oder klein geschrieben)
-margin	20	Rand in Millimetern, auf allen vier Seiten gleich
-landscape	0	1: Querformat, Breite und Höhe getauscht
-orient	1	1: y wächst nach unten, 0: nach oben

Ein anderes Papier lehnt es ab:

```
unknown paper size: din-lang (known: letter a3 legal a4 a5 b5)
```

Die Kurzform `context a4 20 0` ist gleichbedeutend mit `context a4 -margin 20 -orient 0`; das zweite Wort ist also die Richtung, nicht das Querformat. Besser lesbar ist die Form mit Optionen.

4.3.1 Was im Dict steht

```
# 040-seitenkontext.tcl -- Satzspiegel aus page::context, hoch und quer
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init

proc zeige {ctx} {
    foreach k {paper page_w page_h margin left right top bottom
               text_w text_h} {
        set v [dict get $ctx $k]
        if {[string is double -strict $v]} { set v [format %.2f $v] }
        puts [format "  %-7s %s" $k $v]
    }
}

puts "A4 hoch, 20 mm Rand, y nach unten (orient true):"
set ctx [pdf4tcllib::page::context a4 -margin 20]
zeige $ctx
```

```
puts "A4 quer, 15 mm Rand, y nach oben (orient false):"
set quer [pdf4tcllib::page::context a4 -margin 15 -landscape 1 \
    -orient false]
zeige $quer

# Den Satzspiegel einzeichnen: einmal je Richtung, jeweils mit
# passendem pdf4tcl-Objekt
foreach {name c} [list hoch $ctx quer $quer] {
    set pdf [pdf4tcl::new %AUTO% -paper a4 \
        -landscape [dict get $c landscape] \
        -orient [dict get $c orient]]
    $pdf startPage
    set l [dict get $c left]; set r [dict get $c right]
    set t [dict get $c top];  set b [dict get $c bottom]
    $pdf setStrokeColor 0.2 0.4 0.8
    $pdf rectangle $l [expr {min($t, $b)}] \
        [dict get $c text_w] [dict get $c text_h]
    $pdf setFont 10 [pdf4tcllib::fonts::fontSans]
    set y $t
    set mm [pdf4tcllib::units::to_mm [dict get $c text_w]]
    set zeilen [list "Satzspiegel $name" "[format %.0f $mm] mm breit"]
    foreach zeile $zeilen {
        pdf4tcllib::page::_advance $c y 14
        pdf4tcllib::unicode::safeText $pdf $zeile \
            -x [expr {$l + 6}] -y $y
    }
    pdf4tcllib::page::number $pdf $c 1
    $pdf endPage
    $pdf write -file 040-seitenkontext-$name.pdf
    $pdf destroy
}
```

```
A4 hoch, 20 mm Rand, y nach unten (orient true):
  paper   a4
  page_w  595.28
  page_h  841.89
  margin  56.69
  left    56.69
  right   538.59
  top     56.69
  bottom  785.20
  text_w  481.89
  text_h  728.50
A4 quer, 15 mm Rand, y nach oben (orient false):
  paper   a4
  page_w  841.89
```

```
  page_h  595.28
  margin  42.52
  left    42.52
  right   799.37
  top     552.76
  bottom  42.52
  text_w  756.85
  text_h  510.24
```

Schlüssel	Bedeutung	auch als
page_w, page_h	Seite in Punkt	PW, PH
margin	Rand in Punkt	margin_pt
margin_mm	Rand, wie angegeben	–
left, right	linke und rechte Kante des Satzspiegels	SX für left
top, bottom	obere und untere Kante, in der gewählten Richtung	SY für top
text_w, text_h	Breite und Höhe des Satzspiegels	SW, SH
paper, landscape, orient	die Eingaben	–

Die Kurznamen gibt es für älteren Code. Andere Schreibweisen (`pageW`, `textWidth`) gibt es nicht; `dict get` meldet dann einen Fehler.

4.3.2 top und bottom hängen an der Richtung

`top` ist immer die obere Kante des Satzspiegels, `bottom` die untere, aber in der Zählweise der Seite:

	-orient 1 (y nach unten)	-orient 0 (y nach oben)
top	klein (Rand)	groß (Höhe − Rand)
bottom	groß (Höhe − Rand)	klein (Rand)
nächste Zeile	y größer	y kleiner

Die Richtung im Kontext und die Richtung des pdf4tcl-Objekts müssen übereinstimmen. Am einfachsten nimmt man sie aus dem Kontext:

```
set pdf [pdf4tcl::new %AUTO% -paper a4 \
    -landscape [dict get $ctx landscape] \
    -orient [dict get $ctx orient]]
```

Die Bausteine dieses Buchs gehen von `-orient 1` aus, der Vorgabe beider Pakete. Tabellen aus `table::draw` lehnen `-orient 0` mit einem Fehler ab (Kapitel Tabellen mit Seitenumbruch).

4.3.3 Zeile für Zeile weiter

page::_advance schiebt eine y-Variable in die richtige Richtung, ohne dass der Code die Richtung

kennen muss:

```
set y [dict get $ctx top]
foreach zeile $zeilen {
    pdf4tcllib::page::_advance $ctx y 14   ;# erst weiter, dann setzen
    pdf4tcllib::unicode::safeText $pdf $zeile \
        -x [dict get $ctx left] -y $y
}
```

Der Unterstrich im Namen täuscht: Die Prozedur ist dokumentiert und wird von den Formular-Bausteinen selbst benutzt.

Der Zeilenabstand zu einer Schriftgröße kommt aus page::lineheight: Schriftgröße × 1,4, aufgerundet (`page::lineheight 10` ergibt 14). Ein anderer Faktor geht als zweites Argument.

4.3.4 Die erste Zeile

y ist bei `$pdf text` die Grundlinie. Text, der bei `y = top` steht, ragt mit seinen Oberlängen über den Satzspiegel in den Rand. Die erste Zeile beginnt darum eine Schriftgröße tiefer, bei `top` plus Schriftgröße (in Richtung `-orient 1`). Genau das tut das Beispiel oben mit `_advance` vor dem Schreiben.

4.4 Kopf, Fuß und Seitenzahl

Drei Prozeduren schreiben in den Rand, oberhalb und unterhalb des Satzspiegels:

```
# Mitte oben, Linie darunter
pdf4tcllib::page::header $pdf $ctx "Monatsbericht"
# links der Text, rechts "Seite 3"
pdf4tcllib::page::footer $pdf $ctx "Vertraulich" 3
# Mitte unten "- 3 / 10 -"
pdf4tcllib::page::number $pdf $ctx 3 10
```

`page::footer` schreibt die Seitenzahl als "Seite 3". Das ist eine Vorgabe aus der Bibliothek, keine Übersetzung; für ein englisches Dokument:

```
# für alle Aufrufe
set ::pdf4tcllib::page::pageLabelFormat "Page %s"
# nur für diesen Aufruf
pdf4tcllib::page::footer $pdf $ctx "Confidential" 3 9 \
    -pagelabel "Page %s"
```

Die Schrift ist die geladene TrueType-Schrift, ohne TrueType Helvetica; die Größe ist das letzte Zahlenargument (Vorgabe 10 für den Kopf, 9 für Fuß und Seitenzahl). Alle drei gehen über `safeText` (Kapitel Unicode sicher ausgeben).

Eine vierte Prozedur, `page::centerText $pdf $ctx text y ?size? ?font?`, setzt eine Zeile mittig, etwa einen Titel. Ohne Schriftargument nimmt sie **Helvetica**, nicht die geladene TrueType-Schrift; Helvetica wird nicht eingebettet, und das PDF taugt nicht mehr für PDF/A. Darum die Schrift immer mitgeben:

```
pdf4tcllib::page::centerText $pdf $ctx "Jahresbericht" 120 20 \
    [pdf4tcllib::fonts::fontSansBold]
```

Wie viele Seiten ein Dokument hat, weiß man erst am Ende. "Seite 3 von 10" braucht darum einen zweiten Durchlauf; das zeigt das Kapitel Kopf- und Fußzeilen, Seitenzahlen.

4.5 Hilfslinien beim Entwickeln

Prozedur	zeichnet
page::grid $pdf $ctx ?abstand?	ein graues Raster über die ganze Seite, mit Koordinaten am Rand (Vorgabe 50 pt)
page::debugGrid $pdf $ctx ?abstand?	dasselbe, aber nur wenn die Umgebungsvariable PDF4TCL_DEBUG gesetzt ist
page::orientationLegend $pdf $ctx	eine Zeile in der rechten Ecke: Papier, Richtung, Lage des Nullpunkts, Einheit

`debugGrid` kann im Code bleiben: Ohne `PDF4TCL_DEBUG=1` zeichnet es nichts.

```
PDF4TCL_DEBUG=1 tclsh bericht.tcl
```

`page::grid` teilt die Seite nicht in Spalten. Ein Raster für Spalten baut man aus `left` und `text_w` selbst; Text über mehrere Spalten setzt das Paket `pdf4tclflow` (Kapitel Spaltenfluss und Wasserzeichen).

4.6 Zusammenfassung

- pdf4tcllib rechnet in Punkt; Ränder gibt man in Millimetern an.
- pdf4tcl-Objekte ohne `-unit` und ohne `-margin` anlegen.
- `page::context` liefert `left`, `top`, `text_w` und den Rest; die Richtung im Kontext und im pdf4tcl-Objekt muss gleich sein.
- `page::_advance` und `page::lineheight` für die nächste Zeile, die erste Zeile eine Schriftgröße unter `top`.
- `page::header`, `page::footer`, `page::number` für den Rand; "Seite" über `pageLabelFormat` anpassen.

Nächstes Kapitel: Text setzen

5 Text setzen

5.1 Einführung

pdf4tcl setzt eine Zeile dorthin, wo man sie hinschreibt. Ist sie zu lang, läuft sie über den Rand. Für alles darüber hinaus gibt es im Namensraum text vier Werkzeuge: Breite messen, umbrechen, kürzen, verkleinern. Dazu kommt `writeParagraph` für einen ganzen Absatz in einem Zug.

Dieses Kapitel zeigt die vier Werkzeuge und baut daraus eine kleine Prozedur, die Text über beliebig viele Seiten setzt.

5.2 Breite messen

```
pdf4tcllib::text::width text size font ?pdf?
```

Mit dem pdf4tcl-Objekt als letztem Argument fragt `width` pdf4tcl nach den echten Maßen der Schrift. Ohne Objekt schätzt es über Zeichenklassen: schmale Buchstaben wie `i` und `l` zählen weniger, `m` und `W` mehr.

```
# 050-messen-und-umbrechen.tcl -- messen, umbrechen, kuerzen
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set font [pdf4tcllib::fonts::fontSans]
set pdf  [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Breite: geschaetzt (ohne pdf) und gemessen (mit pdf)
set t "Lieferung Mo 17.08. Spedition Nord"
puts [format "geschaetzt %.1f pt, gemessen %.1f pt" \
    [pdf4tcllib::text::width $t 10 $font] \
    [pdf4tcllib::text::width $t 10 $font $pdf]]

# Umbrechen: am Leerzeichen und nach einem Bindestrich
set lang "Die Muster-Beispiel-Stadt liegt an der Bundesstrasse."
foreach z [pdf4tcllib::text::wrap $lang 90 10 $font 0 $pdf] {
    puts "  |$z|"
}

# Ein Wort, das allein zu breit ist: gekuerzt oder zerlegt
set name "pdf4tcllib::unicode::substReport"
set ohne [pdf4tcllib::text::wrap $name 80 10 $font 0 $pdf]
set mit  [pdf4tcllib::text::wrap $name 80 10 $font 0 $pdf 1]
puts "ohne hardBreak: $ohne"
puts "mit hardBreak:  $mit"

# Kuerzen mit "..." oder kleiner setzen
set k "Tour Sped1 Nord"
puts "truncate: [pdf4tcllib::text::truncate $k 60 10 $font $pdf]"
puts "fitSize:  [pdf4tcllib::text::fitSize $k 60 10 $font -pdf $pdf]"
puts "fits:     [pdf4tcllib::text::fits $k 60 7 $font $pdf]"
$pdf destroy
```

```
geschaetzt 156.0 pt, gemessen 159.7 pt
  |Die Muster-Beispiel-|
  |Stadt liegt an der|
  |Bundesstrasse.|
ohne hardBreak: pdf4tcllib::unic...
mit hardBreak:  pdf4tcllib::unicod e::substReport
```

```
truncate: Tour Sped1...
fitSize:  8
fits:     1
```

Bei diesem Satz liegt die Schätzung 2 % daneben, bei anderen Texten mit DejaVu bis etwa 6 %. Das reicht, um eine Zeile umzubrechen, die gepasst hätte, oder eine passen zu lassen, die übersteht. Darum hängen alle Aufrufe in diesem Buch das Objekt `$pdf` an. Die Schätzung ist nur für den Fall da, dass noch kein Dokument existiert.

`width` braucht kein `setFont` vorher: Schrift und Größe stehen im Aufruf.

5.3 Umbrechen

```
pdf4tcllib::text::wrap line maxW size font \
    ?code? ?pdf? ?hardBreak? ?trenner?
```

`wrap` gibt eine Liste von Zeilen zurück, die jede höchstens `maxW` breit sind. Gezeichnet wird nichts. Die Reihenfolge der Argumente ist ungewohnt: `pdf` steht an sechster Stelle, nach `code`.

Argument	Vorgabe	Bedeutung
code	0	1: jede umbrochene Zeile außer der letzten endet auf \, wie eine fortgesetzte Tcl-Zeile
pdf	–	Objekt für genaue Breiten
hardBreak	0	was mit einem Wort geschieht, das allein zu breit ist (siehe unten)
trenner	{- /}	Zeichen, an denen auch innerhalb eines Worts umbrochen werden darf

Umbrochen wird am Leerzeichen und nach einem Bindestrich oder Schrägstrich zwischen zwei Buchstaben oder Ziffern. Der Trenner bleibt am Ende der Zeile stehen ("Muster-Beispiel-"), so sieht der Leser, dass es weitergeht. Ein Strich am Wortanfang trennt nicht: "-5 Euro" bleibt zusammen. pdf4tcl bricht in `drawTextBox` nach denselben Regeln um.

5.3.1 Ein Wort, das nicht passt

Was mit einem Wort geschieht, das allein breiter ist als die Zeile, entscheidet `hardBreak`:

- `0` (Vorgabe): Das Wort wird **gekürzt** und endet auf "...". Der Rest ist weg.
- `1`: Das Wort wird zeichenweise zerlegt, nichts geht verloren.

Die Vorgabe ist für Fließtext gedacht, in dem ein solches Wort selten ist. In Tabellenzellen mit Pfaden, Paketnamen oder Artikelnummern ist sie falsch: Aus `pdf4tcllib::unicode::substReport` wird `pdf4tcllib::unic...`. Dort `hardBreak 1` setzen.

5.4 Kürzen oder verkleinern

```
pdf4tcllib::text::truncate text maxW size font ?pdf?
pdf4tcllib::text::fitSize  text maxW size font ?-min n? ?-pdf obj?
pdf4tcllib::text::fits     text maxW size font ?pdf?
```

text::truncate schneidet ab und hängt drei Punkte an. text::fitSize sucht stattdessen die größte Schriftgröße, bei der der Text ganz passt, in Schritten von 1 Punkt nach unten bis `-min` (Vorgabe 6). Passt er auch dort nicht, kommt `-min` zurück, und der Aufrufer entscheidet.

Was besser ist, hängt am Inhalt. "Tour Sped1 Nord" gekürzt ist "Tour Sped1…": Ob es "Nord" oder "Süd" war, steht nicht mehr da. Bei Namen, Touren, Beträgen und Kennungen ist eine kleinere Schrift besser als ein abgeschnittenes Ende. Bei einer Bemerkung, die ohnehin nur ein Hinweis ist, darf gekürzt werden.

Bei `fitSize` heißt das Objekt `-pdf`, bei den anderen ist es das letzte Argument ohne Namen.

5.5 Einen Absatz in einem Zug

```
set y [pdf4tcllib::text::writeParagraph pdf text x y width \
    ?size? ?align? ?tag?]
```

text::writeParagraph bricht um und zeichnet in einem Schritt, über `drawTextBox` von pdf4tcl. Es gibt die y-Position unter der letzten Zeile zurück. `align` ist `left`, `right`, `center` oder `justify`.

Drei Dinge sollte man wissen:

- Die Schrift ist immer die normale aus `fonts::init` (ohne TrueType Helvetica). Fett oder kursiv geht damit nicht.
- Es gibt keinen Seitenumbruch. Ein Absatz, der unten nicht mehr passt, läuft über den Rand hinaus.
- Der Text geht nicht durch `sanitize` (Kapitel Unicode sicher ausgeben).

Für einen kurzen Absatz an bekannter Stelle ist `writeParagraph` richtig. Für Text, dessen Länge man nicht kennt, braucht es einen Umbruch über Seiten.

5.6 Text über Seiten

Die folgende Prozedur `absatz` bricht einen Absatz mit `wrap` um und setzt ihn Zeile für Zeile. Passt die nächste Zeile nicht mehr über `bottom`, beginnt `neueSeite` eine neue Seite mit Fußzeile. Schrift und Größe sind frei wählbar, auch fett.

```
# 050-text-ueber-seiten.tcl -- Fliesstext mit eigenem Seitenumbruch
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Der Zustand der Seite: wo die naechste Zeile steht
set y 0
```

```
proc neueSeite {} {
    global pdf ctx y
    if {[$pdf currentPage] > 0} {
        set n [$pdf currentPage]
        pdf4tcllib::page::footer $pdf $ctx "Lagerbericht" $n
        $pdf endPage
    }
    $pdf startPage
    set y [dict get $ctx top]
}

# Einen Absatz setzen: umbrechen, Zeile fuer Zeile, bei Bedarf eine
# neue Seite
proc absatz {text {size 11} {font ""}} {
    global pdf ctx y
    if {$font eq ""} { set font [pdf4tcllib::fonts::fontSans] }
    set lh [pdf4tcllib::page::lineheight $size]
    set breite [dict get $ctx text_w]
    foreach zeile [pdf4tcllib::text::wrap $text $breite \
            $size $font 0 $pdf 1] {
        if {$y + $lh > [dict get $ctx bottom]} { neueSeite }
        set y [expr {$y + $lh}]
        $pdf setFont $size $font
        pdf4tcllib::unicode::safeText $pdf $zeile \
            -x [dict get $ctx left] -y $y
    }
    set y [expr {$y + $lh / 2}]
}

neueSeite
absatz "Bestand am Monatsende" 16 [pdf4tcllib::fonts::fontSansBold]
for {set i 1} {$i <= 40} {incr i} {
    absatz "Regal $i: Der Bestand wurde gezählt und mit der Buchung\
        abgeglichen. Abweichungen unter zwei Stück sind ausgebucht,\
        größere gehen an die Lagerleitung zur Klärung."
}
pdf4tcllib::page::footer $pdf $ctx "Lagerbericht" [$pdf currentPage]
$pdf endPage
puts "Seiten: [$pdf pageCount]"
$pdf write -file 050-text-ueber-seiten.pdf
$pdf destroy
```

```
Seiten: 3
```

Was die Prozedur richtig macht:

- Sie prüft **vor** jeder Zeile, ob sie noch passt, nicht danach. Die letzte Zeile einer Seite steht

dann noch im Satzspiegel.
- Sie schiebt y **vor** dem Schreiben weiter, eine Zeilenhöhe unter `top`. So ragt die erste Zeile einer Seite nicht in den Rand (Kapitel Einheiten und Seitenkontext).
- Sie setzt die Schrift vor jeder Zeile neu. `neueSeite` setzt in der Fußzeile eine andere Schrift, und ohne das neue `setFont` liefe der Absatz auf der neuen Seite in der Schrift der Fußzeile weiter.
- Sie nimmt `hardBreak 1`, damit kein Wort gekürzt wird.

Die Seitenzahl kommt aus `$pdf currentPage`; nach dem letzten `endPage` sagt `$pdf pageCount`, wie viele Seiten es geworden sind.

Was die Prozedur nicht macht: Sie verhindert keine einzelne Zeile oben auf einer Seite und keine Überschrift unten. Dafür muss man vor dem Setzen wissen, wie hoch ein Block wird; das zeigt das Kapitel zu `measure` und `layout`.

5.7 Zusammenfassung

- `text::width`, `text::wrap`, `text::truncate`, `text::fits` immer mit dem pdf4tcl-Objekt aufrufen, sonst wird geschätzt.
- `wrap` kürzt ein zu breites Wort; `hardBreak 1`, wo nichts verloren gehen darf.
- `fitSize` statt `truncate` bei Namen, Kennungen und Beträgen.
- `writeParagraph` für einen kurzen Absatz in normaler Schrift; für längeren Text eine eigene Schleife mit Seitenumbruch.

Nächstes Kapitel: Kopf- und Fußzeilen, Seitenzahlen

6 Kopf- und Fußzeilen, Seitenzahlen

6.1 Einführung

Kopf- und Fußzeilen stehen auf jeder Seite, und sie werden in dem Moment geschrieben, in dem eine Seite beginnt oder endet. pdf4tcllib liefert die drei Bausteine dafür (Kapitel Einheiten und Seitenkontext); wann sie aufgerufen werden, entscheidet der eigene Code. Dieses Kapitel legt diese Stelle fest und löst die Frage, die jeder Bericht irgendwann stellt: "Seite 2 von wie vielen?"

6.2 Eine Stelle für den Seitenwechsel

Alles, was auf jede Seite gehört, steht in einer Prozedur, die die alte Seite abschließt und die neue beginnt: im vorigen Kapitel `neueSeite`. Wer Kopf und Fuß an mehreren Stellen schreibt, vergisst sie an einer.

Was	Wann	Prozedur
Fußzeile, Seitenzahl	bevor die Seite endet	page::footer, page::number
Kopfzeile, Wasserzeichen	nachdem die neue Seite beginnt	page::header, drawing::watermark
y auf den Anfang	nach der Kopfzeile	dict get $ctx top

Die Fußzeile gehört an das Ende einer Seite, nicht an ihren Anfang: Erst dann ist sicher, dass die

Seite Inhalt hat und dass ihre Nummer stimmt.

Die drei Bausteine schreiben in den Rand, nicht in den Satzspiegel:

Baustein	Lage (y nach unten)
page::header	Text bei halbem Rand von oben, Linie 2 pt über top
page::footer	Linie 2 pt unter bottom, Text bei halbem Rand von unten
page::number	wie der Text der Fußzeile, in der Mitte

Bei 20 mm Rand steht der Text der Kopf- und Fußzeile 10 mm von der Papierkante. Mit kleinerem Rand rückt er näher heran; bei 10 mm Rand sind es 5 mm, und das kann für einen Drucker zu knapp sein.

6.3 Seite 2 von 3

Wie viele Seiten ein Dokument hat, steht erst fest, wenn es fertig ist. Die Fußzeile der ersten Seite ist dann aber längst geschrieben. Die übliche Lösung: das Dokument zweimal setzen. Der erste Durchlauf geht in ein Dokument, das nur gezählt und dann verworfen wird; der zweite weiß die Zahl.

```
# 060-seite-x-von-y.tcl -- "Seite 2 von 3": zwei Durchlaeufe
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]

# Der ganze Bericht als Prozedur: sie bekommt das Dokument und die
# Gesamtzahl der Seiten ("" im ersten Durchlauf) und hat keine
# Nebenwirkungen -- sie laeuft zweimal.
proc bericht {pdf gesamt} {
    global ctx
    set font [pdf4tcllib::fonts::fontSans]
    set lh   [pdf4tcllib::page::lineheight 11]
    set seite 0
    set y 0
    set neu {
        if {$seite > 0} { rand $pdf $seite $gesamt; $pdf endPage }
        $pdf startPage
        incr seite
        if {$seite > 1} {
            pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"
        }
        set y [dict get $ctx top]
    }
    eval $neu
    for {set i 1} {$i <= 120} {incr i} {
```

```
        if {$y + $lh > [dict get $ctx bottom]} { eval $neu }
        set y [expr {$y + $lh}]
        $pdf setFont 11 $font
        pdf4tcllib::unicode::safeText $pdf "Position $i: geprüft" \
            -x [dict get $ctx left] -y $y
    }
    rand $pdf $seite $gesamt
    $pdf endPage
}

proc rand {pdf seite gesamt} {
    global ctx
    set label "Seite %s"
    if {$gesamt ne ""} { append label " von $gesamt" }
    pdf4tcllib::page::footer $pdf $ctx "Vertraulich" $seite 9 \
        -pagelabel $label
}

# Durchlauf 1: in ein Wegwerf-Dokument, nur um die Seiten zu zaehlen
set probe [pdf4tcl::new %AUTO% -paper a4 -orient true]
bericht $probe ""
set gesamt [$probe pageCount]
$probe destroy

# Durchlauf 2: das echte Dokument
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
bericht $pdf $gesamt
$pdf write -file 060-seite-x-von-y.pdf
$pdf destroy
puts "Seiten: $gesamt"
```

```
Seiten: 3
```

Die Fußzeilen lauten "Seite 1 von 3", "Seite 2 von 3", "Seite 3 von 3". Die erste Seite hat keine Kopfzeile, die folgenden "Lagerbericht Oktober".

Was dafür nötig ist:

- **Der Inhalt steht in einer Prozedur**, die das Dokument als Argument bekommt. Sie läuft zweimal und darf darum nichts tun, was beim zweiten Mal anders ausgeht: nicht in eine Datei anhängen, keinen Zähler außerhalb hochzählen, nicht aus einem Kanal lesen, der danach leer ist. Daten, die aus einer Datenbank kommen, einmal vorher lesen und der Prozedur übergeben.
- **Die Seitenzahl darf den Umbruch nicht ändern.** "Seite 9 von 9" und "Seite 9 von 10" sind verschieden breit, aber beide stehen in der Fußzeile, außerhalb des Satzspiegels. Der Inhalt bricht in beiden Durchläufen gleich um. Wer die Gesamtzahl im Text verwendet ("Dieser Bericht hat 3 Seiten"), kann die Zahl der Zeilen ändern; dann den zweiten Durchlauf gegen den ersten prüfen.
- **-pagelabel nimmt ein Format.** `page::footer` setzt die Seitenzahl mit `format` ein. "Seite %s von 3" ergibt "Seite 2 von 3". Ein Prozentzeichen im Text muss darum `%%` lauten.

Das Wegwerf-Dokument kostet nur Rechenzeit; geschrieben wird es nicht. `$probe pageCount` zählt die abgeschlossenen Seiten, darum endet auch der erste Durchlauf mit `endPage`.

Das Paket `pdf4tcltoc` arbeitet nach demselben Prinzip, um das Inhaltsverzeichnis mit echten Seitenzahlen vor den Text zu setzen (Kapitel Inhaltsverzeichnis).

6.4 Seitenzahl ohne Wort

`page::number` schreibt `- 2 -` oder `- 2 / 3 -` in die Mitte, ohne Sprache. Das passt, wenn es keine Fußzeile mit Text gibt:

```
pdf4tcllib::page::number $pdf $ctx [$pdf currentPage] $gesamt
```

Mit `""` als Gesamtzahl erscheint nur die Nummer. Das ist im ersten Durchlauf praktisch, in dem die Zahl noch nicht bekannt ist.

6.5 Zusammenfassung

- Kopf, Fuß, Wasserzeichen und Seitenanfang gehören in eine Prozedur für den Seitenwechsel.
- Fußzeile vor `endPage`, Kopfzeile nach `startPage`.
- "Seite X von Y": das Dokument zweimal setzen, die Gesamtzahl aus `$probe pageCount`; der Inhalt als Prozedur ohne Nebenwirkungen.
- `-pagelabel "Seite %s von $gesamt"`; ein `%` im Text als `%%`.

Nächstes Kapitel: Spaltenfluss und Wasserzeichen

7 Spaltenfluss und Wasserzeichen

7.1 Einführung

Ein Rundschreiben in zwei Spalten, ein Protokoll mit "ENTWURF" quer über jeder Seite: Beides lässt sich mit der Schleife aus dem Kapitel Text setzen bauen, aber das Paket pdf4tclflow nimmt einem die Spaltenrechnung ab. Es füllt Spalte um Spalte und Seite um Seite, bis der Text aufgebraucht ist. Das Wasserzeichen liegt im Kern von pdf4tcllib.

7.2 Text in Spalten

```
package require pdf4tclflow
set r [pdf4tcllib::flow::columns pdf ctx text ?option wert ...?]
```

Der Text wird an Leerzeilen in Absätze geteilt; Zeilenumbrüche und mehrfache Leerzeichen innerhalb eines Absatzes werden zu einem Leerzeichen. Zwischen zwei Absätzen bleibt eine Leerzeile, außer oben in einer Spalte.

Option	Vorgabe	Bedeutung
-columns	2	Zahl der Spalten
-gap	18	Abstand zwischen den Spalten in Punkt
-size	10	Schriftgröße

Option	Vorgabe	Bedeutung
-font	aus fonts::init	Schrift
-top, -bottom	aus dem Kontext	wo jede Spalte beginnt und endet
-firsty	-top	Anfang der ersten Spalte, etwa unter einer Überschrift
-newpage	–	Skript für den Seitenwechsel
-tag	P	Strukturtyp für barrierefreie PDFs, "" für keinen

Die Spalten sind gleich breit und füllen den Satzspiegel; bei zwei Spalten auf A4 mit 20 mm Rand sind das je 232 Punkt (82 mm). flow::boxes liefert die Lage jeder Spalte als `{x breite}`, für ein Bild in Spalte zwei oder eine Linie zwischen den Spalten.

```
# 070-spalten.tcl -- zwei Spalten ueber mehrere Seiten, Wasserzeichen
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclflow

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Ein langer Text; Absaetze durch eine Leerzeile getrennt
set absatz "Die Inventur beginnt am ersten Werktag des Monats. Jedes\
    Regal wird gezählt, die Zahl mit der Buchung verglichen und die\
    Abweichung notiert. Kleine Abweichungen werden sofort ausgebucht,\
    größere gehen an die Lagerleitung."
set text [join [lrepeat 30 $absatz] "\n\n"]

# erste Grundlinie, im Satzspiegel
set oben [expr {[dict get $ctx top] + 10}]

$pdf startPage
pdf4tcllib::drawing::watermark $pdf $ctx "ENTWURF" -size 110
$pdf setFont 16 [pdf4tcllib::fonts::fontSansBold]
pdf4tcllib::unicode::safeText $pdf "Inventur im Lager Nord" \
    -x [dict get $ctx left] -y [expr {$oben + 6}]

set r [pdf4tcllib::flow::columns $pdf $ctx $text -columns 2 -size 10 \
    -top $oben -firsty [expr {$oben + 36}] -newpage {
        pdf4tcllib::page::number $pdf $ctx [$pdf currentPage]
        $pdf endPage
        $pdf startPage
        pdf4tcllib::drawing::watermark $pdf $ctx "ENTWURF" -size 110
        pdf4tcllib::page::header $pdf $ctx "Inventur im Lager Nord"
    }]
```

```
pdf4tcllib::page::number $pdf $ctx [$pdf currentPage]
$pdf endPage
puts "Ergebnis: $r"

# Ohne -newpage: was nicht passt, kommt in "rest" zurueck
$pdf startPage
set r2 [pdf4tcllib::flow::columns $pdf $ctx $text \
    -columns 3 -top $oben]
$pdf endPage
puts "ohne -newpage: [dict get $r2 lines] Zeilen gesetzt,\
    [llength [split [dict get $r2 rest] \n]] Zeilen in rest"

$pdf write -file 070-spalten.pdf
$pdf destroy
```

```
Ergebnis: column 1 y 472.6929133858268 pages 1 lines 150 rest {}
ohne -newpage: 136 Zeilen gesetzt, 116 Zeilen in rest
```

Das Ergebnis ist ein Dict:

Schlüssel	Bedeutung
column	Spalte, in der der Text endet (ab 0)
y	y unter der letzten Zeile
pages	Zahl der Seitenwechsel
lines	gesetzte Zeilen
rest	was nicht gesetzt wurde, leer wenn alles passte

7.2.1 Der Seitenwechsel gehört dem Aufrufer

Ist die letzte Spalte voll, ruft `columns` das Skript aus `-newpage` im Bereich des Aufrufers auf. Es muss die alte Seite schließen und eine neue öffnen, und alles hineinschreiben, was eine neue Seite braucht: Seitenzahl, Kopfzeile, Wasserzeichen. Danach setzt `columns` Schrift und Größe selbst wieder.

Ohne -newpage hört columns auf, wenn die Spalten der Seite voll sind, und gibt den Rest in `rest` zurück. Im Beispiel passen in drei Spalten 136 Zeilen auf die Seite; 116 Zeilen samt den Leerzeilen zwischen den Absätzen stehen in `rest`. Wer `rest` nicht prüft, verliert den Rest des Texts ohne Meldung.

7.2.2 Die erste Grundlinie

`columns` setzt die erste Zeile jeder Spalte mit ihrer Grundlinie auf `-top`. Ohne Angabe ist das `top` aus dem Kontext, und die Oberlängen ragen in den Rand. Das Beispiel gibt `-top` darum 10 Punkt tiefer an.

`-firsty` gilt nur für die erste Spalte der ersten Seite. Die zweite Spalte daneben beginnt wieder bei `-top`. Eine Überschrift über beide Spalten braucht darum `-firsty` **und** genug Platz in der zweiten Spalte; im Beispiel steht sie nur über der ersten.

7.2.3 Was nicht geht

- `-align` steht in den Vorgaben (`flow::configure`), wirkt aber nicht: Jede Zeile wird linksbündig gesetzt.
- Schrift, Größe und Zeilenabstand gelten für den ganzen Text. Eine Zwischenüberschrift im Fluss geht nur, indem man den Text aufteilt und `columns` mehrmals aufruft, mit `-firsty` aus dem y des vorigen Aufrufs.
- Der Text geht durch `safeText`; typografische Anführungszeichen werden gerade (Kapitel Unicode sicher ausgeben).

Mehr Spalten sparen keine Seiten. Schmale Spalten brechen öfter um und lassen am Zeilenende mehr frei. Spalten sind eine Frage der Lesbarkeit: Eine Zeile mit 60 bis 80 Zeichen liest sich leichter als eine mit 130.

`flow::configure` ändert die Vorgaben für alle späteren Aufrufe im Programm. Eine Bibliothek, die `columns` benutzt, gibt ihre Optionen darum beim Aufruf mit, statt die Vorgaben ihres Aufrufers zu ändern.

7.3 Wasserzeichen

```
pdf4tcllib::drawing::watermark pdf ctx text ?-size pt? ?-angle grad? \
    ?-color {r g b}? ?-font name?
```

drawing::watermark schreibt einen Text in Grau schräg über die Seitenmitte. Es gibt die verwendete Schriftgröße zurück.

Option	Vorgabe	Bedeutung
-size	0	0: so groß, dass der Text die Diagonale weitgehend füllt
-angle	45	Winkel gegen den Uhrzeigersinn
-color	{0.86 0.86 0.86}	Grauton
-font	aus fonts::init	Schrift

Zuerst zeichnen. pdf4tcl kennt keine Transparenz; das Wasserzeichen ist deckendes Hellgrau. Was danach kommt, liegt darüber. Im Beispiel steht es darum direkt nach `startPage`, auch im `-newpage`-Skript.

Mit `-size 0` wird "ENTWURF" auf A4 etwa 180 Punkt groß und reicht von der linken bis fast an die rechte Papierkante. Eine feste Größe um 110 Punkt lässt Rand und wirkt ruhiger.

Das Wasserzeichen ist für Seiten mit `-orient 1` gebaut. Mit `-orient 0` landet es außerhalb der Seite.

`pdftotext` gibt gedrehten Text verwürfelt zurück. Ob das Wasserzeichen richtig steht, sieht man nur in einem Betrachter.

7.4 Zusammenfassung

- `flow::columns` setzt Text in Spalten über Seiten; den Seitenwechsel liefert der Aufrufer als `-newpage`-Skript.
- Ohne `-newpage` steht der Rest in `rest`; immer prüfen.

- `-top` eine Zeile unter `top`, sonst ragt die erste Zeile in den Rand.
- `-align` wirkt nicht; Spalten sparen keine Seiten.
- `drawing::watermark` zuerst auf jeder Seite, mit fester Größe um 110 Punkt, nur mit `-orient 1`.

Nächstes Kapitel: Tabellen mit Seitenumbruch

8 Tabellen mit Seitenumbruch

8.1 Einführung

Eine Tabelle ist in pdf4tcl eine Reihe von Rechtecken, Linien und Texten, die man selbst ausrechnet. Bei einer Seite geht das, bei einer Liste unbekannter Länge nicht mehr: Irgendwo ist die Seite zu Ende, die Kopfzeile muss auf der nächsten wiederholt werden, und die Summe gehört ans Ende. table::draw erledigt das aus einfachen Daten: einer Liste von Spalten und einer Liste von Zeilen.

pdf4tcllib hat daneben ältere Tabellenprozeduren (`table::render`, `table::simpleTable`). Dieses Buch benutzt nur `table::draw`; es ist die Prozedur, auf der auch der Export von `tablelist`-Widgets aufbaut.

8.2 Spalten und Zeilen

```
set y [pdf4tcllib::table::draw pdf x y spalten zeilen \
    ?option wert ...?]
```

Spalten sind eine Liste, je Spalte eine Optionsliste:

Option	Vorgabe	Bedeutung
-header	–	Text der Kopfzelle
-width	auto	Breite in Punkt oder auto (nach dem breitesten Inhalt)
-align	left	left, center, right
-font	reg	reg, bold, italic, bolditalic, mono
-notruncate	0	1: Text dieser Spalte nie kürzen (siehe unten)

Zeilen sind eine Liste von Listen mit reinem Text. Formatierung steht nicht in den Daten, sondern in eigenen Optionen, die Zeile und Spalte nach Nummer ansprechen (ab 0, ohne Kopfzeile).

`table::draw` gibt die y-Position unter der Tabelle zurück.

8.3 Eine Liste über zwei Seiten

```
# 080-tabelle.tcl -- eine Artikelliste ueber mehrere Seiten
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Daten: 70 Artikel, jede Zeile eine Liste von Texten
set data {}
for {set i 1} {$i <= 70} {incr i} {
    set preis [expr {($i * 37 % 500) + 0.5}]
    set text "Schraube M$i, verzinkt, Packung zu 100 Stück"
    lappend data [list [format "A-%04d" $i] $text \
        [expr {$i * 3 % 40}] [format %.2f $preis]]
}
set summe 0.0
foreach z $data { set summe [expr {$summe + [lindex $z 3]}] }

set cols {
    {-header "Nr."         -width auto}
    {-header "Bezeichnung" -width auto}
    {-header "Bestand"     -width 50 -align right}
    {-header "Preis €"     -width 60 -align right}
}

# Seitenwechsel: laeuft im globalen Bereich und muss die neue
# y-Position liefern
proc tabellenUmbruch {} {
    global pdf ctx
    pdf4tcllib::page::footer $pdf $ctx "Artikelliste" \
        [$pdf currentPage]
    $pdf endPage
    $pdf startPage
    pdf4tcllib::page::header $pdf $ctx "Artikelliste (Fortsetzung)"
    return [dict get $ctx top]
}

$pdf startPage
$pdf setFont 16 [pdf4tcllib::fonts::fontSansBold]
pdf4tcllib::unicode::safeText $pdf "Artikelliste" \
    -x [dict get $ctx left] -y [expr {[dict get $ctx top] + 16}]

set y [pdf4tcllib::table::draw $pdf [dict get $ctx left] \
    [expr {[dict get $ctx top] + 30}] $cols $data \
    -ctx $ctx -zebra 1 -fontsize 9 \
    -pagebreakcmd tabellenUmbruch \
```

```
    -cellstyles {4,2 {-fg {0.8 0 0} -font bold}} \
    -footer [list "" "Summe" "" [format %.2f $summe]]]

pdf4tcllib::page::footer $pdf $ctx "Artikelliste" [$pdf currentPage]
$pdf endPage
puts "Seiten: [$pdf pageCount], Tabelle endet bei y = [format %.1f $y]"
$pdf write -file 080-tabelle.pdf
$pdf destroy
```

```
Seiten: 2, Tabelle endet bei y = 542.7
```

Die Kopfzeile steht auf beiden Seiten, die Summe am Ende, die Zeile `A-0005` hat einen roten, fetten Bestand.

8.3.1 Was der Umbruch braucht

- **-ctx** schaltet den Umbruch ein. Ohne Kontext zeichnet `table::draw` alle Zeilen auf die aktuelle Seite, auch über den Rand hinaus.
- **-pagebreakcmd** ist der eigene Seitenwechsel: Fußzeile der alten Seite, neue Seite, Kopfzeile. Die Prozedur muss die y-Position zurückgeben, an der die Tabelle weitergeht.

Zwei Dinge sind anders als bei `-newpage` von `flow::columns`:

- Der Befehl läuft im **globalen** Bereich, nicht im Bereich des Aufrufers. Lokale Variablen der aufrufenden Prozedur sieht er nicht; darum steht im Beispiel `global pdf ctx`.
- Er muss die neue y-Position **zurückgeben**. Gibt er nichts zurück, nimmt `table::draw top` aus dem Kontext.

Ohne `-pagebreakcmd` bricht `table::draw` selbst um: Es schreibt unten rechts `- 1 -` auf die alte Seite und beginnt eine neue, ohne Kopf- und Fußzeile. Die letzte Seite bekommt keine Nummer. Für einen Bericht ist das selten richtig; `-pagebreakcmd` ist der Normalfall.

8.4 Aussehen

Option	Vorgabe	Bedeutung
-fontsize	9	Schriftgröße
-rowheight	0	Zeilenhöhe; 0 heißt 1,8 × Schriftgröße
-pad	4	Innenabstand der Zellen
-border	1	Linien zeichnen
-header	1	Kopfzeile zeichnen
-headerbg, -headerfg	hellblau, schwarz	Farben der Kopfzeile
-zebra	0	jede zweite Zeile hinterlegen; -zebracolor
-maxwidth	text_w aus -ctx, sonst 480	größte Breite der Tabelle
-footer	–	Summenzeile, je Spalte ein Wert, fett

`-width auto` misst den breitesten Inhalt der Spalte samt Kopf. Feste Breiten gelten wie angegeben. Sind alle Breiten zusammen größer als `-maxwidth`, werden sie im selben Verhältnis verkleinert. Breiter als nötig wird eine Tabelle nie; eine über den ganzen Satzspiegel braucht feste Breiten, die zusammen `text_w` ergeben.

8.4.1 Einzelne Zeilen und Zellen

```
-rowstyles  {3 {-bg {1 0.95 0.8} -font bold}}
-cellstyles {4,2 {-fg {0.8 0 0} -font bold} 7,1 {-align right}}
-rowindent  {1 14 2 14}
```

`-rowstyles` nimmt `-bg`, `-fg` und `-font`, `-cellstyles` zusätzlich `-align` und `-notruncate`. Die Zelle schlägt die Zeile, die Zeile das Zebramuster. `-rowindent` rückt die erste Spalte einer Zeile ein, etwa für Unterpositionen unter einer fetten Gruppenzeile.

8.5 Eine Zeile, ein Wert: kürzen

Eine Zeile in `table::draw` ist **eine** Textzeile hoch. Was breiter ist als die Spalte, wird gekürzt und endet auf "...", auch in der Kopfzeile. Es gibt keinen Umbruch innerhalb einer Zelle.

Für Bemerkungen und Artikelnamen ist das in Ordnung. Für Werte, die jemand abtippt (Kennwörter, Seriennummern, Kontonummern), ist es gefährlich: Das Blatt sieht vollständig aus. Für solche Spalten `-notruncate 1`, für einzelne Zellen dasselbe in `-cellstyles`. Dann steht der ganze Wert da, notfalls über den Spaltenrand hinaus. Die Spalte muss also breit genug sein; `table::widths` sagt vorher, wie breit (Abschnitt Wie breit wird die Tabelle?).

```
Text                          Betrag
Kennwort: Xk9-7Rt-22pQ-...      1.00      ohne -notruncate
Kennwort: Xk9-7Rt-22pQ-88aZ     1.00      mit -notruncate
```

8.6 Verbundene Zellen

Eine Zelle darf statt eines Textes ein Dict mit `text` und `colspan` oder `rowspan` sein:

```
set zeilen [list \
    [list [dict create text Nord rowspan 2] Q1 120] \
    [list Q2 90] \
    [list [dict create text Summe colspan 2] 210]]
```

```
R       Q     W
        Q1   120
Nord
        Q2    90
Summe        210
```

Mit `colspan` reicht die Zelle über mehrere Spalten, und die Zeile hat entsprechend weniger Elemente. Mit `rowspan` hält sie ihre Spalte in den folgenden Zeilen; dort fehlt sie wie in HTML einfach: `Q2` ist das erste Element seiner Zeile und steht doch in der zweiten Spalte. Das nächste Kapitel benutzt `colspan` für Zwischensummen.

Was `table::draw` dabei tut:

- Durch eine verbundene Zelle läuft keine Linie, und eine Zelle mit `rowspan` hat den Hintergrund der Zeile, in der sie beginnt, über ihre ganze Höhe. Ihr Text steht in der Mitte des Blocks.
- Zeilen, die `rowspan` verbindet, werden nicht über zwei Seiten getrennt; der ganze Block rückt auf die nächste Seite. Ist er höher als eine Seite, bricht `table::draw` mit einem Fehler ab.
- Im barrierefreien PDF trägt die Zelle `/ColSpan` bzw. `/RowSpan`.

Bis pdf4tcllib 0.6.6 kannte `table::draw` kein `rowspan` (die folgende Zeile rutschte um eine Spalte nach links) und zog die Linien auch durch eine verbundene Zelle ("Summe No|rd").

8.7 Wie breit wird die Tabelle?

Wer vor dem Zeichnen wissen will, wie breit eine Tabelle wird oder ob sie neben ein Bild passt, fragt table::widths:

```
set b [pdf4tcllib::table::widths $cols $data -fontsize 9 -pdf $pdf]
dict get $b natural       ;# je Spalte: so breit, dass nichts fehlt
dict get $b min           ;# je Spalte: das längste Wort
dict get $b naturalTotal  ;# Summe
```

Und wie viele Zeilen auf einen Rest der Seite passen, table::fitRows:

```
set platz [expr {[dict get $ctx bottom] - $y}]
set n [pdf4tcllib::table::fitRows $platz $data -fontsize 9]
```

Damit lässt sich eine Tabelle selbst aufteilen, etwa um nach den ersten `n` Zeilen eine Bemerkung einzuschieben.

8.8 Was man wissen sollte

- **Unbekannte Optionen sind ein Fehler.** `-zebar 1` statt `-zebra 1` bricht mit "unknown option -zebar" und der Liste der erlaubten Optionen ab. Bis pdf4tcllib 0.6.6 ging der Tippfehler still durch: keine Streifen, keine Meldung.
- **-orient 0 geht nicht.** `table::draw` liest die Richtung aus dem Dokument und bricht bei y nach oben mit einem Fehler ab.
- **Zahlen formatiert der Aufrufer.** `table::draw` schreibt den Text so, wie er in den Daten steht; `format %.2f` gehört davor.
- **Text geht durch sanitize.** Die Anführungszeichen "" werden gerade (Kapitel Unicode sicher ausgeben).

8.9 Zusammenfassung

- `table::draw` mit Spalten und reinen Textzeilen; Formatierung über `-rowstyles` und `-cellstyles`.
- Für mehrere Seiten `-ctx` und `-pagebreakcmd`; der Befehl läuft global und gibt die neue y-Position zurück.
- Zellen sind einzeilig und werden gekürzt; `-notruncate` für Werte, die abgetippt werden.
- Verbundene Zellen mit `colspan` und `rowspan`; ein `rowspan`-Block bleibt auf einer Seite.

Nächstes Kapitel: Gruppensummen und Kreuztabellen

9 Gruppensummen und Kreuztabellen

9.1 Einführung

Eine Liste von Buchungen wird selten so gedruckt, wie sie aus der Datenbank kommt. Gefragt ist die Summe je Region, je Kunde, je Monat, oder eine Übersicht mit Regionen als Zeilen und Quartalen als Spalten. table::group und table::pivot rechnen das aus und geben Zeilen zurück, die `table::draw` direkt zeichnet. Gezeichnet wird dabei nichts; beide arbeiten nur auf den Daten.

9.2 Das Beispiel

Neun Umsätze, nach Region sortiert. Ein Feld ist leer: Für West im zweiten Quartal liegt kein Wert vor.

```
# 090-gruppen.tcl -- Zwischensummen je Gruppe und eine Kreuztabelle
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Umsatz je Region, Quartal und Vertreter -- nach Region sortiert
set umsatz {
    {Nord Q1 Albers 1200}  {Nord Q2 Albers 1350}  {Nord Q2 Brandt 400}
    {Nord Q4 Brandt 980}   {Süd  Q1 Celik  2100}
    {Süd  Q3 Celik  1875.5}
    {West Q1 Dorn   640}   {West Q2 Dorn   ""}    {West Q3 Dorn   720}
}

# Zahlen fuer die Ausgabe formatieren; leere Felder bleiben leer
proc betrag {v} {
    if {[string is double -strict $v]} { return [format %.2f $v] }
    return $v
}

# 1. Gruppieren: Zwischensumme je Region, Gesamtsumme am Ende
set g [pdf4tcllib::table::group $umsatz -by 0 -sum {3} \
    -label "Summe %s" -totallabel "Gesamt" -blank 1]
puts "Zwischensummen in Zeile: [dict get $g subtotals],\
    Gesamt: [dict get $g total]"

set zeilen [lmap z [dict get $g rows] {
    lset z end [betrag [lindex $z end]]
    set z
```

```
}]
set stile {}
foreach i [dict get $g subtotals] {
    lappend stile $i {-font bold -bg {0.93 0.93 0.93}}
}
lappend stile [dict get $g total] {-font bold -bg {0.85 0.85 0.85}}

$pdf startPage
set x [dict get $ctx left]
set y [expr {[dict get $ctx top] + 10}]
set y [pdf4tcllib::table::draw $pdf $x $y {
    {-header Region   -width 80}
    {-header Quartal  -width 50}
    {-header Vertreter -width 80}
    {-header Umsatz   -width 70 -align right}
} $zeilen -rowstyles $stile]

# 2. Kreuztabelle: Regionen als Zeilen, Quartale als Spalten
set p [pdf4tcllib::table::pivot $umsatz -row 0 -col 1 -value 3 \
    -rowlabel Region -colorder {Q1 Q2 Q3 Q4}]
puts "Kopf: [dict get $p header]"
foreach z [dict get $p rows] { puts "  $z" }

set kopf [lmap h [dict get $p header] {
    set a [expr {$h eq "Region" ? "left" : "right"}]
    list -header $h -width 60 -align $a
}]
set zeilen [lmap z [dict get $p rows] { lmap v $z { betrag $v } }]
pdf4tcllib::table::draw $pdf $x [expr {$y + 30}] $kopf $zeilen \
    -rowstyles [list [dict get $p total] {-font bold}]

$pdf endPage
$pdf write -file 090-gruppen.pdf
$pdf destroy
```

```
Zwischensummen in Zeile: 4 7 11, Gesamt: 12
Kopf: Region Q1 Q2 Q3 Q4 Gesamt
  Nord 1200 1750 {} 980 3930
  Süd 2100 {} 1875.5 {} 3975.5
  West 640 {} 720 {} 1360
  Gesamt 3940 1750 2595.5 980 9265.5
```

Im PDF steht die gruppierte Liste mit hervorgehobenen Summenzeilen und darunter die Kreuztabelle:

```
Region      Quartal  Vertreter     Umsatz
Nord        Q1       Albers       1200.00
            Q2       Albers       1350.00
            Q2       Brandt        400.00
            Q4       Brandt        980.00
Summe Nord                        3930.00
Süd         Q1       Celik        2100.00
...
Gesamt                            9265.50

Region        Q1       Q2       Q3      Q4   Gesamt
Nord     1200.00  1750.00           980.00  3930.00
Süd      2100.00           1875.50          3975.50
West      640.00            720.00          1360.00
Gesamt   3940.00  1750.00  2595.50  980.00  9265.50
```

9.3 table::group

```
set g [pdf4tcllib::table::group zeilen ?option wert ...?]
```

Option	Vorgabe	Bedeutung
-by	0	Spalte, nach der gruppiert wird
-sum	–	Liste der Spalten, die summiert werden
-label	Summe %s	Text der Zwischensumme; %s wird der Gruppenwert
-total	1	Gesamtsumme anhängen (nur mit -sum)
-totallabel	Gesamt	Text der Gesamtsumme
-header	0	1: vor jeder Gruppe eine Zeile mit dem Gruppenwert
-blank	0	1: den Gruppenwert nur in der ersten Zeile der Gruppe zeigen
-strict	0	1: bei unsortierten Daten einen Fehler werfen

Das Ergebnis ist ein Dict:

Schlüssel	Inhalt
rows	die Zeilen mit eingefügten Zwischensummen
subtotals	die Nummern der Zwischensummenzeilen
groups	die Nummern der Gruppenkopfzeilen (mit -header 1)
total	die Nummer der Gesamtsummenzeile, sonst -1

Die Nummern passen direkt zu `-rowstyles` von `table::draw`; so werden im Beispiel die Summen fett und grau hinterlegt.

9.3.1 Die Daten müssen sortiert sein

`group` sortiert nicht selbst; die Reihenfolge der Zeilen bleibt, wie sie kommt. Stehen dieselben Werte nicht beieinander, zerfällt eine Gruppe in mehrere Blöcke, und jeder bekommt seine eigene Zwischensumme. `group` merkt das, wirft aber nur mit `-strict 1` einen Fehler. Ohne landet die Warnung in `$::pdf4tcl::warnings`, wo sie nur sieht, wer nachschaut. Darum vorher sortieren:

```
set umsatz [lsort -index 0 $umsatz]
```

9.3.2 Leere Felder sind keine Null

Ein Wert, der keine Zahl ist (leer, "k. A."), wird beim Summieren übersprungen, nicht als 0 gezählt. Bei einer Summe macht das keinen Unterschied; bei einem Durchschnitt in der Kreuztabelle schon.

9.3.3 Die Beschriftung reicht über mehrere Spalten

Die Zwischensumme beginnt mit einer verbundenen Zelle (`colspan`), die bis zur ersten Summenspalte reicht. Seit pdf4tcllib 0.6.7 zieht `table::draw` keine Linie mehr hindurch (Kapitel Tabellen mit Seitenumbruch). Mit älteren Versionen muss die erste Spalte so breit sein wie die längste Beschriftung, im Beispiel 80 Punkt für "Summe West", sonst läuft die Linie durch den Text.

9.3.4 Summen formatieren

`group` und `pivot` rechnen mit `expr`. Was herauskommt, ist eine Zahl in Tcl-Schreibweise: `3930`, `3975.5`, bei manchen Beträgen auch `1234.5000000000002`. Für die Ausgabe gehört ein `format` davor; im Beispiel macht das `betrag` für jede Zelle, die eine Zahl ist. Ein Dezimalkomma setzt keines der beiden Pakete:

```
proc betrag {v} {
    if {![string is double -strict $v]} { return $v }
    string map {. ,} [format %.2f $v]
}
```

9.4 table::pivot

```
set p [pdf4tcllib::table::pivot zeilen ?option wert ...?]
```

Option	Vorgabe	Bedeutung
-row	0	Spalte, deren Werte die Zeilen werden
-col	1	Spalte, deren Werte die Spalten werden
-value	2	Spalte mit den Zahlen
-agg	sum	sum, count, avg, min, max
-empty	""	was in einem Feld ohne Wert steht
-colorder	sortiert	Reihenfolge der Spalten
-rowlabel	""	Kopf der ersten Spalte

Option	Vorgabe	Bedeutung
-total	1	Summenspalte und Summenzeile anhängen
-totallabel	Gesamt	deren Beschriftung

Das Ergebnis enthält `header` (die Kopfzeile), `rows`, `cols` (die Spaltenwerte ohne Kopf und Summe) und `total` (Nummer der Summenzeile). Die Kopfzeile ist eine Liste von Texten; für `table::draw` muss daraus eine Spaltenbeschreibung werden, im Beispiel mit `lmap`.

Drei Entscheidungen trifft jede Kreuztabelle, und `pivot` überlässt sie dem Aufrufer:

- **Welche Spalten.** Sie ergeben sich aus den Daten. Ein Quartal ohne einen einzigen Wert fehlt; wer alle vier sehen will, gibt `-colorder {Q1 Q2 Q3 Q4}` an. Ohne `-colorder` wird alphabetisch sortiert, aus Monatsnamen wird dann "Apr Aug Dez ...".
- **Was in einer leeren Zelle steht.** Vorgabe ist nichts, nicht 0. "Kein Umsatz" und "keine Angabe" sind verschiedene Aussagen; wer 0 will, setzt `-empty 0`.
- **Was bei mehreren Werten je Feld geschieht.** Nord hat in Q2 zwei Umsätze (Albers und Brandt); `-agg sum` addiert sie zu 1750.

Mit `-agg avg` ist die Summenzeile der Durchschnitt über alle Werte der Spalte, nicht die Summe der Durchschnitte.

9.5 Zusammenfassung

- `table::group` fügt Zwischensummen ein und sagt, in welchen Zeilen sie stehen; die Nummern gehen an `-rowstyles`.
- Vorher nach der Gruppenspalte sortieren, oder `-strict 1`.
- `table::pivot` für Kreuztabellen; `-colorder` für die Spalten, `-empty` für leere Felder.
- Zahlen vor dem Zeichnen formatieren, auch das Dezimalkomma.

Nächstes Kapitel: Diagramme

10 Diagramme

10.1 Einführung

Zu vielen Berichten gehört ein Bild der Zahlen: Umsatz je Monat als Balken, zwei Jahre als Linien, Anteile als Torte. Das Paket pdf4tclchart zeichnet diese drei Formen direkt mit pdf4tcl, ohne Tk und ohne ein Bild zu erzeugen. Es ist keine Bibliothek für wissenschaftliche Diagramme: keine logarithmischen Achsen, keine Fehlerbalken, keine zweite y-Achse.

10.2 Daten und Kasten

Jede der drei Prozeduren bekommt einen Kasten und die Daten:

```
package require pdf4tclchart
set y [pdf4tcllib::chart::bar  pdf x y breite hoehe daten ?opt ...?]
set y [pdf4tcllib::chart::line pdf x y breite hoehe daten ?opt ...?]
set y [pdf4tcllib::chart::pie  pdf x y breite hoehe daten ?opt ...?]
```

`x y` ist die linke obere Ecke (y nach unten), Breite und Höhe in Punkt. Zurück kommt die

y-Position unter dem Diagramm, so dass Diagramme sich wie Absätze untereinander stapeln.

Die Daten sind Paare aus Beschriftung und Zahl, flach oder als Liste von Paaren:

```
{Jan 120 Feb 145 Mär 98}
{{Jan 120} {Feb 145} {Mär 98}}
```

Ein Wert, der keine Zahl ist, ergibt einen Fehler, der die Beschriftung nennt. Leere Werte sind hier also nicht erlaubt, anders als in `table::pivot`.

10.3 Drei Diagramme auf einer Seite

```
# 100-diagramme.tcl -- Balken, Linien und Torte auf einer Seite
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage

set x [dict get $ctx left]
set w [dict get $ctx text_w]
set y [expr {[dict get $ctx top] + 10}]

# Balken: Paare aus Beschriftung und Wert
set y [pdf4tcllib::chart::bar $pdf $x $y $w 180 \
    {Jan 120 Feb 145 Mär 98 Apr 160 Mai -20 Jun 132} \
    -title "Ergebnis je Monat (Tsd. €)" -values 1 -format %.0f]

# Linien: mehrere Reihen, je {Name {Paare}}
set reihen {
    {2025 {Q1 100 Q2 130 Q3 120 Q4 175}}
    {2026 {Q1 140 Q2 120 Q3 165 Q4 190}}
}
set oben [expr {$y + 20}]
set y [pdf4tcllib::chart::line $pdf $x $oben [expr {$w - 80}] 170 \
    $reihen -title "Umsatz je Quartal"]
# Die Linien haben keine eigene Legende; die Reihen selbst beschriften
pdf4tcllib::chart::legend $pdf \
    [expr {$x + $w - 70}] [expr {$oben + 30}] \
    {{2025 0} {2026 0}} [pdf4tcllib::chart::configure]

# Torte mit Legende und Anteilen
set y [pdf4tcllib::chart::pie $pdf $x [expr {$y + 20}] $w 170 \
    {Wartung 42 Neugeräte 31 Ersatzteile 18 Schulung 9} \
    -title "Umsatz nach Bereich" -legend 1]
```

```
puts "Skala für 137: [pdf4tcllib::chart::niceScale 0 137]"
puts "unten bei y = [format %.1f $y]"
$pdf endPage
$pdf write -file 100-diagramme.pdf
$pdf destroy
```

```
Skala für 137: 0.0 150.0 50.0
unten bei y = 626.7
```

10.3.1 Balken

Ein Balken je Paar, jeder in einer anderen Farbe. Negative Werte stehen unter der Nulllinie, die Skala reicht dann ins Negative (im Beispiel Mai mit −20). `-values 1` schreibt den Wert über jeden Balken, im Format aus `-format`.

10.3.2 Linien

Eine Reihe oder mehrere. Mehrere Reihen sind eine Liste aus `{Name {Paare}}`; die Beschriftungen der x-Achse kommen aus der ersten Reihe, die anderen werden an denselben Stellen gezeichnet. Jeder Datenpunkt bekommt einen Punkt.

Die Namen der Reihen (`2025`, `2026`) zeichnet `line` nicht, und eine Legende hat es nicht. Ohne Legende weiß der Leser nicht, welche Linie welches Jahr ist. Das Beispiel macht darum das Diagramm schmaler und zeichnet daneben eine Legende mit chart::legend, in denselben Farben:

```
pdf4tcllib::chart::legend $pdf $x $y {{2025 0} {2026 0}} \
    [pdf4tcllib::chart::configure]
```

Das vierte Argument sind die Paare, das fünfte die Optionen (für Größe und Farben); `chart::configure` ohne Argumente liefert die Vorgaben.

10.3.3 Torte

Die Stücke laufen im Uhrzeigersinn ab zwölf Uhr, die Anteile rechnet `pie` aus der Summe. Mit `-legend 1` steht rechts eine Legende mit Prozentangaben, und der Kreis wird entsprechend kleiner. Negative Werte oder eine Summe von 0 sind ein Fehler.

10.4 Optionen

Option	Vorgabe	Bedeutung
-title	–	Überschrift über dem Diagramm
-titlesize, -labelsize, -valuesize	11, 8, 8	Schriftgrößen
-values	0	Werte an die Balken schreiben
-format	%g	Format jeder Zahl, auch an der Achse
-colors	sechs Farben	Liste von Farben, der Reihe nach
-color	–	eine Farbe für alles
-grid	4	Zahl der Gitterlinien; 0 für keine

Option	Vorgabe	Bedeutung
-min, -max	aus den Daten	Skala festlegen
-legend	0	nur pie
-alt	-title	Ersatztext im barrierefreien PDF

Die sechs Farben der Vorgabe unterscheiden sich auch in der Helligkeit, nicht nur im Farbton. Ein Diagramm, das grau gedruckt oder kopiert wird, bleibt so lesbar. Wer eigene Farben wählt, sollte das beibehalten.

10.4.1 Die Skala

Die Achse endet nicht beim größten Wert, sondern bei einer runden Zahl: chart::niceScale macht aus 137 die Skala 0 bis 150 in Schritten von 50. Eine Gitterlinie bei 137 hilft niemandem, der den Wert ablesen will. Für zwei Diagramme, die man vergleichen soll, beiden dieselbe Skala mit `-min` und `-max` geben; sonst sieht ein kleiner Wert im einen so groß aus wie ein großer im anderen.

10.4.2 Zahlen mit Komma

`-format` geht an Tcls `format` und kennt kein Dezimalkomma. `%.0f` und `%d` sind darum für deutsche Berichte die sichere Wahl; Nachkommastellen erscheinen mit Punkt.

10.5 Barrierefrei

In einem barrierefreien PDF (Teil 5) ist jedes Diagramm ein Element `Figure` mit dem Ersatztext aus `-alt` oder dem Titel. Balken, Linien und Beschriftungen sind der Inhalt dieses Elements, und der Ersatztext steht für sie. Ein Vorleseprogramm sagt also "Abbildung: Ergebnis je Monat" und nicht die einzelnen Zahlen. Sind die Zahlen wichtig, gehören sie zusätzlich in eine Tabelle.

10.6 Zusammenfassung

- `chart::bar`, `chart::line`, `chart::pie` mit Kasten und Paaren; jede gibt die y-Position darunter zurück.
- Mehrere Linien brauchen eine eigene Legende über `chart::legend`.
- Skala mit `-min`/`-max` festlegen, wenn Diagramme verglichen werden.
- `-format` ohne Dezimalkomma.

Nächstes Kapitel: Inhaltsverzeichnis

11 Inhaltsverzeichnis

11.1 Einführung

Ein Inhaltsverzeichnis steht vorn, kennt aber Seitenzahlen, die erst feststehen, wenn alles dahinter gesetzt ist. Und es verschiebt diese Seitenzahlen selbst, um so viele Seiten, wie es lang ist. Ohne zwei Durchläufe geht es nicht. Das Paket pdf4tcltoc macht beide Durchläufe und schreibt dazu je Überschrift ein Lesezeichen.

11.2 Der Inhalt als Skript

```
package require pdf4tcltoc
set r [pdf4tcllib::toc::document pdf papier skript ?option wert ...?]
```

Der Inhalt wird nicht direkt gezeichnet, sondern als Skript übergeben, denn toc::document muss ihn zweimal ausführen:

1. in ein Wegwerf-Dokument; dabei merkt sich jede Überschrift, auf welcher Seite sie landet;
2. in das echte Dokument, nachdem das Verzeichnis geschrieben ist.

Im Skript gibt es zwei Variablen: `pdf` (im ersten Lauf das Wegwerf-Dokument) und `ctx`. Das Skript beginnt und beendet seine Seiten selbst. Überschriften setzt es mit toc::heading; nur was so gesetzt ist, kommt ins Verzeichnis.

```
# 110-inhaltsverzeichnis.tcl -- Verzeichnis mit echten Seitenzahlen
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcltoc

pdf4tcllib::fonts::init -cid 1

set kapitel {
    {1 "Einleitung"} {2 "Zweck des Berichts"} {2 "Datenquellen"}
    {1 "Bestand"}    {2 "Lager Nord"} {2 "Lager Süd"}
    {1 "Ausblick"}
}
set absatz "Der Bestand wurde am Monatsende gezählt und mit der\
    Buchung abgeglichen. Abweichungen unter zwei Stück sind\
    ausgebucht."

# Der Inhalt als Skript: es sieht die Variablen pdf und ctx,
# beginnt seine Seiten selbst und laeuft zweimal -- also keine
# Nebenwirkungen.
set inhalt {
    $pdf startPage
    set y [dict get $ctx top]
    foreach k $kapitel {
        lassign $k ebene titel
        if {$ebene == 1 && $y > [dict get $ctx top]} {
            $pdf endPage; $pdf startPage; set y [dict get $ctx top]
        }
        pdf4tcllib::toc::heading $pdf $ctx y $ebene $titel
        for {set i 0} {$i < 12} {incr i} {
            if {$y > [dict get $ctx bottom] - 40} {
                $pdf endPage; $pdf startPage; set y [dict get $ctx top]
            }
            set y [pdf4tcllib::text::writeParagraph $pdf $absatz \
                [dict get $ctx left] $y [dict get $ctx text_w] 11]
        }
```

```
    }
    $pdf endPage
}

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
set r [pdf4tcllib::toc::document $pdf a4 $inhalt -title "Inhalt"]
puts "Verzeichnis: [dict get $r tocPages] Seite(n),\
    Inhalt: [dict get $r contentPages] Seiten"
foreach e [dict get $r entries] { puts "  $e" }
$pdf write -file 110-inhaltsverzeichnis.pdf
$pdf destroy
```

```
Verzeichnis: 1 Seite(n), Inhalt: 5 Seiten
  1 Einleitung 2
  2 {Zweck des Berichts} 2
  2 Datenquellen 3
  1 Bestand 4
  2 {Lager Nord} 4
  2 {Lager Süd} 5
  1 Ausblick 6
```

Das Verzeichnis selbst auf Seite 1:

```
Inhalt
Einleitung ..................................... 2
    Zweck des Berichts ......................... 2
    Datenquellen ............................... 3
Bestand ........................................ 4
    Lager Nord ................................. 4
    Lager Süd .................................. 5
Ausblick ....................................... 6
```

Die Seitenzahlen im Ergebnis sind schon um die Länge des Verzeichnisses verschoben; sie stimmen mit dem fertigen PDF überein.

11.2.1 Was das Skript nicht darf

Weil es zweimal läuft, darf das Skript **keine Nebenwirkungen** haben: nicht in eine Datei anhängen, keinen Zähler außerhalb hochzählen, nicht aus einem Kanal lesen, den der erste Lauf schon leer gelesen hat. Daten werden vorher geladen, wie `kapitel` und `absatz` im Beispiel.

11.2.2 Welcher Kontext gilt

`toc::document` baut seinen Kontext selbst, aus dem Papier mit `page::context papier`, also mit 20 mm Rand und y nach unten. Ein `ctx`, den der Aufrufer vorher gebaut hat, wird **nicht** benutzt, sondern überschrieben: Nach dem Aufruf steht in `ctx` der Kontext von `toc::document`, und in `pdf` wieder das echte Dokument. Wer einen anderen Rand braucht, geht den Weg von Hand (Abschnitt Ein Deckblatt davor).

11.3 Überschriften

```
pdf4tcllib::toc::heading pdf ctx yVar ebene text ?option wert ...?
```

Die Ebene (1 bis 6) bestimmt drei Dinge: die Schriftgröße (18 minus 2 je Ebene, mindestens 10), die Einrückung im Verzeichnis und im barrierefreien PDF das Element `H1` bis `H6`. `yVar` ist der Name der y-Variablen, wie bei den Formularprozeduren; sie steht danach unter der Überschrift.

Option	Bedeutung
-size	Schriftgröße statt der Vorgabe der Ebene
-gap	Abstand unter der Überschrift (Vorgabe 4)
-tag	Strukturelement statt H<ebene>

Zwei Dinge macht `heading` nicht:

- **Fett setzen.** Die Überschrift läuft durch `text::writeParagraph` und steht in der normalen Schrift, nur größer. Eine fette Überschrift zeichnet man selbst und ruft `heading` nicht auf, dann fehlt sie aber im Verzeichnis.
- **Umbrechen.** `heading` prüft nicht, ob unter der Überschrift noch Platz ist. Das Beispiel beginnt darum jedes Kapitel der Ebene 1 auf einer neuen Seite und prüft vor jedem Absatz den unteren Rand.

11.4 Optionen von toc::document

Option	Vorgabe	Bedeutung
-title	Contents	Überschrift des Verzeichnisses
-titlesize	16	ihre Größe
-size	11	Größe der Einträge
-leader	.	Füllzeichen bis zur Seitenzahl; "" für keines
-indent	14	Einrückung je Ebene in Punkt
-gap	4	Abstand unter jeder Überschrift
-bookmarks	1	ein Lesezeichen je Überschrift

Die Vorgabe für `-title` ist englisch; für ein deutsches Dokument gehört `-title "Inhalt"` dazu.

`-gap` und `-bookmarks` gelten für die Überschriften dieses Aufrufs; danach steht die Konfiguration des Pakets wieder wie vorher. Für alle folgenden Dokumente im selben Prozess:

```
pdf4tcllib::toc::configure -bookmarks 0
```

Bis pdf4tcltoc 0.1 wirkten die beiden Optionen bei `toc::document` nicht: `heading` las nur die Konfiguration, `-bookmarks 0` schrieb trotzdem Lesezeichen. Beim Weg von Hand (unten) ruft man `heading` ohne `document` auf; dort gilt weiter `toc::configure`.

11.5 Ein Deckblatt davor

`toc::document` schreibt das Verzeichnis als Erstes ins Dokument. Ein Deckblatt davor ist so nicht möglich. Die Bausteine, aus denen `document` besteht, sind aber einzeln aufrufbar: toc::collect, `toc::pageCount` und `toc::render`.

Prozedur	tut
toc::collect skript ?versatz?	führt das Skript aus und gibt die Einträge zurück
toc::pageCount ctx einträge ?option ...?	wie viele Seiten das Verzeichnis braucht
toc::render pdf ctx einträge ?option ...?	schreibt das Verzeichnis, mit eigenen Seiten

Damit lässt sich jede Reihenfolge bauen:

```
# 110-mit-deckblatt.tcl -- Deckblatt vor dem Inhaltsverzeichnis
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcltoc

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]

set inhalt {
    foreach titel {Einleitung Bestand Ausblick} {
        $pdf startPage
        set y [dict get $ctx top]
        pdf4tcllib::toc::heading $pdf $ctx y 1 $titel
        $pdf endPage
    }
}

# 1. Probelauf in ein Wegwerf-Dokument: wo stehen die Ueberschriften?
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
set eintraege [pdf4tcllib::toc::collect $inhalt]
$pdf destroy

# 2. Seitenzahlen verschieben: um das Deckblatt und das Verzeichnis
set vorne 1
set n [pdf4tcllib::toc::pageCount $ctx $eintraege -title "Inhalt"]
set eintraege [lmap e $eintraege {
    lset e 2 [expr {[lindex $e 2] + $vorne + $n}]
}]

# 3. Das echte Dokument: Deckblatt, Verzeichnis, Inhalt
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
pdf4tcllib::page::centerText $pdf $ctx "Bestandsbericht" 300 28 \
```

```
    [pdf4tcllib::fonts::fontSansBold]
$pdf endPage
pdf4tcllib::toc::render $pdf $ctx $eintraege -title "Inhalt"
eval $inhalt

puts "Einträge: $eintraege"
puts "Seiten: [$pdf pageCount]"
$pdf write -file 110-mit-deckblatt.pdf
$pdf destroy
```

```
Einträge: {1 Einleitung 3} {1 Bestand 4} {1 Ausblick 5}
Seiten: 5
```

Hier läuft das Skript im eigenen Bereich, sieht also den eigenen `ctx` mit dem eigenen Rand. `collect` setzt keine Lesezeichen; die entstehen erst beim zweiten Lauf mit `eval`. Die Verschiebung rechnet man selbst: Seiten vor dem Verzeichnis plus Länge des Verzeichnisses.

`pageCount` und `render` müssen dieselben Optionen bekommen, sonst stimmt die Zahl der Verzeichnisseiten nicht. `toc::document` prüft das und wirft einen Fehler; der Weg von Hand prüft nichts.

11.6 Zusammenfassung

- `toc::document` mit dem Inhalt als Skript ohne Nebenwirkungen; es läuft zweimal.
- Überschriften mit `toc::heading`; sie stehen in der normalen Schrift und brechen nicht selbst um.
- Der Kontext kommt aus dem Papier, nicht vom Aufrufer.
- Lesezeichen abschalten mit `-bookmarks 0` beim Aufruf oder `toc::configure -bookmarks 0` für alle Dokumente.
- Deckblatt davor: `collect`, `pageCount`, `render` von Hand.

Nächstes Kapitel: Formular-Layout

12 Formular-Layout

12.1 Einführung

Ein ausfüllbares Formular besteht in pdf4tcl aus Feldern, die man mit `addForm` an eine Stelle setzt. Beschriftung, Abstände, Abschnitte und die Rechnung, wo das nächste Feld hinkommt, bleiben dem Aufrufer. Der Namensraum form in pdf4tcllib übernimmt dieses Layout: eine Beschriftung links, das Feld rechts, eine Zeile tiefer das nächste.

Wie Felder selbst funktionieren (Typen, Werte, Berechnung, JavaScript), erklärt das Buch *pdf4tcl*. Dieses Kapitel zeigt den Aufbau und die Stellen, an denen ein Formular aus pdf4tcllib anders aussieht, als man erwartet.

12.2 Die Bausteine

Die Prozeduren form::section, form::labelField, form::row, `form::separator`, form::orderTable

und form::sumLine nehmen alle `pdf`, `ctx` und den **Namen** der y-Variablen und schieben diese nach unten weiter:

Prozedur	zeichnet
form::section pdf ctx yVar titel	einen grauen Balken mit fettem Titel
form::labelField pdf ctx yVar beschriftung typ ?option ...?	Beschriftung und Feld in einer Zeile
form::row pdf ctx yVar felder	mehrere Paare nebeneinander
form::separator pdf ctx yVar	eine dünne Linie
form::orderTable pdf ctx yVar köpfe breiten ?zeilen? ?option ...?	eine Positionstabelle
form::sumLine pdf ctx yVar breiten text wert ?option ...?	die Summenzeile darunter

Die Typen sind die von `addForm`: `text`, `password`, `checkbox`, `combobox`, `listbox`, `radiobutton`, `pushbutton`, `signature`. Die x-Position kommt aus `SX` im Kontext, die Breite aus `SW`.

12.3 Ein Bestellformular

```
# 120-formular.tcl -- ein ausfuellbares Bestellformular
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
# Formularfelder brauchen eine Schrift mit cp1252: dieselbe
# TrueType-Datei ein zweites Mal angemeldet, fuer form:: eingestellt
pdf4tcl::createFont _Pdf4tcl_Base_Regular FormSans     cp1252
pdf4tcl::createFont _Pdf4tcl_Base_Bold    FormSansBold cp1252
pdf4tcllib::form::configure -fontFamily FormSans \
    -fontFamilyBold FormSansBold
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set y [expr {[dict get $ctx top] + 10}]
# Rahmen und Hintergrund gibt jedes Feld selbst an
set rahmen {-bordercolor {0.6 0.6 0.6} -borderwidth 1
            -bgcolor {0.97 0.97 0.97}}

pdf4tcllib::form::section    $pdf $ctx y "Kunde"
pdf4tcllib::form::labelField $pdf $ctx y "Name:" text -id f_name \
    -init "Müller GmbH" {*}$rahmen
pdf4tcllib::form::labelField $pdf $ctx y "E-Mail:" text -id f_mail \
    {*}$rahmen
```

```
# row nimmt dieselben Angaben als Schluessel ohne Bindestrich
set r [dict create bordercolor {0.6 0.6 0.6} borderwidth 1 \
    bgcolor {0.97 0.97 0.97} labelw 40 type text]
pdf4tcllib::form::row $pdf $ctx y [list \
    [dict merge $r {label "PLZ:" width 120 id f_plz}] \
    [dict merge $r {label "Ort:" width 260 id f_ort}]]
pdf4tcllib::form::labelField $pdf $ctx y "Versand:" combobox \
    -id f_versand -options {Paket Spedition "Abholung Lager Süd"} \
    -init Paket -fieldw 150 {*}$rahmen
# Das Kaestchen bekommt seinen Rahmen als Grafik (siehe Text)
set labelW [dict get [pdf4tcllib::form::configure] labelW]
set xk [expr {[dict get $ctx SX] + $labelW + 4}]
$pdf setStrokeColor 0.6 0.6 0.6
$pdf rectangle $xk $y 12 12
$pdf setStrokeColor 0 0 0
pdf4tcllib::form::labelField $pdf $ctx y "Eilig:" checkbox \
    -id f_eilig -fieldw 12 -fieldh 12
pdf4tcllib::form::separator  $pdf $ctx y

pdf4tcllib::form::section    $pdf $ctx y "Bestellung"
set breiten {40 230 60 90}
set zahl {number decimals 2 sep de}
pdf4tcllib::form::orderTable $pdf $ctx y \
    {Pos Artikel Menge Betrag} $breiten \
    {{1 "Schraube M8" 100 "12.50"}} -emptyRows 3 -cellForm f_pos \
    -cellOpts [list 3 [list -align right -format $zahl]]
# Die Summe rechnet der Betrachter: aus den Feldern der Spalte Betrag
set betraege [lmap r {0 1 2 3} { string cat f_pos_ $r _3 }]
pdf4tcllib::form::sumLine $pdf $ctx y $breiten "Summe:" "" \
    -id f_summe -init "12.50" -calculate [list sum $betraege] \
    -format $zahl
pdf4tcllib::form::labelField $pdf $ctx y "Bemerkung:" text -id f_bem \
    -multiline 1 -fieldh 40 {*}$rahmen

$pdf endPage
$pdf write -file 120-formular.pdf
$pdf destroy
puts "y am Ende: [format %.1f $y]"
```

```
y am Ende: 388.7
```

Im PDF stehen 24 Felder: Name, E-Mail, PLZ, Ort, Versand, Eilig, sechzehn Zellen der Positionstabelle (`f_pos_0_0` bis `f_pos_3_3`), die Summe und die Bemerkung.

12.3.1 labelField

Alles, was `labelField` selbst nicht kennt, geht an `addForm`: `-id`, `-init`, `-options`, `-multiline`, die Rahmenfarben. Drei Optionen gehören `labelField`:

Option	Vorgabe	Bedeutung
-labelw	90	Breite der Beschriftung
-fieldw	Rest der Zeile	Breite des Feldes
-fieldh	16	Höhe des Feldes

`-fieldw` gilt für jeden Typ. Ein Kontrollkästchen ohne `-fieldw` ist also so breit wie die ganze Zeile; im Beispiel steht darum `-fieldw 12 -fieldh 12`.

12.3.2 row

`row` nimmt eine Liste von Dicts mit den Schlüsseln `label`, `type`, `width` (Beschriftung und Feld zusammen), `id`, `init`, `options`, `labelw`, `fieldh` und den Darstellungsschlüsseln `bordercolor`, `borderwidth`, `bgcolor`, `align`, `calculate`, `format`. Ohne `labelw` ist die Beschriftung so breit wie ihr Text.

12.3.3 orderTable und sumLine

`orderTable` zeichnet eine Kopfzeile (weiß auf dunkelblau), die Zeilen aus `zeilen` und `-emptyRows` leere Zeilen dazu. Ohne `-cellForm` sind die Zellen gedruckter Text, mit `-cellForm präfix` Textfelder mit der Kennung `präfix_zeile_spalte`. `-cellOpts {spalte {optionen}}` gibt den Feldern einer Spalte weitere Optionen für `addForm`, im Beispiel rechtsbündig und als Zahl.

`sumLine` setzt den Text rechtsbündig vor die letzte Spalte und den Wert in die letzte. Mit `-id` wird der Wert ein Feld, und mit `-calculate` rechnet der Betrachter:

```
-calculate {sum {f_pos_0_3 f_pos_1_3 f_pos_2_3 f_pos_3_3}}
-format    {number decimals 2 sep de}
```

`sum` geht auch als `product`, `average`, `min`, `max`. `sep de` zeigt die Zahl mit Dezimalkomma. Gerechnet wird nur in Betrachtern mit Formular-JavaScript, etwa Acrobat Reader; in Evince oder Okular bleibt der Wert aus `-init` stehen.

12.4 Schriften in Formularen

Ein Formularfeld hat zwei Schriften: die, in der die Seite den Wert zeigt (die Erscheinung), und die, in der der Betrachter den Wert neu zeichnet, wenn jemand tippt. Die zweite muss der Betrachter selbst kodieren können. Mit den Schriften aus `fonts::init` geht das schief:

- **Mit -cid 1** zeigt eine Auswahlliste Kästchen statt Buchstaben (□□□N□H□ statt `Paket`), sobald der Betrachter sie neu zeichnet. `addForm` kann mit CID-Schriften nicht umgehen.
- **Ohne -cid** kodiert pdf4tcllib die Schrift mit einer eigenen Zeichentabelle. Die Erscheinung stimmt, der gespeicherte Wert aber steht in derselben Tabelle: Aus `Müller` wird beim Exportieren der Formulardaten `MÜller`, aus ß ein ¿.

Abhilfe ist dieselbe TrueType-Datei, ein zweites Mal angemeldet, diesmal mit der Kodierung `cp1252`, und `form` so eingestellt, dass es diese Schrift nimmt:

```
pdf4tcl::createFont _Pdf4tcl_Base_Regular FormSans     cp1252
pdf4tcl::createFont _Pdf4tcl_Base_Bold    FormSansBold cp1252
pdf4tcllib::form::configure -fontFamily FormSans \
    -fontFamilyBold FormSansBold
```

`_Pdf4tcl_Base_Regular` und `_Pdf4tcl_Base_Bold` sind die Namen, unter denen `fonts::init` die Dateien lädt. Der übrige Text der Seite bleibt in der CID-Schrift. Mit `cp1252` gehen deutsche Umlaute, ß und €; Zeichen außerhalb davon gehören nicht in ein Formularfeld.

Ohne `form::configure` nehmen Kopfzeile von `orderTable` und Text von `sumLine` übrigens immer **Helvetica-Bold**, auch nach `fonts::init`. Diese Schrift ist nicht eingebettet; ein Formular für PDF/A oder PDF/UA fällt damit durch.

Die Zellen von `orderTable` erscheinen fett: Mit `-cellForm` übernehmen die Felder die Schrift der Kopfzeile. Eine Option dagegen gibt es nicht.

12.5 Rahmen und Hintergrund

Jedes Feld, das `labelField`, `row`, `orderTable -cellForm` und `sumLine -id` anlegen, bekommt Hintergrund und Rahmen aus `form::configure`: `fieldBg`, `fieldBorder` und `fieldBorderW` werden zu `-bgcolor`, `-bordercolor` und `-borderwidth`. Gibt der Aufruf eine dieser Optionen selbst an, gilt seine. Im Beispiel bekommt jedes Feld einen etwas dunkleren Rahmen:

```
set rahmen {-bordercolor {0.6 0.6 0.6} -borderwidth 1
            -bgcolor {0.97 0.97 0.97}}
```

`fieldBg {}` lässt den Hintergrund weg, `fieldBorderW 0` den Rahmen. Bis pdf4tcllib 0.6.6 kamen die Einstellungen bei keinem Feld an; ohne eigene Angabe hatte ein Feld keinen Rahmen, und auf dem Bildschirm war nicht zu erkennen, wo man klicken soll.

Beim Kontrollkästchen reicht das nicht. Sobald ein Feld des Dokuments rechnet oder formatiert (`-calculate`, `-format`, `-js`), verlangt das PDF vom Betrachter, alle Felder neu zu zeichnen (`/NeedAppearances`). Für Textfelder nimmt er dann Rahmen und Hintergrund aus den Angaben oben. Beim Kontrollkästchen schreibt pdf4tcl den Rahmen aber nur in die eigene Erscheinung, die dann nicht mehr gilt, und das Kästchen ist unsichtbar. Das Beispiel zeichnet seinen Rahmen darum als Rechteck auf die Seite, an derselben Stelle wie das Feld.

12.6 Einstellungen

`form::configure` ohne Argumente gibt alle Einstellungen zurück, mit Paaren setzt es sie. Unbekannte Namen sind ein Fehler.

Name	Vorgabe	Bedeutung
fontFamily, fontFamilyBold	Helvetica, Helvetica-Bold	Schriften (siehe oben)
fontSize, fontSizeLabel, fontSizeSection	9, 9, 10	Größen

Name	Vorgabe	Bedeutung
fieldH	16	Feldhöhe und Zeilenhöhe der Tabelle
labelW, labelGap	90, 4	Breite der Beschriftung, Abstand zum Feld
rowGap, sectionGap	6, 10	Abstand nach Zeile und Abschnitt
sectionBg, lineColor	grau	Abschnittsbalken, Trennlinie
fieldBg, fieldBorder	hellgrau, grau	Hintergrund und Rahmen der Felder
fieldBorderW	1	Rahmenbreite der Felder; 0 = keiner

Die Namen werden ohne Bindestrich gespeichert, aber mit gesetzt: `form::configure -labelW 110`.

12.7 Was man wissen sollte

- **Kein Seitenumbruch.** Die Prozeduren schieben `y` weiter, beginnen aber keine neue Seite. Ein Formular, das über eine Seite geht, teilt man selbst auf.
- **Die Beschriftung steht unten.** Sie sitzt auf der Grundlinie des Feldes; bei einem hohen Feld (`-fieldh 40`) steht "Bemerkung:" also neben dem unteren Rand.
- **Barrierefrei.** Beschriftung und Feld liegen zusammen in einem Element `Form`, mit der Beschriftung als Ersatztext; in der Positionstabelle ist es der Spaltenkopf.

12.8 Zusammenfassung

- `section`, `labelField`, `row`, `orderTable`, `sumLine` bauen ein Formular Zeile für Zeile; sie schieben `y` weiter.
- Für die Felder eine `cp1252`-Schrift anmelden und mit `form::configure` einstellen, sonst falsche Werte oder Kästchen.
- Rahmen bei jedem Feld selbst angeben; ein Kontrollkästchen in einem rechnenden Formular braucht einen gezeichneten Rahmen.
- `sumLine -calculate` rechnet im Betrachter, nicht in Tcl.

Nächstes Kapitel: Etikettenbögen

13 Etikettenbögen

13.1 Einführung

Adressetiketten, Regalschilder, Versandaufkleber: Ein Bogen mit 24 Etiketten ist eine Seite mit 24 Rechtecken, deren Lage auf den Zehntelmillimeter stimmen muss. Das Paket pdf4tcllabels kennt die Maße gängiger Bögen, verteilt die Datensätze auf die Positionen und beginnt neue Bögen. Was auf einem Etikett steht, schreibt der Aufrufer einmal, als Skript.

13.2 Die Bögen

Name	Raster	je Bogen	Etikett (mm)	Rand l/o (mm)
3427	2 × 2	4	105,0 × 148,0	0 / 0
3474	3 × 8	24	70,0 × 37,0	0 / 0
3475	3 × 7	21	70,0 × 42,3	0 / 0,45
3483	3 × 5	15	70,0 × 50,8	0 / 21,5
4737	3 × 9	27	63,5 × 29,6	7,25 / 13,0

Alle auf A4. 3427 ist ein A6-Versandetikett, 4737 der einzige Bogen mit Zwischenräumen zwischen den Etiketten.

labels::sheet gibt die Maße eines Bogens als Dict zurück, in Punkt zum Rechnen und in Millimetern zum Vergleich mit der Packung:

```
set geo [pdf4tcllib::labels::sheet 3474]
dict get $geo cols            ;# 3
dict get $geo w               ;# 198.425... (Punkt)
dict get $geo wmm             ;# 70.0
```

Ein eigener Bogen kommt mit `labels::define` dazu, in Millimetern:

```
pdf4tcllib::labels::define regal {
    w 48.5 h 25.4 cols 4 rows 10 left 8.0 top 21.5
    pitchx 50.0 pitchy 25.4 desc "Regalschilder aus dem Schrank"
}
```

`pitchx` und `pitchy` sind der Abstand von Etikett zu Etikett, nicht die Lücke. Bei Bögen ohne Zwischenraum sind sie gleich der Etikettgröße.

13.3 Adressen auf zwei Bögen

Dreißig Adressen, der Bogen ist schon angebrochen: Die ersten zwei Etiketten fehlen.

```
# 130-etiketten.tcl -- Adressetiketten auf Bogen 3474 (3 x 8)
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcllabels

pdf4tcllib::fonts::init -cid 1

set adressen {}
set lang "Gebrüder Hoffmann-Schwarzenbach Metallbau GmbH & Co. KG"
for {set i 1} {$i <= 30} {incr i} {
    lappend adressen [dict create \
        name    [expr {$i == 5 ? $lang : "Firma Beispiel $i"}] \
        strasse "Industriestraße $i" \
```

```
        ort     "[expr {10000 + $i}] Musterstadt"]
}

set geo [pdf4tcllib::labels::sheet 3474]
puts "Bogen 3474: [dict get $geo cols] x [dict get $geo rows],\
    [dict get $geo wmm] x [dict get $geo hmm] mm"

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
set bogen [pdf4tcllib::labels::render $pdf 3474 $adressen {x y w h a} {
    set font [pdf4tcllib::fonts::fontSans]
    set fett [pdf4tcllib::fonts::fontSansBold]
    set breit [expr {$w - 16}]
    set name [dict get $a name]
    # Name: kleiner setzen, bis 8 pt; reicht das nicht, zwei Zeilen
    set g [pdf4tcllib::labels::fitSize $pdf $fett $name $breit 11 8]
    $pdf setFont $g $fett
    if {[$pdf getStringWidth $name] <= $breit} {
        set zeilen [list $name]
    } else {
        set zeilen [pdf4tcllib::labels::wrap $pdf $name $breit]
        set zeilen [lrange $zeilen 0 1]
    }
    set ty [expr {$y + 26 - 5 * ([llength $zeilen] - 1)}]
    foreach z $zeilen {
        $pdf text $z -x [expr {$x + 8}] -y $ty
        set ty [expr {$ty + $g + 1}]
    }
    $pdf setFont 10 $font
    set tx [expr {$x + 8}]
    $pdf text [dict get $a strasse] -x $tx -y [expr {$y + 46}]
    $pdf text [dict get $a ort]     -x $tx -y [expr {$y + 60}]
} -start 2 -clip 1 -frame layer]
puts "Bögen: $bogen"
$pdf write -file 130-etiketten.pdf
$pdf destroy
```

```
Bogen 3474: 3 x 8, 70.0 x 37.0 mm
Bögen: 2
```

Auf dem ersten Bogen bleiben die Positionen 1 und 2 leer, die Adressen 1 bis 22 füllen den Rest, die übrigen acht stehen auf dem zweiten Bogen.

13.3.1 render

```
set boegen [pdf4tcllib::labels::render pdf bogen datensätze \
    {x y w h satz} skript ?option wert ...?]
```

labels::render öffnet und schließt die Seiten selbst; ein `startPage` davor ist falsch. Für jeden Datensatz läuft das Skript im Bereich des Aufrufers, mit den fünf Variablen aus der Liste: linke

obere Ecke, Breite und Höhe in Punkt, und der Datensatz, so wie er in der Liste steht. Zurück kommt die Zahl der Bögen.

Option	Vorgabe	Bedeutung
-start N	0	die ersten N Positionen frei lassen
-only {i j ...}	–	genau diese Positionen benutzen, für Nachdrucke
-offsetx, -offsety	0.0	Druckerversatz in Millimetern
-clip	0	1: nichts über den Rand des Etiketts hinaus zeichnen
-frame	0	1: Umrisse fest drucken, layer: Umrisse als abschaltbare Ebene
-tag	Sect	Strukturelement je Etikett im barrierefreien PDF

Unbekannte Optionen sind ein Fehler.

`-start` gilt nur für den ersten Bogen. Wie viele Etiketten schon fehlen, weiß nur, wer den Bogen in der Hand hat; ein Programm, das danach fragt, spart einen halben Bogen je Druck.

13.3.2 Umrisse zum Prüfen

`-frame 1` zeichnet die Umrisse jedes Etiketts in die Seite, unter den Inhalt. Das ist ein Probedruck auf Papier, nicht für die Etiketten selbst. `-frame layer` legt die Umrisse in eine Ebene "Label grid", über den Inhalt; der Betrachter kann sie abschalten, gedruckt wird sie nicht. So sieht man am Bildschirm, ob alles im Etikett liegt, und druckt dieselbe Datei. `-frame layer` verlangt pdf4tcl 0.9.4.70 oder neuer und geht nicht mit PDF/A.

13.4 Text, der zu lang ist

Ein Etikett ist schmal: 70 mm sind bei 11 Punkt etwa 35 Zeichen. Ein langer Firmenname läuft sonst über den Rand auf das Nachbaretikett. Das Paket hat drei Hilfen, labels::fitSize, `labels::wrap` und `labels::ellipsize`, alle mit echter Breitenmessung:

Prozedur	tut
labels::fitSize pdf schrift text maxB größe ?min?	verkleinert in Schritten von 0,5 pt, nicht unter min (Vorgabe 6)
labels::wrap pdf text maxB	bricht an Leerzeichen in Zeilen um, mit der aktuellen Schrift
labels::ellipsize pdf text maxB ?ende?	kürzt und hängt … an

Das Beispiel geht zweistufig vor: erst verkleinern bis 8 Punkt; passt der Name dann immer noch nicht, zwei Zeilen. Etikett 5 ("Gebrüder Hoffmann-Schwarzenbach Metallbau GmbH & Co. KG") steht so in zwei Zeilen zu 8 Punkt.

`fitSize` hört bei `min` auf, auch wenn der Text dann noch nicht passt. Das Ergebnis muss man darum nachmessen, wie im Beispiel mit `getStringWidth`. `wrap` misst mit der Schrift, die gerade eingestellt ist; vorher `setFont`.

`ellipsize` ist für Bemerkungen und Referenzen. Eine gekürzte Straße ist keine Adresse.

`text::fitSize` aus Kapitel Text setzen tut dasselbe wie `labels::fitSize`, aber mit anderer Reihenfolge der Argumente und in ganzen Punkten. Beide nicht verwechseln.

13.4.1 -clip als Sicherung

`-clip 1` schneidet alles ab, was über den Rand des Etiketts reicht. Das ersetzt nicht das Anpassen, denn der Rest des Namens fehlt dann einfach. Es verhindert aber, dass ein vergessener Fall den Nachbarn beschriftet.

13.5 Den Drucker vermessen

Kaum ein Drucker setzt auf den Millimeter genau. Ein bis zwei Millimeter Versatz sind üblich, und auf einem Bogen ohne Rand wie 3474 rutscht die erste Zeile damit halb vom Etikett.

```
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
pdf4tcllib::labels::calibration $pdf 3474
$pdf write -file kalibrierung.pdf
```

`calibration` schreibt einen Bogen mit Millimeterskala an den Rändern und den Umrissen aller Etiketten. Ausdrucken, auf einen echten Bogen legen, gegen das Licht halten, Versatz ablesen. Die zwei Zahlen gehen von da an als `-offsetx` und `-offsety` an `render`, einmal je Drucker.

Gedruckt wird in **tatsächlicher Größe**. "An Seite anpassen" verkleinert um ein paar Prozent: in der ersten Zeile unsichtbar, in der letzten mehrere Millimeter.

13.6 Zusammenfassung

- `labels::render` mit Bogenname, Datensätzen und einem Skript je Etikett; es macht die Seiten selbst.
- `-start` für angebrochene Bögen, `-only` für Nachdrucke.
- Lange Namen mit `fitSize` und `wrap` anpassen, `-clip 1` als Sicherung.
- Den Drucker einmal mit `calibration` vermessen und den Versatz angeben.

Nächstes Kapitel: Bilder

14 Bilder

14.1 Einführung

pdf4tcl setzt ein Bild mit `putImage` in ein Rechteck. Gibt man Breite und Höhe an, wird das Bild auf genau diese Fläche gezogen; passt das Seitenverhältnis nicht, ist es verzerrt, und nichts weist darauf hin. pdf4tcllib hat zwei Prozeduren, die das vermeiden: image::place passt ein Bild in einen Rahmen, ohne es zu verzerren, und image::gallery setzt viele Bilder mit Unterschriften in ein Raster über mehrere Seiten.

Beide arbeiten mit Bildern, die pdf4tcl mit `addImage` geladen hat (PNG, JPEG). Tk ist dafür nicht nötig.

14.2 Laden

```
set id [$pdf addImage datei.png]
lassign [$pdf getImageSize $id] breite hoehe     ;# in Pixel
```

`addImage` gibt eine Kennung zurück. Das Bild steht danach einmal in der Datei, auch wenn es auf vielen Seiten erscheint.

Eine Eigenheit von pdf4tcl: **Ein JPEG, das vor dem ersten startPage geladen wird, beginnt selbst eine Seite.** Ein PNG tut das nicht. Wer danach `startPage` aufruft, hat eine leere erste Seite. Darum im Beispiel erst die Seite, dann die Bilder.

14.3 Ein Bild in einen Rahmen

```
lassign [pdf4tcllib::image::place pdf id x y breite hoehe \
    ?option wert ...?] bx by bb bh
```

Option	Vorgabe	Bedeutung
-fit	contain	contain, cover oder stretch
-anchor	center	nw n ne w center e sw s se
-dpi	–	warnen, wenn die Auflösung darunter liegt
-alt	–	Ersatztext; das Bild wird ein Element Figure
-clip	1	bei cover den Überstand abschneiden

- **contain**: Das ganze Bild ist zu sehen, ein Rand des Rahmens bleibt frei.
- **cover**: Der Rahmen ist ganz gefüllt, was übersteht, wird abgeschnitten.
- **stretch**: wie `putImage` mit Breite und Höhe, also verzerrt.

`-anchor` sagt bei `contain`, wo das Bild im Rahmen sitzt, und bei `cover`, welcher Teil erhalten bleibt. Ein Porträt mit dem Kopf oben will `n`.

Zurück kommt die Fläche, die das Bild wirklich einnimmt: bei `contain` kleiner als der Rahmen, bei `cover` der Rahmen selbst. Darunter geht es mit der nächsten Zeile weiter.

Bei `cover` wird nur verdeckt, nicht beschnitten: Das ganze Bild steht in der Datei. Wer einen kleinen Ausschnitt eines großen Fotos braucht, schneidet vorher zu.

14.4 Das Beispiel

Seite 1 setzt dasselbe Hochformat (400 × 600 Pixel) in drei gleiche Rahmen von 150 × 120 Punkt und darunter ein zu kleines Bild. Seite 2 und 3 sind eine Galerie mit fünf Bildern, vier je Seite.

```
# 140-bilder.tcl -- Bilder einpassen und als Galerie setzen
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
```

```
# Bilder einmal laden; addImage gibt eine Kennung zurueck.
# Erst die Seite beginnen: ein JPEG vor startPage beginnt selbst eine.
$pdf startPage
set dir [file join [file dirname [info script]] bilder]
foreach name {hochformat.png querformat.jpg quadrat.png
              panorama.jpg klein.png} {
    set bild($name) [$pdf addImage [file join $dir $name]]
}
puts "hochformat.png: [$pdf getImageSize $bild(hochformat.png)] Pixel"

# Seite 1: dasselbe Hochformat in drei gleich grossen Rahmen
set x [dict get $ctx left]
set y [expr {[dict get $ctx top] + 20}]
$pdf setFont 9 [pdf4tcllib::fonts::fontSans]
foreach fit {contain cover stretch} {
    set r [pdf4tcllib::image::place $pdf $bild(hochformat.png) \
        $x $y 150 120 \
        -fit $fit -anchor n -alt "Beispielbild ($fit)"]
    puts [format "%-8s -> %s" $fit [lmap v $r {format %.0f $v}]]
    $pdf setStrokeColor 0.8 0 0
    $pdf rectangle $x $y 150 120              ;# der Rahmen
    $pdf setStrokeColor 0 0 0
    $pdf text "-fit $fit" -x $x -y [expr {$y + 134}]
    set x [expr {$x + 170}]
}

# Zu wenig Aufloesung: 80 x 60 Pixel auf 160 x 120 Punkt sind 36 dpi
pdf4tcllib::image::place $pdf $bild(klein.png) [dict get $ctx left] \
    [expr {$y + 160}] 160 120 -dpi 150
set w [lindex $::pdf4tcl::warnings end]
puts "Warnung: [string map {" (" "\n  ("} $w]"
$pdf endPage

# Seite 2 und 3: Galerie mit Unterschriften, 2 x 2 je Seite
proc kopf {} {
    global pdf ctx
    pdf4tcllib::page::header $pdf $ctx "Bildergalerie"
}
set liste [list \
    [list $bild(querformat.jpg) "Querformat 900 x 500"] \
    [list $bild(hochformat.png) "Hochformat 400 x 600"] \
    [list $bild(quadrat.png)    "Quadrat"] \
    [list $bild(panorama.jpg) \
        "Panorama 1200 x 300, eine sehr lange Unterschrift"] \
    [list $bild(querformat.jpg) "noch einmal quer"]]
$pdf startPage
```

```
kopf
set seiten [pdf4tcllib::image::gallery $pdf $liste \
    [dict get $ctx left] [expr {[dict get $ctx top] + 10}] \
    [dict get $ctx text_w] 500 \
    -cols 2 -rows 2 -gap 12 -anchor s -pagecmd kopf]
$pdf endPage
puts "Galerie: $seiten Seiten"

$pdf write -file 140-bilder.pdf
$pdf destroy
```

```
hochformat.png: 400 600 Pixel
contain  -> 92 77 80 120
cover    -> 227 77 150 120
stretch  -> 397 77 150 120
Warnung: image::place: image11 at 36 dpi
  (80x60 pixels on 160.0x120.0 points), below 150
Galerie: 2 Seiten
```

Bei `contain` ist das Bild 80 Punkt breit und steht mittig im Rahmen; links und rechts bleiben je 35 Punkt frei. Bei `cover` füllt es den Rahmen, mit `-anchor n` bleibt der obere Teil, die Beschriftung "hoch" am unteren Rand fällt weg. Bei `stretch` ist der Kreis eine Ellipse.

14.4.1 Auflösung

80 × 60 Pixel auf 160 × 120 Punkt sind 36 dpi; gedruckt ist das treppig. `-dpi 150` schreibt dann eine Warnung nach `$::pdf4tcl::warnings`. Sie geht nicht auf den Bildschirm; wer sie sehen will, gibt die Liste am Ende aus. pdf4tcl warnt selbst schon unter 150 dpi; mit `-dpi` gilt die eigene Schwelle, und die Warnung kommt nur einmal.

14.5 Galerie

```
set seiten [pdf4tcllib::image::gallery pdf bilder x y breite hoehe \
    ?option wert ...?]
```

Jeder Eintrag in `bilder` ist `{id "Unterschrift"}` oder nur `id`. Der Kasten `x y breite hoehe` wird in `-cols` × `-rows` Zellen geteilt; ist eine Seite voll, beginnt `gallery` die nächste und setzt dort an dieselbe Stelle. Zurück kommt die Zahl der Seiten.

Option	Vorgabe	Bedeutung
-cols, -rows	2, 3	Raster je Seite
-gap	8	Abstand zwischen den Zellen
-caption	unten	unten oder keine
-captionH	14	Höhe des Bandes für die Unterschrift
-fontsize, -font	8, Sans	Schrift der Unterschrift
-fit, -anchor, -dpi	wie bei place	je Bild

Option	Vorgabe	Bedeutung
-pagecmd	–	läuft nach jedem neuen startPage

`-caption` erwartet die deutschen Wörter. Jeder andere Wert, auch `below`, schaltet die Unterschriften stillschweigend ab.

Die Unterschrift steht am unteren Rand der Zelle, das Bild mit `-anchor center` in der Mitte darüber. Bei einem Querformat in einer hohen Zelle liegt dazwischen eine Lücke. Mit `-anchor s` steht jedes Bild direkt auf seiner Unterschrift, wie im Beispiel.

Eine Unterschrift, die breiter als die Zelle ist, wird kleiner gesetzt (bis 5 Punkt), nicht gekürzt. Bei der Panorama-Unterschrift im Beispiel ist das zu sehen.

`-pagecmd` läuft im Bereich des Aufrufers. Das erste `startPage` und seine Kopfzeile macht der Aufrufer selbst, das letzte `endPage` auch.

Ist der Kasten zu klein für das Raster, bricht `gallery` mit `the frame is too small for 2x3 cells` ab (Zahlen je nach Raster). Bis pdf4tcllib 0.6.6 kam wegen eines Fehlers in der Meldung stattdessen `invalid command name "2"`.

14.6 Bilder aus Tk

Für ein Bild, das als Tk-`photo` vorliegt (aus einem Canvas, aus einer Kamera), gibt es `image::insert` und `image::insertAt`. Sie brauchen Tk, skalieren auf eine Breite und beginnen bei Bedarf eine neue Seite, haben aber zwölf Argumente. Wenn das Bild auch als Datei vorliegt, ist `addImage` mit `place` einfacher.

14.7 Zusammenfassung

- Bilder mit `addImage` laden, erst nach dem ersten `startPage`.
- `image::place` mit `-fit contain` oder `cover`, nie verzerrt; `-anchor` wählt Lage oder Ausschnitt.
- `-dpi` warnt bei zu geringer Auflösung, in `$::pdf4tcl::warnings`.
- `image::gallery` für Raster über mehrere Seiten; `-anchor s` setzt die Bilder auf die Unterschrift.

Nächstes Kapitel: Pfeile und Callouts

15 Pfeile und Callouts

15.1 Einführung

Ein Ablaufschema, ein beschrifteter Plan, ein Hinweis neben einer Tabelle: Dafür braucht man Kästen, Pfeile zwischen ihnen und gelegentlich eine Sprechblase, die auf etwas zeigt. pdf4tcl hat Linien, Rechtecke und Vielecke; die Rechnung, in welche Richtung eine Pfeilspitze zeigt und wo die Linie davor aufhören muss, macht der Namensraum drawing in pdf4tcllib.

15.2 Die Prozeduren

Vier Prozeduren: drawing::arrow, drawing::arrowHead, drawing::callout und

drawing::roundedRect.

Prozedur	zeichnet
drawing::arrow pdf x1 y1 x2 y2 ?option ...?	eine Linie mit Spitze
drawing::arrowHead pdf x y winkel ?option ...?	nur die Spitze, an einem Punkt
drawing::callout pdf x y b h zielX zielY text ?option ...?	einen Kasten mit Zeiger und Text
drawing::roundedRect pdf x y b h r ?rand? ?füllen?	ein Rechteck mit runden Ecken

Farbe und Linienstärke sind die, die gerade eingestellt sind; keine der Prozeduren ändert sie (außer `callout`, siehe unten).

15.3 Ein Ablaufschema

Drei Kästen in einer Reihe, ein vierter darunter, gerade und geknickte Pfeile und zwei Sprechblasen.

```
# 150-pfeile.tcl -- ein Ablaufschema mit Pfeilen und einer Sprechblase
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage

# Ein Kasten mit runden Ecken und Text in der Mitte
proc kasten {x y w h text} {
    global pdf
    $pdf setFillColor 0.88 0.92 0.97
    pdf4tcllib::drawing::roundedRect $pdf $x $y $w $h 6 1 1
    $pdf setFillColor 0 0 0
    $pdf setFont 11 [pdf4tcllib::fonts::fontSansBold]
    $pdf text $text -align center \
        -x [expr {$x + $w / 2.0}] -y [expr {$y + $h / 2.0 + 4}]
}

set top [expr {[dict get $ctx top] + 120}]
set links [dict get $ctx left]
set b 110; set h 40; set abst 50
foreach name {Auftrag Lager Versand} {
    kasten $links $top $b $h $name
    set rechts($name) [expr {$links + $b}]
    set mitte($name)  [expr {$links + $b / 2.0}]
    set links [expr {$links + $b + $abst}]
```

```
}
set unten [expr {$top + 110}]
kasten [expr {$mitte(Lager) - $b / 2.0}] $unten $b $h "Rücksendung"

# Gerade Pfeile: von Kante zu Kante, die Spitze endet genau am Kasten
set my [expr {$top + $h / 2.0}]
$pdf setLineWidth 1.2
foreach k {Auftrag Lager} {
    pdf4tcllib::drawing::arrow $pdf \
        $rechts($k) $my [expr {$rechts($k) + $abst}] $my
}

# Geknickt, Versand -> Ruecksendung: Linie selbst ziehen, die
# Spitze mit arrowHead; sie zeigt nach links (Winkel pi)
set yr [expr {$unten + $h / 2.0}]
set xr [expr {$mitte(Lager) + $b / 2.0}]          ;# rechte Kante
$pdf setStrokeColor 0.7 0 0
$pdf setFillColor 0.7 0 0
$pdf line $mitte(Versand) [expr {$top + $h}] $mitte(Versand) $yr
$pdf line $mitte(Versand) $yr [expr {$xr + 8}] $yr
pdf4tcllib::drawing::arrowHead $pdf $xr $yr [expr {acos(-1)}]
# Beidseitig, nur umrissen: Lager <-> Ruecksendung
pdf4tcllib::drawing::arrow $pdf $mitte(Lager) [expr {$top + $h}] \
    $mitte(Lager) $unten -at both -filled 0
$pdf setStrokeColor 0 0 0
$pdf setFillColor 0 0 0

# Sprechblase ueber dem Lager, Zeiger auf die obere Kante
set n [pdf4tcllib::drawing::callout $pdf [expr {$mitte(Lager) + 20}] \
    [expr {$top - 95}] 170 50 $mitte(Lager) $top \
    "Der Bestand wird jeden Morgen um 6 Uhr mit der Buchhaltung\
     abgeglichen." \
    -fill {1 0.97 0.8} -radius 6]
puts "Sprechblase: $n Zeilen"

# Zu viel Text fuer den Kasten: wird kleiner, nicht gekuerzt
set n [pdf4tcllib::drawing::callout $pdf \
    [dict get $ctx left] [expr {$top - 95}] 120 40 \
    $mitte(Auftrag) $top \
    "Aufträge vor 14 Uhr gehen am selben Tag raus, alle anderen am\
     nächsten Werktag." \
    -fill {0.93 0.93 0.93}]
puts "Kleine Blase: $n Zeilen"

$pdf endPage
$pdf write -file 150-pfeile.pdf
$pdf destroy
```

```
Sprechblase: 2 Zeilen
Kleine Blase: 3 Zeilen
```

15.3.1 Gerade Pfeile

`arrow` zieht die Linie von `x1 y1` nach `x2 y2` und setzt die Spitze an das Ende. Die Spitze **endet** genau am Zielpunkt, und die Linie hört eine Spitzenlänge davor auf. Läge die Linie bis zum Ende, ragte sie durch die gefüllte Spitze und hinterließe einen Strich in ihrer Mitte. Für Pfeile zwischen Kästen heißt das: von Kante zu Kante rechnen, nicht von Mitte zu Mitte.

Option	Vorgabe	Bedeutung
-at	end	end, start oder both
-length	8	Länge der Spitze
-width	6	Breite der Spitze
-filled	1	0: Spitze nur umrissen

Unbekannte Optionen und ein falscher Wert für `-at` sind Fehler.

15.3.2 Geknickte Pfeile

`arrow` zeichnet nur gerade Linien. Für eine Verbindung um die Ecke zieht man die Linien selbst und setzt die Spitze mit `arrowHead` an das Ende. Der Winkel ist im Bogenmaß, 0 zeigt nach rechts, π nach links. Mit y nach unten zeigt π/2 nach **unten**.

Auch hier muss die letzte Linie vor der Spitze aufhören, im Beispiel 8 Punkt (die Vorgabe für `-length`) vor dem Kasten. Für eine schräge letzte Strecke gibt `atan2(dy, dx)` den Winkel.

15.4 Sprechblasen

`callout` zeichnet einen Kasten mit runden Ecken, einen dreieckigen Zeiger zum Punkt `zielX zielY` und den Text darin, umbrochen auf die Breite. Auf welcher Seite der Zeiger sitzt, ergibt sich aus der Lage des Ziels: liegt es über dem Kasten, oben; darunter, unten; sonst links oder rechts. Zurück kommt die Zahl der Textzeilen.

Option	Vorgabe	Bedeutung
-fill	–	Füllfarbe {r g b}; ohne nur der Umriss
-stroke	1	Umriss zeichnen
-radius	4	Eckenradius
-tail	12	Breite des Zeigers am Kasten
-pad	5	Innenabstand
-fontsize, -font	9, Sans	Schrift
-alt	–	Ersatztext; das Ganze wird ein Element Figure

Passt der Text nicht in die Höhe, wird er kleiner gesetzt, in Schritten von 1 Punkt bis 5 Punkt. Gekürzt wird nicht: In der kleinen Blase links steht der ganze Satz, in drei Zeilen zu 7 Punkt statt 9. Ob das noch lesbar ist, entscheidet der Aufrufer; die Zahl der Zeilen im Ergebnis ist ein

Anhaltspunkt, die Schriftgröße bekommt man nicht zurück.

`callout` setzt die Füllfarbe für den Text auf Schwarz und lässt sie so stehen.

Zeiger und Kasten sind zwei Formen, jede mit eigenem Umriss. Am Ansatz des Zeigers bleibt darum eine Linie stehen; die Blase ist dort nicht offen wie in einem Comic. Wer das nicht will, nimmt `-stroke 0` und eine kräftigere Füllfarbe.

15.5 Runde Ecken

```
# Radius 6, mit Umriss, gefüllt
pdf4tcllib::drawing::roundedRect $pdf $x $y $b $h 6 1 1
```

Die Ecken sind aus je acht geraden Stücken zusammengesetzt, nicht aus Kurven. Bei kleinen Radien sieht man das nicht, bei einem Radius von 30 Punkt in starker Vergrößerung schon.

Die Option `-clip 1` soll laut Dokumentation einen Beschneidungspfad mit runden Ecken setzen, etwa für ein Foto mit runden Ecken. Sie beschneidet aber auf das **eckige** Rechteck. Für runde Ecken um ein Bild gibt es in pdf4tcllib keinen Weg.

15.6 Barrierefrei

Pfeile, Spitzen, Kästen und Zeiger sind im barrierefreien PDF Zierrat (Artefakte); ein Vorleseprogramm überspringt sie. Der Text einer Sprechblase ist Inhalt und steht als Absatz im Strukturbaum. Ein Schaubild als Ganzes braucht einen Ersatztext, sonst bleibt von ihm nur eine Folge von Wörtern ohne Pfeile; mehr dazu in Teil 5.

15.7 Zusammenfassung

- `drawing::arrow` von Kante zu Kante; die Linie endet vor der Spitze.
- Geknickte Pfeile: Linien selbst, Spitze mit `arrowHead`, Winkel im Bogenmaß.
- `drawing::callout` wählt die Seite des Zeigers selbst und verkleinert zu langen Text.
- `roundedRect -clip` beschneidet eckig.

Nächstes Kapitel: Formeln

16 Formeln

16.1 Einführung

Ein Prüfbericht mit einer Toleranzformel, ein Datenblatt mit einer Kennlinie, eine Rechnung mit Hochzahlen: Formeln kommen auch in Dokumenten vor, die kein Lehrbuch sind. pdf4tcl schreibt Text auf eine Grundlinie; hochgestellte Zeichen, Brüche und Wurzeln muss man selbst zusammensetzen. Der Namensraum math in pdf4tcllib tut das für zwei Schreibweisen: eine kleine Teilmenge von LaTeX und eine ältere eqn-artige Schreibweise aus dem Tcler's Wiki.

Es ist kein Formelsatz wie in TeX. Für mehrzeilige Gleichungen, Matrizen oder Klammern, die mit dem Inhalt wachsen, ist ein externes Programm (etwa KaTeX nach SVG, dann als Bild) der bessere Weg.

16.2 Die Schrift

Griechische Buchstaben, ±, ∑, ∞ und √ stehen in keiner der Standardschriften. Ohne TrueType-Schrift im CID-Modus werden sie zu?, ohne Fehlermeldung:

```
pdf4tcllib::fonts::init -cid 1
```

16.3 Die LaTeX-Teilmenge

```
set x [pdf4tcllib::math::renderLatex pdf x y formel \
    ?-size n? ?-font name?]
lassign [pdf4tcllib::math::measureLatex pdf formel ?-size n?] \
    breite oben unten
```

math::renderLatex setzt die Formel mit der Grundlinie auf y und gibt die x-Position dahinter zurück. math::measureLatex zeichnet nichts und gibt Breite, Höhe über und Tiefe unter der Grundlinie zurück.

Eingabe	Ergebnis
x^2, a_i, x_{i,j}^{2}	hoch-und tiefgestellt, {} für mehr als ein Zeichen
\frac{a}{b}	Bruch mit Strich, auch geschachtelt
\sqrt{x+1}	Wurzel mit Strich darüber
\sum_{i=1}^{n}, \int_0^1, \prod	große Operatoren mit Grenzen
\alpha … \omega, \Gamma … \Omega	griechische Buchstaben
\pm \cdot \le \ge \ne \infty …	Zeichen
\, \;	ein kleiner Abstand

Nicht dabei: `\text`, `\mathrm`, `\left`/`\right`, `\quad`, Umgebungen, Matrizen. Ein unbekannter Befehl wird als Text gesetzt, mit Backslash, und nach stderr gemeldet:

```
pdf4tcllib::math: unknown symbol '\quad' rendered as literal text
```

16.3.1 Leerzeichen zählen nicht

Wie in LaTeX werden Leerzeichen in der Eingabe überlesen. Anders als LaTeX setzt `renderLatex` aber auch keinen Abstand um = und +: Aus `a = b` wird a=b, eng aneinander. Wo Luft nötig ist, gehört `\,` hinein:

```
{E \, = \, mc^2}
```

Die dritte abgesetzte Formel im Beispiel zeigt den Unterschied, sie ist ohne `\,` gesetzt.

16.3.2 Das Minuszeichen

`-`b setzt einen Bindestrich, kein Minus. Er ist kürzer und steht tiefer. Wer ein echtes Minus will, schreibt das Zeichen U+2212 selbst in die Formel, etwa mit `[format %c 0x2212]`.

16.4 Im Satz und abgesetzt

```
# 160-formeln.tcl -- Formeln im Text und abgesetzt
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1      ;# Griechisch und Operatoren
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set font [pdf4tcllib::fonts::fontSans]
set x0 [dict get $ctx left]
set y  [expr {[dict get $ctx top] + 20}]

# 1. Eine Formel mitten im Satz: Text, Formel, Text
$pdf setFont 11 $font
$pdf text "Die Energie ist " -x $x0 -y $y
set x [expr {$x0 + [$pdf getStringWidth "Die Energie ist "]}]
set x [pdf4tcllib::math::renderLatex $pdf $x $y {E \, = \, mc^2} \
    -size 11]
$pdf setFont 11 $font       ;# renderLatex verstellt die Schrift
$pdf text ", die Masse also " -x $x -y $y
set x [expr {$x + [$pdf getStringWidth ", die Masse also "]}]
set x [pdf4tcllib::math::renderLatex $pdf $x $y \
    {m \, = \, \frac{E}{c^2}} -size 11]
$pdf setFont 11 $font
$pdf text "." -x $x -y $y

# 2. Abgesetzte Formeln, mittig, mit Abstand nach ihrer Hoehe.
#    \, ist ein kleiner Abstand; Leerzeichen zaehlen nicht.
set w [dict get $ctx text_w]
set y [expr {$y + 30}]
set nr 0
foreach f {
    {x \, = \, \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}}
    {\sum_{i=1}^{n} i \, = \, \frac{n(n+1)}{2}}
    {\alpha + \beta + \gamma = \pi}
} {
    lassign [pdf4tcllib::math::measureLatex $pdf $f -size 14] \
        fb oben unten
    set y [expr {$y + $oben}]
    set xm [expr {$x0 + ($w - $fb) / 2.0}]
    pdf4tcllib::math::renderLatex $pdf $xm $y $f -size 14
    puts [format "Formel %d: Breite %5.1f  oben %4.1f  unten %4.1f" \
        [incr nr] $fb $oben $unten]
    set y [expr {$y + $unten + 16}]
}
```

```
# 3. Die eqn-Schreibweise: Zeichen durch Leerzeichen getrennt
set y [expr {$y + 10}]
pdf4tcllib::math::renderFormula $pdf $x0 $y \
    "H _ 2 O ~ + ~ CO _ 2" -size 12
set y [expr {$y + 24}]
pdf4tcllib::math::renderFormula $pdf $x0 $y \
    "SUM from i=0 to infty ~ a _ i x ^ i" -size 12

$pdf endPage
$pdf write -file 160-formeln.pdf
$pdf destroy
```

```
Formel 1: Breite  95.1  oben 23.4  unten 12.9
Formel 2: Breite  79.6  oben 17.1  unten 14.7
Formel 3: Breite  63.0  oben 10.1  unten  2.8
```

16.4.1 Im Satz

Text und Formel wechseln sich ab, und jedes Stück beginnt dort, wo das vorige aufgehört hat: beim Text über `getStringWidth`, bei der Formel über den Rückgabewert von `renderLatex`.

Nach `renderLatex` ist die **Schrift verstellt**: Es bleibt die Größe des letzten hochgestellten Zeichens eingestellt, im Beispiel 7,7 statt 11 Punkt. Darum steht nach jeder Formel ein neues `setFont`.

Ein Bruch im Satz ragt nach oben und unten über die Zeile hinaus. `m = E/c²` reicht bei 11 Punkt 14 Punkt über und 14 Punkt unter die Grundlinie. Mit dem üblichen Zeilenabstand von 1,4 ragt er in die nächste Zeile hinein; für Brüche im Fließtext ist `a/b` in einer Zeile oft die bessere Wahl.

`text::writeParagraph` kennt keine Formeln. Ein Absatz mit Formeln wird Stück für Stück gesetzt, wie im Beispiel, und umbricht nicht von selbst.

16.4.2 Abgesetzt

Für eine Formel in eigener Zeile misst `measureLatex` vorher. Die Breite gibt die Mitte, `oben` sagt, wie weit die Grundlinie unter der vorigen Zeile liegen muss, `unten`, wo die nächste beginnen darf. Das Beispiel rechnet so den Abstand jeder Formel nach ihrer eigenen Höhe.

16.5 Die eqn-Schreibweise

```
set x [pdf4tcllib::math::renderFormula pdf x y formel \
    ?-size n? ?-font name?]
```

math::renderFormula ist die ältere Schreibweise, ein Port von Arjen Markus' "MathFormula" aus dem Tcler's Wiki. Jedes Element ist durch Leerzeichen getrennt:

Zeichen	Bedeutung
^ x	x hochgestellt
_ x	x tiefgestellt

Zeichen	Bedeutung
~	ein Leerzeichen, das stehen bleibt
SUM, INT, PROD	große Operatoren
from a, to b	Grenzen nach einem Operator
alpha, infty, pm …	Zeichen, ohne Backslash

`from` und `to` sind **immer** Grenzen; ein Pfeil heißt `rightarrow`. Brüche und Wurzeln mit Strich kennt diese Schreibweise nicht. Die Grenzen über `SUM` sind klein und stehen sehr dicht über dem Zeichen (im Beispiel das ∞).

`renderFormula` stellt danach die Schrift auf die Grundgröße zurück, in der normalen Schrift, auch wenn vorher fett eingestellt war.

Für neue Dokumente ist `renderLatex` die bessere Wahl: Es kann mehr, und die Schreibweise kennen mehr Leser. Die Dokumentation von pdf4tcllib sagt im Abschnitt über `renderFormula`, Brüche und Wurzeln gingen nicht; das gilt nur für `renderFormula`.

16.6 Zusammenfassung

- Ohne `fonts::init -cid 1` werden Formelzeichen zu?.
- `renderLatex` für Hoch, Tief, Bruch, Wurzel, Summe; `measureLatex` misst vorher.
- Leerzeichen zählen nicht, Abstände mit `\,`; Minus ist ein Bindestrich.
- Nach `renderLatex` die Schrift neu setzen.
- Unbekannte Befehle werden als Text gesetzt.

Nächstes Kapitel: Treeview

17 Treeview

17.1 Einführung

Viele Tk-Programme zeigen ihre Daten in einem `ttk::treeview`: Lager mit Artikeln, Projekte mit Aufgaben, Ordner mit Dateien. Der Wunsch "das als PDF" heißt dann: genau das, was im Fenster steht. Das Paket pdf4tcltreeview liest das Widget aus und zeichnet es, entweder als Tabelle mit eingerückter Baumspalte oder als Gliederung.

Es ist eine dünne Schicht über `table::draw` (Kapitel Tabellen mit Seitenumbruch). Es braucht Tk, denn es liest ein Widget.

17.2 Tabelle oder Gliederung

Ein Treeview ist zugleich Tabelle und Baum. Im PDF muss man sich entscheiden, und darum gibt es zwei Prozeduren statt einer Option, treeview::render und treeview::outline:

Prozedur	zeichnet	im barrierefreien PDF
treeview::render	Tabelle, die Baumspalte eingerückt	Table, TR, TD
treeview::outline	nur die Baumspalte, als Liste	L, LI, LBody

Beide geben die y-Position nach dem Letzten zurück.

17.3 Ein Lagerbestand

```
# 170-treeview.tcl -- ein ttk::treeview als Tabelle und als Gliederung
package require Tk
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcltreeview

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]

# Das Widget: Baumspalte #0 plus zwei Wertespalten
ttk::treeview .tv -columns {menge wert} -show {tree headings}
.tv heading #0    -text "Lager / Artikel"
.tv heading menge -text "Menge"
.tv heading wert  -text "Wert €"
.tv column #0    -width 260
.tv column menge -width 80 -anchor e
.tv column wert  -width 100 -anchor e

foreach {lager offen artikel} {
    "Lager Nord" 1 {{"Schraube M6" 600 29.40} {"Schraube M8" 400 31.20}
                    {"Mutter M8" 900 18.00}}
    "Lager Süd"  1 {{"Dübel 8 mm" 1200 42.00}
                    {"Winkel 40 mm" 150 67.50}}
    "Außenlager" 0 {{"Palette leer" 34 0.00}}
} {
    set summe 0.0
    foreach a $artikel { set summe [expr {$summe + [lindex $a 2]}] }
    set k [.tv insert {} end -text $lager -open $offen \
        -values [list "" [format %.2f $summe]]]
    foreach a $artikel {
        lassign $a name menge wert
        .tv insert $k end -text $name \
            -values [list $menge [format %.2f $wert]]
    }
}
pack .tv
update idletasks

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Seite 1: wie auf dem Bildschirm -- das Aussenlager ist zugeklappt
$pdf startPage
set x [dict get $ctx left]
```

```
set y [pdf4tcllib::treeview::render $pdf .tv $x [dict get $ctx top] \
    -maxwidth 300 -ctx $ctx]
puts "Tabelle wie angezeigt endet bei y = [format %.1f $y]"
# Dasselbe mit allen Zweigen
set y [pdf4tcllib::treeview::render $pdf .tv $x [expr {$y + 30}] \
    -maxwidth 300 -ctx $ctx -all 1]
puts "Tabelle mit -all 1 endet bei y = [format %.1f $y]"
$pdf endPage

# Seite 2: nur die Baumspalte als eingerueckte Liste, mit einem
# Lesezeichen je Knoten
$pdf startPage
set y [pdf4tcllib::treeview::outline $pdf .tv $x [dict get $ctx top] \
    -all 1]
puts "Gliederung endet bei y = [format %.1f $y]"
$pdf endPage

$pdf write -file 170-treeview.pdf
$pdf destroy
exit                        ;# sonst wartet Tk auf Ereignisse
```

```
Tabelle wie angezeigt endet bei y = 200.7
Tabelle mit -all 1 endet bei y = 390.7
Gliederung endet bei y = 182.7
```

Seite 1 als Text:

```
Lager / Artikel   Menge   Wert €
Lager Nord                 78.60
  Schraube M6       600    29.40
  Schraube M8       400    31.20
  Mutter M8         900    18.00
Lager Süd                 109.50
  Dübel 8 mm       1200    42.00
  Winkel 40 mm      150    67.50
Außenlager                  0.00
```

Die zweite Tabelle hat darunter noch "Palette leer".

17.3.1 Das Skript braucht ein exit

Mit `package require Tk` wartet `tclsh` nach dem letzten Befehl auf Ereignisse, statt sich zu beenden. Das `exit` am Ende gehört darum dazu. Das Fenster muss nicht zu sehen sein, aber Tk braucht eine Anzeige. Auf einem Server ohne Bildschirm läuft das Skript mit `xvfb-run tclsh 170-treeview.tcl`.

Das `update idletasks` vor dem Export empfiehlt die Dokumentation des Pakets. Im Beispiel ändert es nichts, weil alle Spalten eine feste Breite haben; bei Spalten, die Tk erst beim Anzeigen ausrechnet, ist es der sichere Weg.

17.4 render

```
set y [pdf4tcllib::treeview::render pdf tv x y ?option wert ...?]
```

Option	Vorgabe	Bedeutung
-maxwidth	480	Breite der Tabelle
-fontsize	9	Schriftgröße
-all	0	1: auch zugeklappte Zweige
-tree	1	0: Baumspalte nicht einrücken
-indentW	12	Einrückung je Ebene
-zebra, -border	1, 1	Streifen, Linien
-ctx	–	Seitenkontext; schaltet den Umbruch ein
-font, -boldfont	–	Schriften statt der aus fonts::init
-summary	–	Zusammenfassung der Tabelle (barrierefrei)

Was zugeklappt ist, bleibt zugeklappt. Das PDF zeigt, was im Fenster zu sehen ist; im Beispiel fehlt in der ersten Tabelle der Inhalt des Außenlagers. `-all 1` nimmt alles.

Die Spaltenbreiten kommen in Pixeln aus dem Widget und werden auf `-maxwidth` umgerechnet; das Verhältnis bleibt. Die Ausrichtung kommt aus `-anchor` der Spalte: `e` wird rechtsbündig.

Was `render` nicht liest: Farben und Schriften einzelner Einträge (`-tags` mit `tag configure`), Bilder. Die Knoten "Lager Nord" usw. stehen in derselben Schrift wie die Artikel. `-rowstyles` von `table::draw` reicht `render` nicht durch; wer Gruppenzeilen fett will, liest das Widget selbst aus und ruft `table::draw` direkt auf.

17.4.1 Seitenumbruch

Mit `-ctx` bricht die Tabelle am unteren Rand um, aber mit dem eingebauten Umbruch von `table::draw`: `- 1 -` unten rechts, keine Kopf- und Fußzeile auf der Folgeseite. `-pagebreakcmd` gibt es bei `render` nicht; wer sie angibt, bekommt einen Fehler mit der Liste der bekannten Optionen.

17.5 outline

```
set y [pdf4tcllib::treeview::outline pdf tv x y ?option wert ...?]
```

Option	Vorgabe	Bedeutung
-fontsize	10	Schriftgröße
-indentW	14	Einrückung je Ebene
-lineH	0	Zeilenabstand; 0 heißt 1,4 × Größe
-all	0	auch zugeklappte Zweige

Option	Vorgabe	Bedeutung
-bookmarks	1	ein Lesezeichen je Knoten
-font	–	Schrift

Ein Knoten ohne Text nimmt den ersten Wert.

17.5.1 Lesezeichen

`-bookmarks 1` legt für jeden Knoten ein Lesezeichen an, in der Tiefe des Baums: "Schraube M6" hängt unter "Lager Nord". Im Beispiel ist das die Vorgabe. Bis pdf4tcltreeview 0.1 entstand keines: `outline` rief `bookmarkAdd` in einer Form auf, die pdf4tcl nicht versteht, und fing den Fehler still ab. Wer noch mit 0.1 arbeitet, legt sie selbst an:

```
proc lesezeichen {tv knoten ebene} {
    global pdf
    foreach k [$tv children $knoten] {
        $pdf bookmarkAdd -title [$tv item $k -text] -level $ebene
        lesezeichen $tv $k [expr {$ebene + 1}]
    }
}
lesezeichen .tv {} 0
```

Die Lesezeichen zeigen auf den Anfang der Seite, auf der sie angelegt werden; eine Stelle auf der Seite kennt `bookmarkAdd` nicht.

17.6 Beides in einem Dokument

Tabelle und Gliederung beschreiben dieselben Daten. Ein Vorleseprogramm liest beide vor, wenn beide da sind. Wer beide braucht (Übersicht vorn, Tabelle hinten), setzt sie auf getrennte Seiten und sagt in einer Überschrift, was folgt.

17.7 Zusammenfassung

- `treeview::render` für die Tabelle, `treeview::outline` für die Gliederung; beide brauchen Tk.
- Das Skript endet mit `exit`; ohne Bildschirm mit `xvfb-run`.
- Zugeklappte Zweige fehlen, außer mit `-all 1`.
- `outline` legt je Knoten ein Lesezeichen an (seit 0.1.1).
- Umbruch nur der eingebaute, ohne Kopfzeile.

Nächstes Kapitel: Barrierefreie PDFs

18 Barrierefreie PDFs

18.1 Einführung

Ein Vorleseprogramm sieht keine Seite, sondern einen Baum: Überschrift, Absatz, Tabelle mit Zeilen und Zellen, Abbildung mit Beschreibung. Steht dieser Strukturbaum nicht im PDF, liest das Programm die Seite so vor, wie die Texte im Datenstrom liegen, samt Seitenzahl, Kopfzeile

und Tabellenlinien. Behörden und viele Firmen verlangen deshalb barrierefreie PDFs nach PDF/UA (ISO 14289).

pdf4tcl schreibt den Strukturbaum, wenn man ihn einschaltet. pdf4tcllib sorgt dafür, dass seine Bausteine ihn richtig füllen: Eine Tabelle wird `Table` mit `TR` und `TD`, eine Kopfzeile wird Schmuck. Was man selbst zeichnet, muss man selbst einordnen. Dieses Kapitel zeigt beides an einem Bericht.

18.2 Was ein Dokument für PDF/UA braucht

Anforderung	in pdf4tcl
Strukturbaum	$pdf tagged 1
Sprache des Dokuments	tagged 1 -lang de-DE
Titel, im Fenster statt des Dateinamens	metadata -title ..., viewerPreferences -displaydoctitle 1
eingebettete Schriften	fonts::init; keine der 14 Standardschriften
jeder Inhalt in einem Element oder als Schmuck markiert	Bausteine von pdf4tcllib, eigenes mit tag::
Ersatztext für Abbildungen	-alt
Kennzeichnung als PDF/UA	tagged 1 -ua 1

`-ua 1` schreibt nur die Behauptung in die Metadaten, dass das Dokument PDF/UA-1 erfüllt. Prüfen kann pdf4tcl das nicht. Ein Dokument, das die Behauptung aufstellt und sie nicht hält, ist schlechter als eines, das nichts behauptet; darum ist `-ua` nicht die Vorgabe.

Auch `-displaydoctitle` ist nicht die Vorgabe, und `-ua 1` setzt es nicht mit. PDF/UA verlangt es aber: Ohne zeigt der Betrachter den Dateinamen statt des Titels.

18.3 Ein Bericht mit Strukturbaum

```
# 180-barrierefrei.tcl -- ein Bericht mit Strukturbaum fuer PDF/UA
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart

pdf4tcllib::fonts::init -cid 1           ;# eingebettete Schrift
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Strukturbaum an, Sprache, Anspruch auf PDF/UA-1, Titel
$pdf tagged 1 -lang de-DE -ua 1
$pdf metadata -title "Lagerbericht Oktober" -author "Lager Nord"
$pdf viewerPreferences -displaydoctitle 1      ;# Titel statt Dateiname
set ::pdf4tcl::warnings {}
```

```
$pdf startPage
set x [dict get $ctx left]
set w [dict get $ctx text_w]
pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"

# Ueberschrift: writeParagraph mit dem Typ H1 statt des Vorgabe-P
set y [pdf4tcllib::text::writeParagraph $pdf "Bestand am Monatsende" \
    $x [expr {[dict get $ctx top] + 20}] $w 16 left H1]
set y [pdf4tcllib::text::writeParagraph $pdf "Der Bestand wurde am\
    Monatsende gezählt. Die Tabelle nennt die Abweichungen je Regal." \
    $x [expr {$y + 6}] $w 11]

# Eine Tabelle: Table, TR, TH, TD von selbst
set y [pdf4tcllib::table::draw $pdf $x [expr {$y + 10}] {
    {-header Regal -width 80}
    {-header Soll -width 60 -align right}
    {-header Ist -width 60 -align right}
} {{A-01 120 118} {A-02 80 80} {B-01 45 47}} \
    -summary "Soll und Ist je Regal"]

# Ein Diagramm: ein Element Figure mit Ersatztext
set y [pdf4tcllib::chart::bar $pdf $x [expr {$y + 20}] 300 150 \
    {A-01 -2 A-02 0 B-01 2} -title "Abweichung je Regal" \
    -alt "Balkendiagramm: A-01 minus 2, A-02 null, B-01 plus 2"]

# Selbst gezeichnet: Text als P, die Linie als Schmuck
pdf4tcllib::tag::begin $pdf P
$pdf setFont 9 [pdf4tcllib::fonts::fontSans]
$pdf text "Gezählt von: M. Albers" -x $x -y [expr {$y + 20}]
pdf4tcllib::tag::end $pdf
pdf4tcllib::tag::artifact $pdf -type Layout
$pdf line $x [expr {$y + 26}] [expr {$x + $w}] [expr {$y + 26}]
pdf4tcllib::tag::artifactEnd $pdf

pdf4tcllib::page::footer $pdf $ctx "Intern" 1
puts "ohne Element: [$pdf getUntaggedCount]"
$pdf endPage
$pdf write -file 180-barrierefrei.pdf
$pdf destroy
foreach m $::pdf4tcl::warnings { puts "Warnung: $m" }
```

```
ohne Element: 0
```

Bis pdf4tclchart 0.1.1 stand darunter noch eine Warnung: "the Figure element is empty". Das Diagramm war damals ganz als Artefakt markiert, die `Figure` hatte keinen Inhalt. Seit 0.1.2 ist das Gezeichnete ihr Inhalt, und die Warnung entfällt.

Der Strukturbaum des Ergebnisses, mit einem PDF-Werkzeug ausgelesen:

```
Document
  H1          [Inhalt]
  P           [Inhalt]
  Table       Summary="Soll und Ist je Regal"
    TR
      TH  TH  TH        Scope=Column
    TR
      TD  TD  TD
    TR  ...
  Figure      Alt="Balkendiagramm: A-01 minus 2, A-02 null,
                   B-01 plus 2"  [Inhalt]
  P           [Inhalt]
```

Kopf- und Fußzeile und die Linie unter "Gezählt von" stehen nicht im Baum. Sie sind als Artefakt markiert: Schmuck, den ein Vorleseprogramm überspringt.

Die Linien der Tabelle dagegen hängen als Inhalt am Element `Table`. Die Dokumentation von pdf4tcllib nennt sie Artefakte; im Datenstrom stehen sie mit einer Inhaltsnummer unter `Table`. Ein Vorleseprogramm liest Linien nicht vor, und `getUntaggedCount` zählt sie nicht. Ob ein Prüfprogramm Inhalt direkt unter `Table` beanstandet, ist hier nicht geprüft.

18.4 Was die Bausteine selbst tun

Baustein	im Strukturbaum
text::writeParagraph	P, oder der Typ im letzten Argument
table::draw	Table, TR, TH mit Scope=Column, TD; -summary
chart::bar, line, pie	Figure mit Ersatztext aus -alt oder -title; Balken und Achsen sind ihr Inhalt
image::place, gallery	Figure mit -alt; Unterschrift als Caption
toc::heading, toc::render	H1 bis H6; TOC und TOCI
form::labelField, row	Form mit Beschriftung und Feld
labels::render	ein Sect je Etikett
drawing::callout	Text als P, Kasten und Zeiger als Artefakt
page::header, footer, number, drawing::watermark	Artefakt (Seitengestaltung)

Ist `tagged` aus, tun die Bausteine nichts davon. Derselbe Code erzeugt dann ein gewöhnliches PDF.

18.4.1 Überschriften

`writeParagraph` setzt `P`, wenn man nichts sagt. Für eine Überschrift gehört der Typ als achtes

Argument dazu, mit Größe und Ausrichtung davor:

```
pdf4tcllib::text::writeParagraph $pdf "Bestand" $x $y $w 16 left H1
```

`writeParagraph` stellt dabei immer die normale Schrift ein; ein `setFont` mit Fett davor hat keine Wirkung. Die Überschrift im Beispiel ist darum nur größer, nicht fett. Für eine fette Überschrift mit Strukturelement zeichnet man sie selbst (Abschnitt Selbst gezeichnetes einordnen).

Die Ebenen dürfen nicht springen: nach `H1` ein `H2`, nicht gleich `H3`. Das prüft pdf4tcl nicht.

18.4.2 Diagramme

Ein Diagramm wird ein `Figure` mit Ersatztext; Balken, Achsen und Beschriftungen sind sein Inhalt. Ein Vorleseprogramm sagt den Ersatztext, nicht die einzelnen Striche. Der Titel ist als Ersatztext meist zu wenig: "Abweichung je Regal" sagt nicht, wie die Abweichung aussieht. Im Beispiel stehen die Werte im Ersatztext.

Mehr als den Ersatztext bekommt ein Leser vom Diagramm nicht. Sind die Zahlen wichtig, gehören sie zusätzlich in eine Tabelle.

18.4.3 Tabellen

`-summary` beschreibt die Tabelle in einem Satz, bevor die Zellen kommen. Die Kopfzeile wird `TH` mit `Scope=Column`; damit weiß ein Leser bei jeder Zelle, zu welcher Spalte sie gehört.

18.5 Selbst gezeichnetes einordnen

Was man direkt mit pdf4tcl zeichnet, gehört zu nichts, bis man es einordnet. Dafür gibt es vier Prozeduren im Namensraum tag:

Prozedur	tut
tag::begin pdf typ ?-alt text?	ein Element öffnen (P, H2, Figure …)
tag::end pdf	es schließen
tag::artifact pdf ?-type Layout|Pagination?	Schmuck beginnen
tag::artifactEnd pdf	Schmuck beenden

Eine fette Überschrift, selbst gesetzt:

```
pdf4tcllib::tag::begin $pdf H2
$pdf setFont 13 [pdf4tcllib::fonts::fontSansBold]
$pdf text "Abweichungen" -x $x -y $y
pdf4tcllib::tag::end $pdf
```

Alle vier tun nichts, wenn `tagged` aus ist. Darum können sie in Code stehen bleiben, der auch gewöhnliche PDFs erzeugt.

`Layout` ist Schmuck auf der Seite (Linien, Flächen, Rahmen), `Pagination` gehört zur Seitengestaltung (Kopfzeile, Seitenzahl, Wasserzeichen).

18.5.1 getUntaggedCount

`$pdf getUntaggedCount` zählt die Zeichenbefehle, die weder in einem Element noch in

einem Artefakt stehen. Null ist das Ziel. Ohne die vier `tag::`-Zeilen im Beispiel wären es 2: der Text "Gezählt von" und die Linie. pdf4tcl schreibt dann beim Speichern zusätzlich eine Warnung nach `$::pdf4tcl::warnings`.

Gezählt wird nur, was gezeichnet wird. Eine Farbe oder Schrift außerhalb eines Elements stört nicht.

18.6 Prüfen

Null in `getUntaggedCount` heißt nur, dass alles eingeordnet ist. Ob die Einordnung stimmt, sagt die Zahl nicht. Drei Stufen:

1. **getUntaggedCount und $::pdf4tcl::warnings** im Programm, bei jedem Lauf. Das Beispiel gibt beides aus.
2. **Ein Prüfprogramm** für die Norm. veraPDF prüft PDF/UA-1:

   `qpdf --check` prüft nur, ob die Datei sauber gebaut ist, nicht die Norm.

   ```
   verapdf -f ua1 bericht.pdf
   ```

3. **Ein Mensch mit Vorleseprogramm** (NVDA, JAWS, Orca). Ein Dokument kann jede Regel erfüllen und trotzdem in einer Reihenfolge vorgelesen werden, die keinen Sinn ergibt.

18.7 Zusammenfassung

- `tagged 1 -lang de-DE`, Titel mit `metadata -title` und `viewerPreferences -displaydoctitle 1`, eingebettete Schrift.
- Die Bausteine füllen den Strukturbaum selbst; Überschriften brauchen den Typ als Argument.
- Selbst Gezeichnetes mit `tag::begin`/`end` oder als Artefakt.
- `getUntaggedCount` muss 0 sein; danach veraPDF und ein Mensch.
- `-ua 1` ist eine Behauptung, keine Prüfung.

Nächstes Kapitel: Grafikzustand sichern

19 Grafikzustand sichern

19.1 Einführung

Jeder Baustein stellt Schrift und Farben so ein, wie er sie braucht, und lässt sie danach so stehen. Wer vor einer Tabelle 20 Punkt fett rot eingestellt hat, schreibt danach in 9 Punkt normal schwarz weiter, ohne es bemerkt zu haben. In kurzen Skripten fällt das sofort auf; in einem Bericht aus vielen Prozeduren sucht man den Grund lange.

state::with hält den Zustand über einen Block fest und stellt ihn danach wieder her.

19.2 Was verloren geht

```
# 190-zustand.tcl -- Schrift und Farbe ueber einen Baustein retten
package require pdf4tcl
package require pdf4tcllib

```

```
pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set x [dict get $ctx left]
set fett [pdf4tcllib::fonts::fontSansBold]
set cols {{-header Regal} {-header Bestand -align right}}
set rows {{A-01 118} {A-02 80}}

# pdf4tcl hat keine Abfrage fuer die Schrift; die Breite eines
# Probetextes zeigt, welche Groesse gerade eingestellt ist
proc probe {} { format %.1f [$::pdf getStringWidth "Probe"] }

# 1. Ohne Schutz: danach ist die Schrift der Tabelle eingestellt
$pdf setFont 20 $fett
$pdf setFillColor 0.8 0 0
puts "vorher:          [probe]"
set y [pdf4tcllib::table::draw $pdf $x 80 $cols $rows]
puts "nach table:      [probe]"
$pdf text "Nachher (ohne Schutz)" -x $x -y [expr {$y + 30}]

# 2. Mit state::with: Schrift und Farbe bleiben erhalten
$pdf setFont 20 $fett
$pdf setFillColor 0.8 0 0
set y [pdf4tcllib::state::with $pdf [list 20 $fett] {0.8 0 0} {
    pdf4tcllib::table::draw $pdf $x [expr {$y + 60}] $cols $rows
}]
puts "nach state::with: [probe]"
$pdf text "Nachher (mit state::with)" -x $x -y [expr {$y + 30}]

$pdf endPage
$pdf write -file 190-zustand.pdf
$pdf destroy
```

```
vorher:          59.5
nach table:      22.9
nach state::with: 59.5
```

pdf4tcl kann nicht sagen, welche Schrift gerade eingestellt ist; es gibt `setFont`, aber kein `getFont`. Das Beispiel misst darum die Breite des Wortes "Probe": 59,5 Punkt in 20 Punkt fett, 22,9 Punkt in der Schrift, mit der `table::draw` zuletzt geschrieben hat. Im PDF steht die erste Zeile "Nachher" klein und schwarz, die zweite groß und rot.

19.3 state::with

```
set ergebnis [pdf4tcllib::state::with pdf {größe schrift} {r g b} \
    skript]
```

Das Skript läuft im Bereich des Aufrufers. Danach sind Schrift und Füllfarbe wieder die angegebenen, auch wenn das Skript einen Fehler wirft. Zurück kommt, was das Skript zurückgibt; im Beispiel das `y` der Tabelle.

Weil pdf4tcl den Zustand nicht herausgibt, muss man ihn **selbst angeben**. `state::with` stellt wieder her, was man ihm sagt, nicht was vorher eingestellt war. Wer `{20 Helvetica}` angibt, während in Wirklichkeit 11 Punkt Sans eingestellt war, bekommt danach 20 Punkt Helvetica.

Was man weglässt (ein leeres `{}` statt Schrift oder Farbe), wird nicht angefasst. Zeilenabstand und Unterschneidung fragt `state` bei pdf4tcl selbst ab und stellt sie mit her.

19.3.1 Von Hand

```
set z [pdf4tcllib::state::save $pdf 20 $fett {0.8 0 0}]
pdf4tcllib::table::draw $pdf ...
pdf4tcllib::state::restore $pdf $z
```

`save` nimmt als fünftes Argument noch die Strichfarbe. Es gibt ein Dict zurück, das `restore` wieder einsetzt. Ein Fehler zwischen den beiden Aufrufen lässt den Zustand verstellt; `with` ist darum meist die bessere Wahl.

19.4 Warum nicht gsave und grestore

PDF hat ein eigenes Paar zum Sichern des Zustands, `q` und `Q`, in pdf4tcl `gsave` und `grestore`. Innerhalb einer Seite ist es das richtige Werkzeug, etwa um einen Beschneidungspfad wieder aufzuheben. Über einen Seitenwechsel hinweg geht es nicht: Jede Seite hat ihren eigenen Datenstrom, ein `gsave` auf Seite 1 und das `grestore` auf Seite 2 ergeben zwei unausgeglichene Ströme. Weder pdf4tcl noch qpdf melden das. Eine Tabelle mit `-ctx` bricht aber genau so um.

Innerhalb einer Seite stellt `grestore` auch die Schrift wieder her, die pdf4tcl sich zum Messen merkt: Nach `gsave`, `setFont 9` und `grestore` misst "Probe" wieder 59,5 Punkt.

19.5 Zusammenfassung

- Bausteine hinterlassen ihre Schrift und Farbe.
- `state::with pdf {größe schrift} {r g b} skript` stellt sie danach wieder her, auch bei einem Fehler.
- Den Zustand selbst angeben; pdf4tcl kann ihn nicht liefern.
- `gsave`/`grestore` nur innerhalb einer Seite.

Nächstes Kapitel: Messen und stapeln

20 Messen und stapeln

20.1 Einführung

Jeder Bericht enthält dieselbe Schleife: einen Block zeichnen, das zurückgegebene `y` nehmen, einen Abstand dazurechnen, den nächsten Block zeichnen. Und bevor eine Tabelle kommt, die Frage: Passt sie noch auf diese Seite, oder gehört sie ganz auf die nächste? Die Bausteine von

pdf4tcllib beantworten das nicht; sie zeichnen und sagen hinterher, wo sie aufgehört haben.

Zwei Namensräume helfen: measure misst einen Block, bevor er gezeichnet wird, und layout::stack setzt Blöcke untereinander und beginnt eine neue Seite, wenn ein Block nicht mehr ganz passt.

20.2 Messen durch Zeichnen

```
set m [pdf4tcllib::measure::block varName skript ?option wert ...?]
```

measure::block zeichnet den Block wirklich, aber in ein Wegwerf-Dokument, das nie geschrieben wird. Was gemessen wird, ist damit genau das, was später herauskommt. Im Skript steht das Wegwerf-Dokument in der Variablen, deren Namen man übergibt (im Beispiel `mp`). Das Ergebnis ist ein Dict:

Schlüssel	Inhalt
height	Höhe des Blocks, aus seinem Rückgabewert
pages	Seiten, über die er lief
overflow	1, wenn er unten über das Blatt hinauslief
value	was das Skript zurückgab
sheet	Höhe des Blatts

Optionen: `-paper` (Vorgabe `a4`), `-orient`, `-landscape`, `-margin`, `-y`. Die Höhe ist der Abstand zwischen `-y` (Vorgabe 0) und dem Rückgabewert; der Block sollte darum bei y = 0 beginnen oder bei dem `y`, das man mit `-y` angibt. Gibt das Skript keine Zahl zurück, ist `height` leer.

Schriften, die `fonts::init` geladen hat, kennt auch das Wegwerf-Dokument; sie gelten für alle Dokumente eines Prozesses.

20.2.1 pages und overflow

Die Seitenzahl steigt nur, wenn der Block selbst umbricht, also etwa `table::draw` mit `-ctx`. Ohne `-ctx` zeichnet `table::draw` über den Seitenrand hinaus, und `pages` bleibt 1. Gemessen wird so, wie man später zeichnet.

`writeParagraph` bricht nie um. Ein langer Absatz läuft unten aus dem Blatt, `pages` bleibt 1, und erst `overflow` sagt, dass etwas nicht stimmt. Im Beispiel ist der lange Absatz 896 Punkt hoch, das Blatt nur 842.

measure::fits `platz varName skript` fasst das zusammen: 1, wenn der Block höchstens `platz` hoch ist, auf einer Seite bleibt und nicht überläuft. measure::height gibt nur die Höhe zurück.

20.3 Das Beispiel

```
# 200-messen-und-stapeln.tcl -- Hoehe vorab messen, Bloecke stapeln
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set x [dict get $ctx left]
set w [dict get $ctx text_w]

set text "Der Bestand wurde am Monatsende gezählt und mit der Buchung\
    abgeglichen. Abweichungen unter zwei Stück sind ausgebucht."
set cols {
    {-header Regal -width 120}
    {-header Soll  -width 60 -align right}
    {-header Ist   -width 60 -align right}
}
set rows {}
for {set i 1} {$i <= 30} {incr i} {
    set soll [expr {$i * 7}]
    lappend rows [list "Regal $i" $soll [expr {$soll - $i % 3}]]
}

# 1. Messen: wie hoch wird die Tabelle, passt der Absatz?
set m [pdf4tcllib::measure::block mp {
    pdf4tcllib::table::draw $mp 0 0 $cols $rows -maxwidth $w
}]
puts "Tabelle: Höhe [format %.1f [dict get $m height]],\
    Seiten [dict get $m pages], Überlauf [dict get $m overflow]"
set lang [string repeat "$text " 40]
set m [pdf4tcllib::measure::block mp {
    pdf4tcllib::text::writeParagraph $mp $lang 0 0 $w 11
}]
puts "Langer Absatz: Höhe [format %.1f [dict get $m height]],\
    Seiten [dict get $m pages], Überlauf [dict get $m overflow]"
puts "passt in 200 pt: [pdf4tcllib::measure::fits 200 mp {
    pdf4tcllib::text::writeParagraph $mp $text 0 0 $w 11
}]"

# 2. Stapeln: Bloecke untereinander, Tabelle und Diagramm am Stueck
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
proc kopf {} {
    global pdf ctx
    pdf4tcllib::page::header $pdf $ctx "Lagerbericht"
}
$pdf startPage
kopf
```

```
# Die Linie der Kopfzeile liegt 2 pt unter top, im Satzspiegel. stack
# beginnt jede Seite bei SY (= top); also SY tiefer, SH kuerzer.
set oben [expr {[dict get $ctx top] + 12}]
set sh   [expr {[dict get $ctx SH] - 12}]
set sctx [dict replace $ctx SY $oben SH $sh]

set absatz  {pdf4tcllib::text::writeParagraph $bpdf $text \
                 $bx $by $bw 11}
set balken  {pdf4tcllib::chart::bar $bpdf $bx $by $bw 200 \
                 {Nord 120 Süd 95 West 60} -title "Bestand je Lager"}
set tabelle {pdf4tcllib::table::draw $bpdf $bx $by $cols $rows \
                 -maxwidth $bw}
set ende    {pdf4tcllib::text::writeParagraph $bpdf "Ende." \
                 $bx $by $bw}
set blocks [list \
    [list script $absatz] \
    [list script $balken keep 1] \
    [list script $absatz] \
    [list script $tabelle keep 1] \
    {gap 10} \
    [list script $ende]]
set r [pdf4tcllib::layout::stack $pdf $x $oben $w $blocks \
    -ctx $sctx -gap 14 -font [list 11 [pdf4tcllib::fonts::fontSans]] \
    -pagecmd kopf]
puts "Seitenwechsel: [dict get $r pages], Ende bei y =\
    [format %.1f [dict get $r y]]"
foreach b [dict get $r boxes] { puts "  [lmap v $b {format %.0f $v}]" }
$pdf endPage
$pdf write -file 200-messen-und-stapeln.pdf
$pdf destroy
```

```
Tabelle: Höhe 502.2, Seiten 1, Überlauf 0
Langer Absatz: Höhe 896.0, Seiten 1, Überlauf 1
passt in 200 pt: 1
Seitenwechsel: 1, Ende bei y = 625.9
  57 69 482 32
  57 115 482 200
  57 329 482 32
  57 69 482 502
  57 595 482 17
```

Auf Seite 1 stehen Absatz, Diagramm und Absatz. Die Tabelle (502 Punkt) passt ab y = 375 nicht mehr in den Satzspiegel; mit `keep 1` kommt sie ganz auf Seite 2, mit Kopfzeile. Danach "Ende."

20.4 layout::stack

```
set r [pdf4tcllib::layout::stack pdf x y breite blöcke \
    ?option wert ...?]
```

Jeder Eintrag in `blöcke` ist eines von beiden:

Eintrag	Bedeutung
script skript ?keep 1?	ein Block
gap n	n Punkt Abstand vor dem nächsten Block

Im Skript stehen vier Variablen bereit: `$bpdf`, `$bx`, `$by`, `$bw`, das Dokument, die linke obere Ecke und die Breite. Das Skript gibt, wie jeder Baustein, das `y` darunter zurück (oder eine Fläche `{x y b h}`).

Im Skript $bpdf, nicht $pdf. Beim Messen ist `$bpdf` das Wegwerf-Dokument. Ein Skript, das `$pdf` benutzt, zeichnet beim Messen ins echte Dokument, und der Block steht zweimal da.

Das Skript läuft im Bereich des Aufrufers und sieht dessen Variablen (`$text`, `$cols`, `$rows`). Im Beispiel stehen die Skripte darum in geschweiften Klammern: Die Variablen werden erst beim Ausführen eingesetzt. Ein Skript über mehrere Zeilen braucht `\` am Zeilenende wie jeder andere Tcl-Befehl.

Option	Bedeutung
-ctx	Seitenkontext; ohne ihn wird nie umgebrochen
-gap	Abstand nach jedem Block
-font {größe schrift}, -fill {r g b}	vor jedem Block gesetzt, danach wiederhergestellt
-pagecmd	läuft nach jedem Seitenwechsel, im Bereich des Aufrufers
-paper	Papier fürs Messen

Zurück kommt ein Dict mit `y` (unter dem letzten Block), `pages` (Zahl der Seitenwechsel, die `stack` selbst gemacht hat) und `boxes`, je Block `{x y b h}`.

20.4.1 Was keep tut und was nicht

Nur Blöcke mit `keep 1` werden gemessen. Passt so ein Block nicht mehr bis zum unteren Rand, beendet `stack` die Seite, beginnt eine neue, ruft `-pagecmd` auf und setzt den Block oben hin.

Blöcke **ohne** `keep` werden weder gemessen noch geprüft. Ein Absatz ohne `keep` am Ende der Seite läuft über den unteren Rand hinaus, auch mit `-ctx`. Darum lohnt `keep 1` auch für kurze Blöcke, wenn man nicht weiß, wo sie landen.

Ein Block, der höher als eine ganze Seite ist, passt auch auf die neue Seite nicht. `stack` setzt ihn trotzdem dorthin; umbrechen muss der Block dann selbst (eine Tabelle mit `-ctx`). Die Seiten, die er dabei anfängt, zählt `pages` nicht mit, und `boxes` sagt nicht, auf welcher Seite ein Block steht.

20.4.2 Die Kopfzeile und der obere Rand

`stack` beginnt jede neue Seite bei `SY` aus dem Kontext, das ist `top`. `page::header` zieht seine Linie aber 2 Punkt **unter** `top`, also im Satzspiegel. Ein Absatz, der genau bei `top`

beginnt, wird von der Linie durchgestrichen. Das Beispiel gibt `stack` darum einen Kontext, in dem SY 12 Punkt tiefer liegt und SH entsprechend kürzer ist:

```
set oben [expr {[dict get $ctx top] + 12}]
set sh   [expr {[dict get $ctx SH] - 12}]
set sctx [dict replace $ctx SY $oben SH $sh]
```

SY und SH sind die Schlüssel, die `stack` liest; die übrigen Bausteine nehmen `top` und `bottom`. Die geänderte Kopie gilt darum nur für `stack`.

20.5 Zusammenfassung

- `measure::block` zeichnet zur Probe und gibt Höhe, Seiten und Überlauf zurück; `overflow` sieht, was `pages` nicht sieht.
- `layout::stack` setzt Blöcke untereinander; in den Skripten `$bpdf`, `$bx`, `$by`, `$bw`.
- Nur `keep 1` wird gemessen und bei Bedarf auf die nächste Seite gesetzt.
- Für Platz unter der Kopfzeile SY und SH im Kontext für `stack` anpassen.

Nächstes Kapitel: Fallstudie: Lagerbericht

21 Fallstudie: Lagerbericht

21.1 Einführung

Ein Bericht über die Lagerbewegungen eines Monats, von den Rohdaten bis zum fertigen PDF: Zwischensummen und Kreuztabelle, ein Diagramm, eine Tabelle über mehrere Seiten mit Kopf- und Fußzeilen, ein Inhaltsverzeichnis mit echten Seitenzahlen und ein Strukturbaum. Jeder Teil stammt aus einem früheren Kapitel; hier laufen sie in einem Skript zusammen, und an den Nahtstellen zeigt sich, worauf es ankommt.

Teil	Kapitel
table::group, table::pivot	Gruppensummen und Kreuztabellen
chart::bar	Diagramme
table::draw mit -ctx und -pagebreakcmd	Tabellen mit Seitenumbruch
page::header, page::footer	Kopf- und Fußzeilen
toc::document, toc::heading	Inhaltsverzeichnis
tagged, metadata	Barrierefreie PDFs

21.2 Das Skript

```
# 210-fallstudie.tcl -- Lagerbericht: Tabellen, Diagramm, Verzeichnis
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart
package require pdf4tcltoc
```

```
pdf4tcllib::fonts::init -cid 1

# --- 1. Daten: vor dem Inhalt laden, er laeuft zweimal
# Bewegungen {Lager Gruppe Artikel Menge Wert}, nach Lager sortiert
set bewegungen {}
set nr 0
foreach lager {Nord Süd West} {
    incr nr
    for {set i 1} {$i <= 22} {incr i} {
        set gruppe [lindex {Schrauben Dübel Werkzeug} [expr {$i % 3}]]
        set menge  [expr {($i * 7 + $nr * 11) % 40 - 8}]
        set wert   [format %.2f [expr {$menge * (1.5 + $i % 5)}]]
        lappend bewegungen [list $lager $gruppe \
            [format "%s-%03d" [string index $lager 0] $i] $menge $wert]
    }
}
set g [pdf4tcllib::table::group $bewegungen -by 0 -sum {3 4} \
    -label "Summe %s" -totallabel "Gesamt" -blank 1]
set p [pdf4tcllib::table::pivot $bewegungen -row 0 -col 1 -value 4 \
    -rowlabel Lager -colorder {Schrauben Dübel Werkzeug}]

# Wert je Lager fuer das Diagramm: die Zeile "Gesamt" der Kreuztabelle
# hat die Spaltensummen, die letzte Spalte jeder Zeile die Zeilensumme
set balken {}
foreach z [lrange [dict get $p rows] 0 end-1] {
    lappend balken [lindex $z 0] [lindex $z end]
}

set stile {}
foreach i [dict get $g subtotals] { lappend stile $i {-font bold} }
lappend stile [dict get $g total] {-font bold -bg {0.85 0.85 0.85}}

proc betrag {v} {
    if {[string is double -strict $v]} { return [format %.2f $v] }
    return $v
}

# --- 2. Seitenwechsel: Fuss auf die alte, Kopf auf die neue Seite
# Laeuft im globalen Bereich; dort ist pdf im ersten Durchlauf das
# Wegwerf-Dokument, im zweiten das echte.
proc seitenwechsel {} {
    global pdf ctx
    pdf4tcllib::page::footer $pdf $ctx "Lagerbericht Oktober" \
        [$pdf currentPage]
    $pdf endPage
    $pdf startPage
```

```
    pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"
    return [expr {[dict get $ctx top] + 10}]
}

# --- 3. Der Inhalt als Skript
set inhalt {
    set x [dict get $ctx left]
    set w [dict get $ctx text_w]

    $pdf startPage
    set y [dict get $ctx top]
    pdf4tcllib::toc::heading $pdf $ctx y 1 "Überblick"
    set y [pdf4tcllib::text::writeParagraph $pdf "Bewegungen der drei\
        Lager im Oktober, bewertet zum Einstandspreis. Negative Mengen\
        sind Abgänge." $x $y $w 11]
    set y [pdf4tcllib::chart::bar $pdf $x [expr {$y + 10}] $w 160 \
        $balken -title "Wert je Lager (€)" -values 1 -format %.0f \
        -alt "Balkendiagramm: Wert der Bewegungen je Lager"]

    set y [expr {$y + 16}]
    pdf4tcllib::toc::heading $pdf $ctx y 2 "Lager und Warengruppe"
    set kopf [lmap h [dict get $p header] {
        set a right
        if {$h eq "Lager"} { set a left }
        list -header $h -width 90 -align $a
    }]
    set y [pdf4tcllib::table::draw $pdf $x $y $kopf \
        [lmap z [dict get $p rows] { lmap v $z { betrag $v } }] \
        -rowstyles [list [dict get $p total] {-font bold}] \
        -summary "Wert je Lager und Warengruppe"]
    pdf4tcllib::page::footer $pdf $ctx "Lagerbericht Oktober" \
        [$pdf currentPage]
    $pdf endPage

    $pdf startPage
    pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"
    set y [expr {[dict get $ctx top] + 10}]
    pdf4tcllib::toc::heading $pdf $ctx y 1 "Bewegungen"
    set y [pdf4tcllib::table::draw $pdf $x $y {
        {-header Lager   -width 50}
        {-header Gruppe  -width 80}
        {-header Artikel -width 60}
        {-header Menge   -width 50 -align right}
        {-header Wert    -width 70 -align right}
    } [lmap z [dict get $g rows] {
        lset z end [betrag [lindex $z end]]
    }] \
```

```
        -ctx $ctx -zebra 1 -rowstyles $stile \
        -pagebreakcmd seitenwechsel -summary "Bewegungen je Lager"]
    pdf4tcllib::page::footer $pdf $ctx "Lagerbericht Oktober" \
        [$pdf currentPage]
    $pdf endPage
}

# --- 4. Dokument: Strukturbaum, Verzeichnis, zwei Durchlaeufe
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf tagged 1 -lang de-DE
$pdf metadata -title "Lagerbericht Oktober" -author "Lagerverwaltung"
$pdf viewerPreferences -displaydoctitle 1
set ::pdf4tcl::warnings {}
set r [pdf4tcllib::toc::document $pdf a4 $inhalt -title "Inhalt"]

puts "Verzeichnis: [dict get $r tocPages] Seite(n),\
    Inhalt: [dict get $r contentPages] Seiten"
foreach e [dict get $r entries] { puts "  $e" }
puts "ohne Element: [$pdf getUntaggedCount]"
$pdf write -file 210-fallstudie.pdf
$pdf destroy
foreach m $::pdf4tcl::warnings { puts "Warnung: $m" }
```

```
Verzeichnis: 1 Seite(n), Inhalt: 3 Seiten
  1 Überblick 2
  2 {Lager und Warengruppe} 2
  1 Bewegungen 3
ohne Element: 0
```

Das PDF hat vier Seiten:

Seite	Inhalt
1	Verzeichnis "Inhalt" mit drei Einträgen
2	"Überblick": Absatz, Balkendiagramm, Kreuztabelle
3	"Bewegungen": Kopfzeile, Tabelle bis zu "S-018"
4	Kopfzeile, Fortsetzung der Tabelle mit wiederholter Spaltenzeile, "Summe West" und "Gesamt"

Das Skript besteht aus vier Teilen, die Kommentare zählen sie.

21.3 1. Die Daten zuerst

`toc::document` führt den Inhalt zweimal aus (Inhaltsverzeichnis). Alles, was nur einmal geschehen darf, steht darum davor: die Daten erzeugen, gruppieren, umformen. Im Bericht eines echten Programms käme hier die Abfrage an die Datenbank; ein zweites Lesen aus demselben Kanal fände ihn leer.

Aus einer Liste entstehen drei Darstellungen:

- `table::group` fügt je Lager eine Zwischensumme und am Ende die Gesamtsumme ein, für die lange Tabelle. Die Zeilen müssen nach Lager sortiert sein; `group` sortiert nicht.
- `table::pivot` macht daraus die Kreuztabelle Lager mal Warengruppe. `-colorder` legt die Spalten fest, sonst stünden sie alphabetisch.
- Das Diagramm braucht Paare aus Name und Wert. Die liefert die Kreuztabelle schon: Die letzte Spalte jeder Zeile ist die Zeilensumme, die letzte Zeile ("Gesamt") bleibt weg.

Die Summen aus `group` und `pivot` sind rohe Tcl-Zahlen. `betrag` formatiert sie erst beim Zeichnen, damit die Daten selbst Zahlen bleiben.

21.4 2. Der Seitenwechsel

`-pagebreakcmd` läuft im globalen Bereich (Tabellen mit Seitenumbruch). Die Prozedur `seitenwechsel` holt sich `pdf` und `ctx` darum mit `global`. Das geht hier gut, weil `toc::document` die Variable `pdf` im Bereich des Aufrufers setzt: Im ersten Durchlauf steht darin das Wegwerf-Dokument, im zweiten das echte. Eine Prozedur, die stattdessen das Dokument in einer eigenen Variablen hielte, würde im ersten Durchlauf in das falsche schreiben.

Die Seitennummer in der Fußzeile ist `[$pdf currentPage]`. Im ersten Durchlauf fehlt die Verzeichnisseite, die Nummern sind dort um eins zu klein. Das schadet nicht: Die Fußzeile verschiebt nichts, und gedruckt wird nur der zweite Durchlauf. Dort zählt das Verzeichnis mit, und die Fußzeile sagt "Seite 2", wo das Verzeichnis "Überblick ... 2" sagt.

21.5 3. Der Inhalt

Das Skript beginnt und beendet seine Seiten selbst. Die erste Inhaltsseite hat keine Kopfzeile, nur die Fußzeile; ab der zweiten setzt das Skript beide, und auf den Folgeseiten der Tabelle tut es `seitenwechsel`.

Zwei Abstände sind von Hand gesetzt: 10 Punkt vor dem Diagramm und 16 Punkt danach. `chart::bar` gibt die Unterkante seines Kastens zurück, `toc::heading` setzt die Überschrift genau dorthin. Ohne den Abstand klebt "Lager und Warengruppe" an der Achsenbeschriftung.

Die Zeile "Summe Nord" beginnt mit einer verbundenen Zelle über drei Spalten. Seit pdf4tcllib 0.6.7 zieht `table::draw` keine senkrechte Linie mehr hindurch; vorher stand dort "Summe No|rd".

21.6 4. Das Dokument

Drei Zeilen machen den Bericht barrierefrei:

- `tagged 1 -lang de-DE` schaltet den Strukturbaum ein und nennt die Sprache;
- `metadata -title` gibt den Titel;
- `viewerPreferences -displaydoctitle 1` lässt den Betrachter den Titel statt des Dateinamens zeigen.

Die Bausteine erledigen den Rest: `toc::heading` setzt `H1` und `H2`, `text::writeParagraph` ein `P`, `table::draw` die Tabellen mit Kopfzellen, `chart::bar` ein `Figure` mit Ersatztext, und Kopf- und Fußzeilen sind Artefakte. `getUntaggedCount` meldet 0. Das Wegwerf-Dokument des ersten Durchlaufs hat keinen Strukturbaum; dort tun die Bausteine

nichts davon.

`::pdf4tcl::warnings` bleibt leer. Mit pdf4tclchart 0.1.1 stand hier noch "the Figure element is empty": Das Diagramm war ganz als Artefakt markiert, und das `Figure` hatte keinen Inhalt.

21.7 Was bewusst fehlt

- **"Seite X von Y".** `toc::document` nennt die Gesamtzahl erst hinterher (`contentPages` plus `tocPages`), wenn die Fußzeilen schon gezeichnet sind. Der Weg von Hand aus Inhaltsverzeichnis kennt sie früher: Nach dem Probelauf mit `toc::collect` hat das Wegwerf-Dokument `pageCount` Seiten, dazu kommen die Seiten aus `toc::pageCount`.
- **Ein Deckblatt.** `toc::document` schreibt das Verzeichnis als erste Seite. Ein Deckblatt davor geht nur von Hand, mit `collect`, `pageCount` und `render`.
- **Mehrzeilige Zellen.** `table::draw` kürzt zu lange Texte; für Bemerkungen wäre `table::render` die Wahl.
- **Zahlen mit Dezimalkomma.** `format %.2f` schreibt einen Punkt. Für einen deutschen Bericht gehört ein `string map {.,}` in `betrag`; im Diagramm geht das nicht, `-format` kennt kein Dezimalkomma.

21.8 Zusammenfassung

- Daten vor `toc::document` laden und umformen; der Inhalt läuft zweimal.
- `group` für die lange Tabelle, `pivot` für die Übersicht, die Diagrammwerte aus der Kreuztabelle.
- `-pagebreakcmd` mit `global pdf`: im ersten Durchlauf das Wegwerf-Dokument, im zweiten das echte.
- Abstände um Diagramme von Hand; `chart::bar` endet an seiner Unterkante.
- `tagged`, `metadata -title`, `displaydoctitle`; den Rest machen die Bausteine.

Nächstes Kapitel: Fallen

22 Fallen

Kurze Liste, nach Thema. Die Begründung steht im genannten Kapitel.

22.1 Einrichten

- `package require pdf4tcllib` lädt pdf4tcl nicht mit; beide Zeilen schreiben (Einrichten).
- Tcl 8.6 liest Skripte in der Systemkodierung. Ohne UTF-8-Locale werden Umlaute im Quelltext zu "Ã¼" im PDF.
- Ohne passende TrueType-Datei läuft alles mit Helvetica weiter; nur stderr sagt es. `fonts::hasTtf` prüfen.
- Die Vorgabefamilie `DejaVuSansCondensed` liegt unter Debian in `fonts-dejavu-extra`, nicht in `fonts-dejavu-core`.
- Dateinamen nach dem Schema `<familie>.ttf`, `<familie>-Bold.ttf`; Liberation-Schriften passen nicht.
- Der CID-Modus wird beim ersten `fonts::init` festgelegt. Ein späteres `fonts::init -cid 1` (auch mit `-force 1`) lädt nichts neu, meldet bei `fonts::isCidMode` aber 1.

22.2 Text und Zeichen

- `unicode::sanitize` richtet sich nach `fonts::init`, nicht nach `setFont`. Helvetica nach `fonts::init` verliert →, ✓ und Rahmenlinien ohne Meldung (Unicode).
- `safeText` macht aus "…" immer "...", auch im CID-Modus; ebenso die Kopf- und Fußzeilen aus `page::`.
- `text::writeParagraph` ruft `sanitize` nicht auf. Fehlende Zeichen zählt dann pdf4tcl (`$pdf getSubstCount`), nicht pdf4tcllib.
- `unicode::substCount` zählt über den ganzen Prozess; vor jedem Dokument `resetSubst`.
- Im 256-Zeichen-Modus gibt es keine deutschen Anführungszeichen.

22.3 Text setzen

- `text::width`, `wrap`, `truncate`, `fits` ohne `$pdf` schätzen die Breite nur (Text setzen).
- `text::wrap` kürzt ein Wort, das allein zu breit ist, auf "..."; `hardBreak 1` zerlegt es stattdessen.
- Bei `text::wrap` steht `pdf` an sechster Stelle, nach `code`; bei `fitSize` heißt es `-pdf`.
- `text::writeParagraph` setzt immer die normale Schrift und bricht nicht über Seiten um.
- Nach einer Fuß- oder Kopfzeile ist deren Schrift gesetzt; vor dem nächsten Text `setFont` erneut aufrufen.

22.4 Seiten und Spalten

- "Seite X von Y" braucht zwei Durchläufe; der Inhalt darf keine Nebenwirkungen haben (Seitenzahlen).
- `-pagelabel` ist ein `format`-Text: `%` als `%%` schreiben.
- `flow::columns` ohne `-newpage` gibt den Rest in `rest` zurück, statt umzubrechen (Spaltenfluss).
- `flow::columns` setzt die erste Grundlinie auf `-top`; ohne Angabe ragt die erste Zeile in den Rand.
- `-firsty` gilt nur für die erste Spalte; `-align` wirkt nicht.
- `drawing::watermark` vor allem anderen zeichnen; mit `-orient 0` liegt es außerhalb der Seite.

22.5 Tabellen und Diagramme

- `table::draw` ohne `-ctx` bricht nicht um; ohne `-pagebreakcmd` gibt es auf den Folgeseiten keine Kopf- und Fußzeile (Tabellen).
- `-pagebreakcmd` läuft im globalen Bereich und muss die neue y-Position zurückgeben.
- Zellen sind einzeilig und werden mit "..." gekürzt, auch die Kopfzeile; `-notruncate 1` für Werte, die abgetippt werden.
- Ein Block aus Zeilen, die `rowspan` verbindet, wird nicht getrennt; ist er höher als eine Seite, bricht `table::draw` mit einem Fehler ab.
- `table::group` sortiert nicht; unsortierte Daten ergeben mehrere Zwischensummen je Gruppe (Gruppen).
- Summen aus `group` und `pivot` sind rohe Tcl-Zahlen; vor dem Zeichnen formatieren.
- `table::pivot` sortiert die Spalten alphabetisch; Monate und Quartale mit `-colorder`.
- `chart::line` zeigt die Namen der Reihen nicht; die Legende selbst zeichnen (Diagramme).
- `-format` der Diagramme kennt kein Dezimalkomma.

22.6 Dokumente

- Das Skript für `toc::document` läuft zweimal; keine Nebenwirkungen. `toc::document` baut seinen eigenen Kontext und überschreibt `ctx` (Inhaltsverzeichnis).
- `toc::heading` setzt nicht fett und bricht nicht um.
- `-gap` und `-bookmarks` von `toc::document` gelten nur für diesen Aufruf; beim Weg von Hand zählt `toc::configure`.
- Ein Deckblatt vor dem Verzeichnis geht nur über `collect`, `pageCount`, `render`.
- Formularfelder mit den Schriften aus `fonts::init`: im CID-Modus Kästchen in Auswahllisten, sonst falsch gespeicherte Umlaute. Eine `cp1252`-Schrift anmelden und mit `form::configure` einstellen (Formular-Layout).
- `orderTable` und `sumLine` nehmen ohne `form::configure` immer Helvetica-Bold.
- Ein Kontrollkästchen in einem Formular mit `-calculate` oder `-format` ist in Poppler-Betrachtern unsichtbar; den Rahmen zeichnen.
- `labelField ... checkbox` ohne `-fieldw` ist so breit wie die Zeile.
- `labels::render` beginnt die Seiten selbst; kein `startPage` davor (Etikettenbögen).
- `labels::fitSize` hört bei `min` auf, auch wenn der Text nicht passt; nachmessen. Argumente anders als bei `text::fitSize`.
- Drucken in tatsächlicher Größe, nicht "an Seite anpassen".
- Ein JPEG, das vor dem ersten `startPage` geladen wird, beginnt eine Seite (Bilder).
- `image::gallery -caption` kennt nur `unten` und `keine`; andere Werte schalten die Unterschriften still ab.
- `roundedRect -clip 1` beschneidet eckig (Pfeile und Callouts).
- Ohne `fonts::init -cid 1` werden Formelzeichen zu? (Formeln).
- `renderLatex` setzt keinen Abstand um `=`; `\,` einfügen. Danach die Schrift neu setzen.
- Ein Skript mit Tk braucht `exit` am Ende und eine Anzeige (Treeview).
- Eine CID-Schrift, die mit `setFont` eingestellt, aber nie benutzt wird, landet ganz in der Datei: rund 330 KB für DejaVu Sans Bold.

22.7 Barrierefreiheit und Aufbau

- `tagged 1 -ua 1` setzt `DisplayDocTitle` nicht; PDF/UA verlangt es: `viewerPreferences -displaydoctitle 1` (Barrierefreie PDFs).
- `writeParagraph` setzt `P`, wenn kein Typ angegeben ist; für Überschriften `H1` usw. als achtes Argument. Es stellt immer die normale Schrift ein.
- Ein Diagramm ist für ein Vorleseprogramm nur sein Ersatztext. Wichtige Zahlen zusätzlich als Tabelle. (Bis pdf4tclchart 0.1.1 blieb das `Figure` ohne Inhalt, und pdf4tcl warnte.)
- Selbst Gezeichnetes in `tag::begin`/`end` oder als Artefakt, sonst ist `getUntaggedCount` nicht 0.
- Bausteine hinterlassen ihre Schrift und Farbe; `state::with` (Grafikzustand).
- `state::with` stellt wieder her, was man angibt, nicht was eingestellt war.
- `gsave`/`grestore` über einen Seitenwechsel ergibt unausgeglichene Ströme, ohne Meldung.
- `measure::block`: `pages` bleibt 1, wenn der Block nicht selbst umbricht; `overflow` lesen (Messen und stapeln).
- In Skripten für `layout::stack $bpdf` statt `$pdf`.
- `layout::stack` prüft nur Blöcke mit `keep 1`.
- `page::header` zieht seine Linie 2 pt unter `top`; Inhalt ab `top` wird durchgestrichen.

22.8 Seite

- pdf4tcl-Objekt ohne `-unit mm`: pdf4tcllib rechnet in Punkt, sonst landet Text außerhalb der Seite (Seitenkontext).
- pdf4tcl-Option `-margin` und `page::context` zusammen ergeben den doppelten Rand.
- `-orient` im Kontext und im pdf4tcl-Objekt müssen gleich sein.
- `page::context a4 20 0`: das zweite Wort ist die Richtung, nicht das Querformat.
- y ist die Grundlinie: Text bei `y = top` ragt in den Rand.
- `page::footer` schreibt "Seite N", auch in einem englischen Dokument; `pageLabelFormat` setzen.
- `page::grid` ist ein Hilfsraster zum Entwickeln, kein Spaltenraster.
- `page::centerText` ohne Schriftargument setzt Helvetica, auch nach `fonts::init`; die Schrift mitgeben.

Nächstes Kapitel: API je Modul

23 API je Modul

Kurzreferenz pdf4tcllib 0.6.7 und der Zusatzpakete. Die Aufrufformen sind aus den Prozeduren selbst ausgelesen (`info args`); Argumente mit?…? haben eine Vorgabe. Die Optionen sind eine Auswahl. Vollständig und auf Englisch: `docs/en/reference/` im Quellbaum von pdf4tcllib.

Alle Namen stehen im Namensraum `pdf4tcllib::`, der hier weggelassen ist: `fonts::init` heißt ausgeschrieben `pdf4tcllib::fonts::init`.

23.1 Pakete

Paket	Version	Namensraum	Kapitel
pdf4tcllib	0.6.7	fonts unicode text math page tag state measure layout table drawing image units form	2 bis 20
pdf4tclchart	0.1.2	chart	10
pdf4tclflow	0.1	flow	7
pdf4tcltoc	0.1.1	toc	11
pdf4tcllabels	0.3	labels	13
pdf4tclforms	0.2.3	forms	–
pdf4tclcanvas	0.1	canvas	–
pdf4tcltable	0.3.2	tablelist	–
pdf4tcltext	0.3	textwidget	–
pdf4tcltreeview	0.1.1	treeview	17

"–": im Buch kein eigenes Kapitel, nur die Aufrufe hier.

```
package require pdf4tcl
package require pdf4tcllib
puts [pdf4tcllib::version]          ;# 0.6.7
```

23.2 Schriften: fonts

```
fonts::init ?-cid 0|1? ?-fontdir dir? ?-family name? ?-force 0|1?
fonts::ready
fonts::hasTtf
fonts::hasTtfItalic
fonts::isCidMode
fonts::fontSans
fonts::fontSansBold
fonts::fontSansItalic
fonts::fontSansBoldItalic
fonts::fontMono
fonts::setFont pdf size ?family? ?style?
fonts::isMonospace fontName
fonts::widthFactor fontName
fonts::subset
fonts::inSubset codepoint
```

`init` einmal je Prozess; der CID-Modus gilt ab dem ersten Aufruf. `style` bei `setFont`: `Bold`, `Italic`, `BoldItalic`.

23.3 Zeichen: unicode

```
unicode::sanitize text ?-mono 0|1? ?-tabwidth n?
unicode::safeText pdf text ?-x x -y y -align a …?
unicode::readFile datei
unicode::preprocessBytes bytes
unicode::substCount
unicode::substReport                ;# dict: count, chars
unicode::resetSubst
```

23.4 Text setzen: text

```
text::width text size font ?pdf?
text::wrap text maxW size font ?code? ?pdf? ?hardBreak? ?trenner?
text::truncate text maxW size font ?pdf?
text::fits text maxW size font ?pdf?
text::fitSize text maxW wunschgroesse font ?-min n? ?-pdf pdf?
text::writeParagraph pdf text x y breite ?size? ?align? ?tag?
text::expandTabs zeile ?tabbreite?
text::detectFont zeile
text::superscript pdf text x y size font
```

```
text::subscript pdf text x y size font
text::mathSymbol name
text::mathSymbolNames
```

`writeParagraph` gibt das y unter dem Absatz zurück. `trenner` bei `wrap`: Vorgabe `{- /}`.

23.5 Formeln: math

```
math::renderFormula pdf x y formel ?-size n? ?-font f?
math::analyseFormula formel
math::renderLatex pdf x y latex ?-size n? ?-font f?
math::measureLatex pdf latex ?-size n?
              ?-font f?   ;# {breite oben unten}
math::pdfBackend pdf
math::canvasBackend canvas ?-family f? ?-fill farbe? ?-tags t?
math::renderLatexOn backend x y latex ?-size n? ?-font f?
math::measureLatexOn backend latex ?-size n? ?-font f?
```

23.6 Seite: page, units

```
page::context papier ?-margin mm? ?-orient 0|1? ?-landscape 0|1?
page::lineheight size ?faktor?
page::header pdf ctx text ?size?
page::footer pdf ctx text seite ?size? ?-pagelabel format?
page::number pdf ctx seite ?gesamt? ?size?
page::centerText pdf ctx text y ?size? ?font?
page::grid pdf ctx spalten spalte
page::debugGrid pdf ctx ?abstand?
page::orientationLegend pdf ctx

units::mm wert      units::to_mm pt
units::cm wert      units::to_cm pt
units::inch wert    units::to_inch pt
```

Schlüssel des Kontexts: `page_w page_h left top bottom text_w text_h margin`, dazu die Kurzformen `PW PH SX SY SW SH`. Vorgabe für `pageLabelFormat`: `Seite %s`.

23.7 Strukturbaum: tag

```
tag::begin pdf typ ?-alt t? ?-scope s? ?-colspan n? ?-rowspan n?
              ?-summary t?
tag::end pdf
tag::artifact pdf ?-type Layout|Pagination|Page?
tag::artifactEnd pdf
tag::isActive pdf
tag::depth pdf
tag::forget pdf
```

Ohne `$pdf tagged 1` tun alle nichts.

23.8 Zustand, Messen, Stapeln: state, measure, layout

```
state::save pdf ?size? ?font? ?fill? ?stroke?
state::restore pdf zustand
state::with pdf {size font} {r g b} skript

measure::block varName skript ?-paper p? ?-orient 0|1? ?-margin mm?
              ?-landscape 0|1? ?-y y?
measure::height varName skript ?option …?
measure::fits platz varName skript ?option …?

layout::stack pdf x y breite bloecke ?-ctx ctx? ?-gap n?
              ?-font {size name}? ?-fill {r g b}? ?-pagecmd cmd?
              ?-paper p?
```

`measure::block` liefert ein Dict: `height pages value overflow sheet`. Blöcke für `layout::stack`: `{script {…} ?keep 1?}` oder `{gap n}`; im Skript `$bpdf $bx $by $bw`.

23.9 Tabellen: table

```
table::draw pdf x y spalten zeilen ?option …?
table::simpleTable pdf x y breiten zeilen ?-zebra 0|1? ?-pad n?
              ?-header_bg {r g b}? ?-row_height n? ?-font_size n?
table::render pdf daten x0 yVar maxW yTop yBot pageNoVar
              pageW pageH margin size lineH ?debug? ?pageBreakCmd?
table::group zeilen ?-by n? ?-sum {…}? ?-label fmt? ?-total 0|1?
              ?-totallabel t? ?-header 0|1? ?-blank 0|1? ?-strict 0|1?
table::pivot zeilen ?-row n? ?-col n? ?-value n?
              ?-agg sum|count|avg|min|max? ?-empty t? ?-total 0|1?
              ?-rowlabel t? ?-totallabel t? ?-colorder {…}?
table::widths spalten zeilen ?-pdf pdf? ?-fontsize n? ?-pad n?
table::breakPoints zeilen ?-fontsize n? ?-rowheight n? ?-header 0|1?
              ?-groups {…}?
table::fitRows platz zeilen ?option …?
```

Optionen von `draw`: `-ctx -maxwidth -header -headerbg -headerfg -zebra`

`-zebracolor -zebrastart -fontsize -pad -border -rowheight -cellstyles -rowstyles -rowindent -footer -footerbg -footerbold -yvar -pagevar -orient -pagebreakcmd -fontreg -fontbold -summary`. Seit 0.6.7 ist jede andere Option ein Fehler.

Je Spalte: `-header -width (Zahl oder auto) -align -font -notruncate`. Eine Zelle darf ein Dict `{text … colspan n rowspan n}` sein.

23.10 Zeichnen: drawing

```
drawing::frame pdf x y w h ?linienbreite?
drawing::separator pdf x y w ?farbe? ?linienbreite?
drawing::roundedRect pdf x y w h r ?stroke? ?fill? ?-clip 1?
drawing::polygon pdf cx cy radius ecken ?stroke? ?fill?
drawing::star pdf cx cy radius ?zacken? ?verhaeltnis? ?stroke? ?fill?
drawing::gradient_v pdf x y w h farbe1 farbe2 ?schritte?
drawing::gradient_h pdf x y w h farbe1 farbe2 ?schritte?
drawing::interpolate farbe1 farbe2 t
drawing::arrow pdf x1 y1 x2 y2 ?-at end|start|both? ?-length n?
              ?-width n?
              ?-filled 0|1?
drawing::arrowHead pdf x y winkel ?-length n? ?-width n? ?-filled 0|1?
drawing::callout pdf x y w h zielX zielY text
              ?-radius -fill -stroke -tail
              -pad -fontsize -font -alt?
drawing::watermark pdf ctx text ?-angle grad? ?-size n?
              ?-color {r g b}?
              ?-layer 1|id?
drawing::textRotated pdf text x y winkel size ?font?
drawing::textScaled pdf text x y sx sy size ?font?
drawing::textSkewed pdf text x y skewX skewY size ?font?
```

`arrowHead` nimmt den Winkel im Bogenmaß.

23.11 Bilder: image

```
image::place pdf id x y w h ?-fit contain|cover|stretch? ?-anchor a?
              ?-dpi n? ?-alt t?
image::gallery pdf bilder x y w h ?-cols n? ?-rows n? ?-gap n?
              ?-caption unten|keine? ?-captionH n? ?-fontsize n?
              ?-font f?
              ?-fit …? ?-anchor …? ?-dpi n? ?-pagecmd cmd?
image::insert pdf tkBild x yVar maxW yTop yBot pageNoVar
              pageW pageH margin size ?debug?
image::insertAt pdf tkBild xPos yVar maxW yTop yBot pageNoVar
              pageW pageH margin size ?debug?
```

`place` gibt die belegte Fläche `{x y w h}` zurück, `gallery` die Zahl der Seiten. `insert` und

`insertAt` brauchen Tk.

23.12 Formular-Layout: form

```
form::configure ?-option wert …?
form::fieldHeight
form::rowHeight
form::section pdf ctx yVar titel
form::labelField pdf ctx yVar label typ ?-labelw n? ?-fieldw n?
              ?-fieldh n?
              ?addForm-Optionen …?
form::row pdf ctx yVar felder
form::separator pdf ctx yVar ?abstand?
form::orderTable pdf ctx yVar koepfe breiten ?daten? ?-emptyRows n?
              ?-rowh n? ?-headerBg {r g b}? ?-cellForm präfix?
              ?-cellOpts {…}?
form::sumLine pdf ctx yVar breiten label wert ?-id id?
              ?-calculate {op {felder}}?
              ?-init wert?
```

Alle schieben `yVar` weiter. Ein Feld in `form::row`: Dict mit `label type width` und wahlweise `id init options labelw fieldh readonly multiline`. Hintergrund und Rahmen der Felder kommen aus `fieldBg`, `fieldBorder` und `fieldBorderW` von `form::configure`, wenn der Aufruf keine eigenen angibt.

23.13 Diagramme: chart (pdf4tclchart)

```
chart::bar pdf x y w h daten ?option …?
chart::line pdf x y w h reihen ?option …?
chart::pie pdf x y w h daten ?option …?
chart::legend pdf x y paare optionen
chart::niceScale min max ?teilungen?        ;# {min max schritt}
chart::configure ?-option wert …?
```

Optionen: `-title -titlesize -labelsize -valuesize -values -format -color -colors -grid -gridcolor -axiscolor -min -max -legend -alt`. Daten: `{name wert name wert …}` oder `{{name wert} …}`; Reihen für `line`: `{{name {paare}} …}`. Rückgabe: das y unter dem Diagramm.

23.14 Spaltenfluss: flow (pdf4tclflow)

```
flow::columns pdf ctx text ?-columns n? ?-gap n? ?-size n? ?-font f?
              ?-tag typ? ?-top y? ?-bottom y? ?-firstx x? ?-firsty y?
              ?-newpage skript?
flow::boxes ctx ?option …?
flow::measure pdf text breite size font
flow::configure ?-option wert …?
```

Ohne `-newpage` gibt `columns` den Rest zurück, statt umzubrechen; `-newpage` ist ein Skript und läuft im Bereich des Aufrufers.

23.15 Inhaltsverzeichnis: toc (pdf4tcltoc)

```
toc::document pdf papier skript ?-title t? ?-titlesize n? ?-size n?
              ?-leader z? ?-indent n? ?-gap n? ?-bookmarks 0|1?
toc::heading pdf ctx yVar ebene text ?-size n? ?-gap n? ?-tag typ?
toc::collect skript ?versatz?
toc::pageCount ctx eintraege ?option …?
toc::render pdf ctx eintraege ?option …?
toc::entries
toc::configure ?-bookmarks 0|1? ?…?
```

`document` liefert ein Dict: `entries tocPages contentPages`. Ein Eintrag ist `{ebene titel seite}`.

23.16 Etiketten: labels (pdf4tcllabels)

```
labels::sheets
labels::sheet name
labels::define name spec
labels::place geometrie index                ;# {x y w h}
labels::render pdf name datensaetze {x y w h satz} skript
              ?-start n? ?-only {…}? ?-offsetx mm? ?-offsety mm?
              ?-frame 0|1|layer? ?-tag typ?
labels::fitSize pdf font text maxW size ?min?
labels::wrap pdf text maxW
labels::ellipsize pdf text maxW ?ende?
labels::calibration pdf name
```

`render` beginnt die Seiten selbst. Das vierte Argument nennt die fünf Variablen, die das Skript je Etikett bekommt.

23.17 Ausfüllbare Formulare: forms (pdf4tclforms)

```
forms::renderSchema pdf ctx spec ?-yvar name? ?-pagebreak 0|1?
forms::renderSection pdf ctx yVar abschnitt ?pagebreak?
forms::field pdf ctx yVar feld ?pagebreak?
forms::checkboxLine pdf ctx yVar feld ?pagebreak?
forms::radioGroup pdf ctx yVar feld ?pagebreak?
forms::buttonBar pdf ctx yVar feld ?pagebreak?
forms::signatureLine pdf ctx yVar feld ?pagebreak?
forms::entryTable pdf ctx yVar tabelle ?pagebreak?
forms::template name
              ?…?          ;# callnote inventory checklist order
forms::read datei ?-schema s? ?-strict 0|1? ?-raw 0|1?
forms::write ein aus werte ?-schema s? ?-strict 0|1?
```

23.18 Tk-Widgets: canvas, tablelist, textwidget, treeview

```
canvas::place pdf canvas x y w h ?-fit contain|stretch? ?-anchor a?
              ?-alt t?
              ?-caption t? ?-captionH n? ?-fontsize n? ?-font f?
              ?-bbox {…}? ?-fontmap {…}? ?-textscale f? ?-border 0|1?
canvas::fitPage pdf canvas ctx ?option …?

tablelist::render pdf tbl x y ?-maxwidth -fontsize -rowheight -zebra
              -border -tree -indentW -formatted -headerbg -headerfg
              -footer -footerbg -footerbold -ctx -yvar -font
              -boldfont?
tablelist::renderRange pdf tbl x y ?-firstrow n? ?-lastrow n? ?…?
tablelist::report                    ;# was beim letzten render fehlte

textwidget::render pdf textwidget x y ?-maxwidth n? ?-fontsize n?
              ?-fontfamily f? ?-linespacing n? ?-skipelided 0|1?
              ?-skipinternal 0|1? ?-ctx ctx? ?-yvar name?

treeview::render pdf tv x y ?-maxwidth -fontsize -rowheight -tree -all
              -zebra -border -indentW -ctx -yvar -pagevar -font
              -boldfont
              -summary?
treeview::outline pdf tv x y
              ?-fontsize -indentW -lineH -all -bookmarks
              -font -yvar?
```

Alle vier brauchen Tk und eine Anzeige.

23.19 Sonstiges

```
pdf4tcllib::version
pdf4tcllib::validate_pdf datei     ;# dict: valid size error
```

Nächstes Kapitel: Glossar

24 Glossar

Artefakt — Inhalt ohne Bedeutung für den Text (Linien, Kopfzeile, Seitenzahl); ein Vorleseprogramm überspringt ihn. `tag::artifact`.

CID-Modus — Schrift mit Identity-H-Kodierung; jedes Zeichen der TrueType-Datei ist verfügbar. Einschalten mit `fonts::init -cid 1`.

256-Zeichen-Modus — Vorgabe von `fonts::init`: ASCII, Latin-1 und 32 ausgewählte Zeichen in einer einfachen Kodierung.

Ersatztext — Beschreibung einer Abbildung für Leser, die sie nicht sehen; `-alt`.

Erscheinung — was ein Formularfeld auf der Seite zeigt, solange niemand hineinklickt. Mit `/NeedAppearances` baut der Betrachter sie neu auf.

Grundlinie — die Linie, auf der die Buchstaben stehen; `y` bei `$pdf text` bezeichnet sie.

Lesezeichen — Eintrag in der Gliederung am Rand des Betrachters; `toc::heading` legt je Überschrift eines an.

Punkt — Maßeinheit von PDF und pdf4tcl, 1/72 Zoll, etwa 0,353 mm.

Rahmen (Bild) — die Fläche, in die `image::place` ein Bild einpasst; `contain` lässt Rand frei, `cover` schneidet ab.

Satzspiegel — die Fläche innerhalb der Ränder, in der der Inhalt steht; `left`, `top`, `text_w`, `text_h` im Seitenkontext.

Seitenkontext — das Dict aus `page::context` mit allen Maßen der Seite.

Strukturbaum — die Gliederung eines barrierefreien PDFs in Überschriften, Absätze, Tabellen und Abbildungen; `$pdf tagged 1`.

Zugriffsprozedur — `fonts::fontSans` und Verwandte; liefern den Namen, unter dem `fonts::init` eine Schrift angemeldet hat.

Nächstes Kapitel: Fehlermeldungen

25 Fehlermeldungen

Die Meldungen von pdf4tcllib 0.6.7 und den Zusatzpaketen, nach Modul, mit der üblichen Ursache. Die Texte sind englisch und stehen hier so, wie Tcl sie ausgibt; **…** steht für den eingesetzten Wert.

Drei Arten gibt es:

- **Fehler** brechen den Aufruf ab (`catch` fängt sie).
- **Warnungen** landen in `::pdf4tcl::warnings`, einer Liste, die niemand von selbst leert. Vor dem Dokument `set ::pdf4tcl::warnings {}`, danach auslesen.
- **Hinweise auf stderr** schreibt pdf4tcllib selbst, meist nur einmal je Prozess.

25.1 Für alle Bausteine

25.1.1 …: unknown option -… (allowed: …) / (known: …)

Tippfehler oder eine Option, die es in dieser Version nicht gibt. Die Meldung nennt die erlaubten. Seit 0.6.7 gilt das auch für `table::draw`; vorher ging dort jede Option still durch.

25.1.2 …: expected an even number of option/value words

25.1.3 …: expected option/value pairs

Einer Option fehlt ihr Wert, oft weil eine Liste nicht geklammert ist: `-colors $a $b` statt `-colors [list $a $b]`.

25.2 Seite und Schriften

25.2.1 unknown paper size: … (known: …)

`page::context` kennt das Papier nicht. Groß- und Kleinschreibung spielt keine Rolle; die Liste in der Meldung nennt die bekannten Namen.

25.2.2 pdf4tcllib::page::footer: unknown option …

Bei `footer` steht die Größe an fünfter Stelle, die Optionen danach: `page::footer $pdf $ctx "Text" $seite 9 -pagelabel "Seite %s"`.

25.2.3 pdf4tcllib::fonts: No TTF fonts found, falling back to Helvetica

Hinweis auf stderr, danach die durchsuchten Ordner. Ab hier gilt der 256-Zeichen-Modus mit Helvetica; Umlaute gehen, →, ✓ und Anführungszeichen nicht. Unter Debian fehlt meist `fonts-dejavu-extra` (Einrichten und Schriften).

25.2.4 pdf4tcllib::fonts: TTF error: … / Fallback on Helvetica

Die Schriftdatei wurde gefunden, ließ sich aber nicht laden: beschädigt, keine TrueType-Datei, oder ein anderes Programm hat sie gesperrt.

25.2.5 pdf4tcllib::unicode: sanitize: at least one character is missing from the font and was replaced with '?' (first: U+…). …

Hinweis auf stderr, einmal je Prozess: Zeichen fehlten in der Schrift und wurden zu?. Welche, sagt `unicode::substReport`. Für Griechisch und Formelzeichen `fonts::init -cid 1` (Unicode sicher ausgeben).

25.2.6 pdf4tcllib::math: unknown symbol '\…' rendered as literal text

`math::renderLatex` kennt den Befehl nicht und setzt ihn als Text. Welche Namen es gibt, sagt `text::mathSymbolNames`.

25.3 Text, Messen, Stapeln

25.3.1 text::fitSize: unknown option …

`fitSize` nimmt `-min` und `-pdf`; `pdf` steht hier nicht an fester Stelle wie bei `text::wrap`.

25.3.2 layout::stack: an entry needs a script (or must be {gap })

Ein Block in der Liste hat weder `script` noch die Form `{gap n}`. Häufig: die Blöcke als ein Skript statt als Liste von Dicts übergeben.

25.3.3 measure::block: unknown option …

Erlaubt: `-paper -orient -margin -landscape -y`.

25.4 Tabellen

25.4.1 table::draw: -orient 0 is not supported. …

Das Dokument wurde mit `-orient 0` (y nach oben) angelegt. `table::draw` zeichnet nur bei y nach unten. Das pdf4tcl-Objekt mit `-orient 1` anlegen, der Vorgabe (Seitenkontext).

25.4.2 table::draw: rows … to … are joined by rowspan and need … pt, more than a page holds (… pt)

Ein Block aus Zeilen, die `rowspan` verbindet, wird nie getrennt. Hier passt er auf keine Seite. `rowspan` verkleinern oder die Gruppe in mehrere Blöcke teilen.

25.4.3 table::group: -by … is outside the row (0…..)

Die Gruppenspalte gibt es nicht; die Zählung beginnt bei 0.

25.4.4 table::group: the rows are not sorted by column …; a group falls into several blocks and each gets its own subtotal

Warnung in `::pdf4tcl::warnings`, mit `-strict 1` ein Fehler. Die Daten vorher nach der Gruppenspalte sortieren, etwa mit `lsort -index 0`. `group` sortiert absichtlich nicht.

25.4.5 table::pivot: -agg must be sum, count, avg, min or max, not "…"

25.4.6 orderTable: column widths total … pt, the text width is only … pt -- the table would be drawn past the edge of the page

`form::orderTable` passt nicht in den Satzspiegel. Breiten verkleinern oder einen kleineren Rand im Kontext.

25.4.7 orderTable: … header(s) but only … column width(s)

Eine Breite je Kopf.

25.4.8 tablelist::render: … could not be reproduced (…). …

Warnung von `pdf4tcltable`: Bilder, eingebettete Fenster oder umbrochene Zellen des Widgets kommen nicht ins PDF. Was genau, sagt `tablelist::report`.

25.5 Diagramme

25.5.1 chart::…: no data

25.5.2 chart::…: data must be label/value pairs or a list of {label value}, got … element(s)

Eine ungerade Liste: `{Jan 120 Feb}`. Leere Werte sind als Paar zu schreiben, z. B. `{Feb 0}`.

25.5.3 chart::bar: value for "…" is not a number: "…"

Gilt ebenso für `line` und `pie`. Ein leeres Feld ist keine Null (Diagramme).

25.5.4 chart::bar: box too small -- … x … pt leaves no room for axis and labels

Der Kasten ist kleiner als Achse und Beschriftung zusammen. Höhe vergrößern oder `-labelsize` kleiner.

25.5.5 chart::pie: negative value for "…" -- a slice of a whole cannot be negative

25.5.6 chart::pie: the values add up to 0

25.6 Spaltenfluss und Verzeichnis

25.6.1 flow::…: -columns must be at least 1

25.6.2 flow::boxes: … columns with a gap of … pt leave no width in … pt

Zu viele Spalten oder ein zu großes `-gap` für die Breite.

25.6.3 toc::heading: level must be 1..6, got "…"

25.6.4 toc::document: the contents was calculated as … page(s) and written as … -- every page number in it would be off by …

`pageCount` und `render` liefen mit verschiedenen Optionen, oder `toc::configure` wurde zwischendurch geändert. Das Verzeichnis wäre falsch nummeriert, darum bricht `document` ab (Inhaltsverzeichnis).

25.7 Bilder und Zeichnen

25.7.1 image::place: -fit must be contain, cover or stretch, not "…"

25.7.2 image::place: width and height must be positive

25.7.3 image::place: image … has no size

Die Kennung stammt nicht aus `$pdf addImage`, oder das Bild ist leer.

25.7.4 image::place: … at … dpi (…x… pixels on …x… points), below …

Warnung bei `-dpi`: Das Bild ist für die Fläche zu grob aufgelöst.

25.7.5 image::gallery: -cols and -rows must be at least 1

25.7.6 image::gallery: the frame is too small for …x… cells

Der Rahmen reicht für das Raster nicht, meist wegen `-gap` oder `-captionH`. Bis 0.6.6 kam stattdessen `invalid command name "2"`: Die Meldung führte die Spaltenzahl als Befehl aus.

25.7.7 pdf4tcllib::image needs Tk: …

`image::insert` und `insertAt` arbeiten mit Tk-Bildern. Für Dateien ohne Tk: `$pdf addImage` und `image::place`.

25.7.8 drawing::arrow: -at must be end, start or both, not "…"

25.7.9 drawing::callout: width and height must be positive

25.7.10 drawing::watermark: no text

25.7.11 canvas::place: no such widget: …

25.7.12 canvas::place: no room left for the drawing after the caption

`-captionH` ist so groß wie der Kasten.

25.7.13 canvas::fitPage: the context leaves no height

25.8 Etiketten

25.8.1 unknown label sheet "…"; known: …

Der Bogen ist nicht eingetragen. Eigene Bögen mit `labels::define` (Etikettenbögen).

25.8.2 label format …: missing "…"

`labels::define` ohne einen Pflichtschlüssel.

25.8.3 label format …: paper "roll" means one label per page, so cols and rows must both be 1

25.8.4 argSpec must name five variables: {x y w h record}

Das vierte Argument von `labels::render`.

25.8.5 -start must be between 0 and … for sheet …

25.8.6 -only: "…" is not a position on sheet … (0…..)

25.8.7 -only names … position(s) for … record(s)

Je Datensatz eine Position.

25.8.8 -frame: 0, 1 or layer, not "…"

25.8.9 -frame layer needs pdf4tcl 0.9.4.70 or newer: … ignores -print 0, the grid would print

25.8.10 -frame layer under -pdfa …: PDF/A forbids /AS, so -print 0 has no effect and the grid would print; use -frame 1 on a separate proof sheet

Das Hilfsraster liegt in einer Ebene, die nicht gedruckt werden soll. Ältere pdf4tcl und PDF/A können das nicht; das Raster käme aufs Etikett.

25.9 Ausfüllbare Formulare

25.9.1 forms::read: cannot read "…" / forms::write: cannot read "…"

Datei fehlt oder ist kein PDF mit Formular.

25.9.2 forms::write: … has no field(s): …

Warnung, mit `-strict 1` ein Fehler: Diese Namen gibt es im Formular nicht. Oft fehlt der volle

Name mit Punkt (`person.city`, nicht `city`).

25.9.3 forms::write: nothing to fill in

Das Dict mit den Werten ist leer, oder keiner der Namen kommt im Formular vor.

25.9.4 pdf4tclforms::renderSchema: spec needs 'sections' key

25.9.5 pdf4tclforms::renderSchema: 'sections' must be a dict (name -> section), not a list of sections

25.9.6 pdf4tclforms: field entry must be a dict, got: …

25.9.7 pdf4tclforms::template: unknown template '…' (callnote inventory checklist order)

25.10 Tk-Widgets

25.10.1 treeview::render: no such widget: … / treeview::outline: no such widget: …

Der Pfad stimmt nicht, oder das Widget ist schon zerstört.

25.11 Aus pdf4tcl

Zwei Warnungen von pdf4tcl selbst, die mit pdf4tcllib häufig sind:

25.11.1 tagged: the … element is empty -- no content, no child element and no annotation. …

Ein Strukturelement ohne Inhalt. Bis pdf4tclchart 0.1.1 löste jedes Diagramm sie aus (`Figure`); seit 0.1.2 nicht mehr. Bleibt sie, steht ein eigenes `tag::begin`/`tag::end` ohne Zeichnung dazwischen.

25.11.2 … must contain at least one …

Fehler beim Schließen eines Elements, etwa `Table` ohne `TR`. Kommt bei selbst gebauten Tabellen vor, nicht bei `table::draw` (Barrierefreie PDFs).

Zurück zum Anfang

26 Lizenz

```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen
Bedingungen 4.0 International (CC BY-SA 4.0)
Codebeispiele des Autors: MIT-Lizenz (unten)

---
```

Text

Dieses Buch ist unter der Creative-Commons-Lizenz
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0) lizenziert.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen
und weiterverbreiten sowie es remixen, verändern und darauf aufbauen,
auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben
  machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen
  vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material
  remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen
  Sie Ihre Beiträge nur unter derselben Lizenz wie das Original
  verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

Codebeispiele

Die Codebeispiele des Autors (in den Codeblöcken des Buchs und als
Skripte daneben) stehen zusätzlich unter der MIT-Lizenz. Übernommene
Codebeispiele und zitierte Texte aus anderen Projekten (Tcl/Tk,
pdf4tcl und andere) behalten die Lizenz ihres Originals; die Quelle ist
jeweils angegeben.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

```
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

A

Artefakt 84

C

chart::legend 50
chart::niceScale 51

D

drawing 69
drawing::arrow 69
drawing::arrowHead 69
drawing::callout 69
drawing::roundedRect 70
drawing::watermark 38

F

flow::boxes 36
form 56
form::labelField 56
form::orderTable 56
form::row 56
form::section 56
form::sumLine 57

I

image::gallery 65
image::place 65

L

labels::fitSize 64
labels::render 63
labels::sheet 62
layout::stack 89

M

math 73
math::measureLatex 74
math::renderFormula 76
math::renderLatex 74
measure 89
measure::block 89
measure::fits 89
measure::height 89

P

page::_advance 25
page::context 22
page::lineheight 26

PDF/UA 82
pdf4tcl 9
pdf4tclchart 48
pdf4tclflow 35
pdf4tcllabels 61
pdf4tcllib 9
pdf4tcltoc 51
pdf4tcltreeview 77

S

state::with 86
Strukturbaum 81

T

table::draw 39
table::fitRows 43
table::group 44
table::pivot 44
table::widths 43
tag 85
text 27
text::fitSize 30
text::truncate 30
text::writeParagraph 30
toc::collect 55
toc::document 52
toc::heading 52
treeview::outline 77
treeview::render 77

U

unicode::readFile 21
unicode::sanitize 17
units 22

V

veraPDF 86